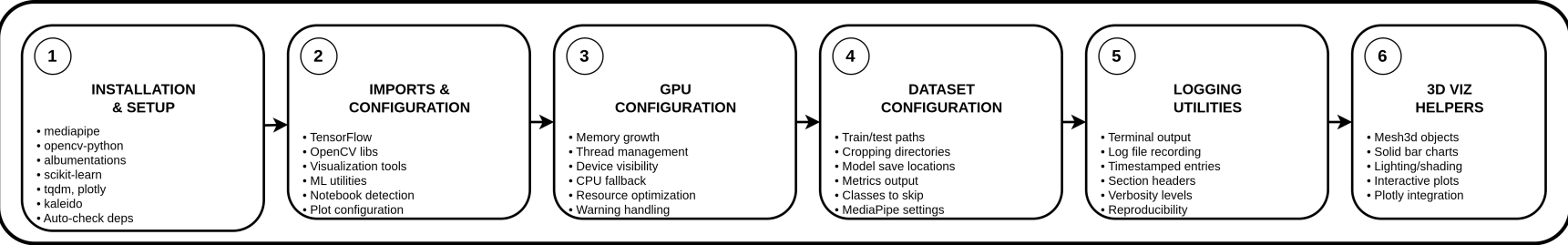
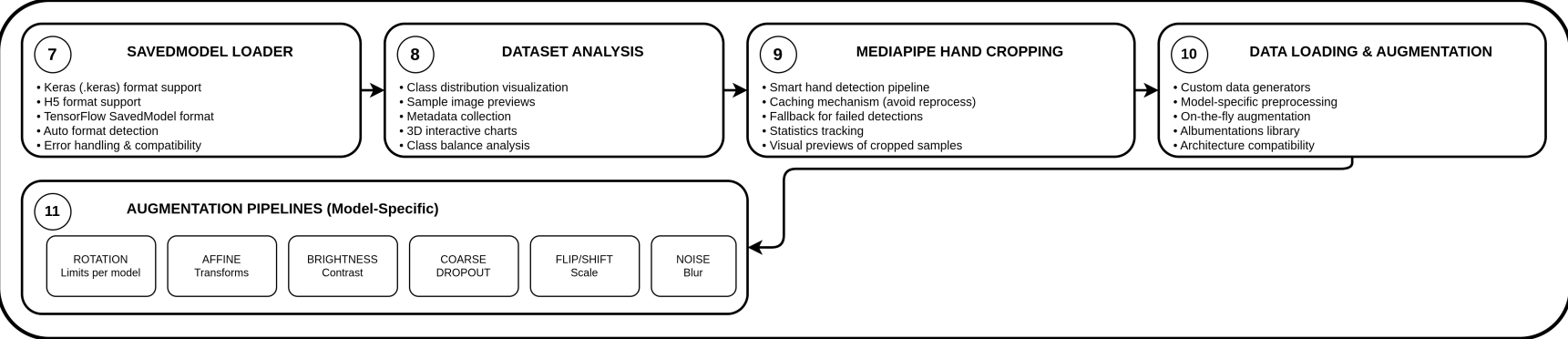


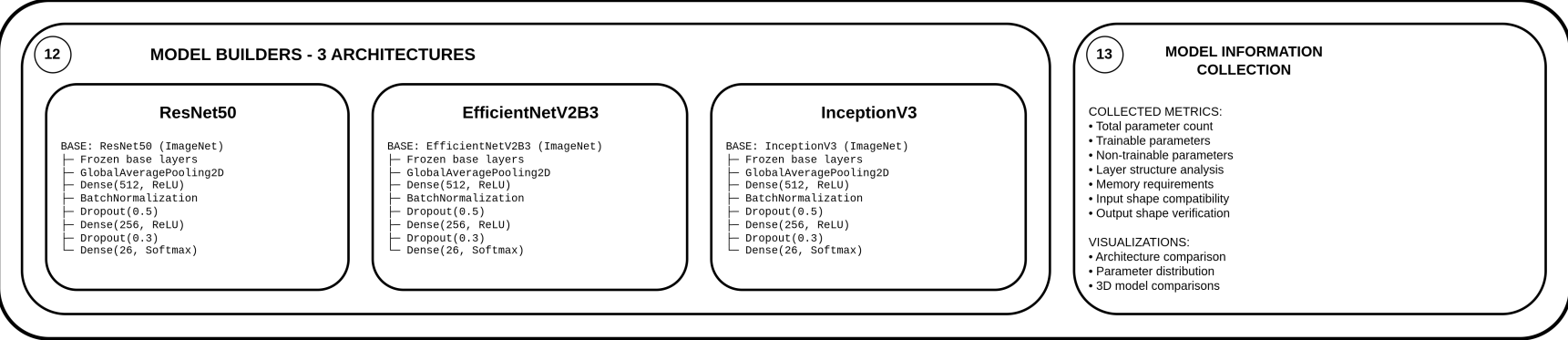
ENVIRONMENT SETUP



DATA PROCESSING & PREPARATION



MODEL ARCHITECTURE



1. ✓ Installation and Setup

Set up required Python packages including mediapipe, opencv-python, albumentations, scikit-learn, tqdm, plotly, and kaleido. Implement automatic installation checking to ensure all dependencies are available before running the main pipeline.

2. ✓ Imports and Configuration

Import all necessary libraries and set up configuration parameters. This includes TensorFlow, computer vision libraries, visualization tools, and machine learning utilities. Configure notebook environment detection for proper plotting behavior.

3. ✓ GPU Configuration

Configure TensorFlow to use GPU resources optimally. Set environment variables for memory growth, thread management, and device visibility. Include fallback behavior for CPU-only environments with appropriate warnings.

4. ✓ Dataset Configuration

Set up paths for training and testing datasets. Configure cropping directories, model saving locations, and metrics output paths. Define dataset parameters including classes to skip, image sampling strategies, and MediaPipe processing settings.

5. ✓ Logging Utilities

Implement comprehensive logging system that records all training activities to both terminal and log files. Include timestamped entries, section headers, and configurable verbosity levels. Ensure logs are saved for later analysis and reproducibility.

6. ✓ 3D Visualization Helpers

Create specialized functions for generating interactive 3D visualizations, particularly solid 3D bar charts using Mesh3d objects with proper lighting and shading. These visualizations provide intuitive understanding of complex data relationships.

7. ✓ SavedModel Loader

Develop robust model loading functionality that supports multiple formats including Keras (.keras), H5, and TensorFlow SavedModel formats. Implement format detection, error handling, and compatibility checking to ensure seamless model loading.

8. ✓ Dataset Analysis

Implement comprehensive dataset analysis with class distribution visualization, sample image previews, and metadata collection. Generate interactive 3D visualizations showing class balance and sample characteristics.

9. ✓ MediaPipe Hand Cropping

Create smart hand detection and cropping pipeline using MediaPipe. Implement caching mechanism to avoid reprocessing, fallback strategies for failed detections, and comprehensive statistics tracking. Include visual previews of cropped samples.

10. ✓ Data Loading and Augmentation

Build custom data generators with model-specific preprocessing pipelines. Implement on-the-fly data augmentation with Albumentations library, ensuring compatibility with different model architectures and requirements.

11. ✓ Augmentation Pipelines

Design model-specific augmentation strategies with appropriate transformations. Tailor rotation limits, affine transformations, brightness/contrast adjustments, and regularization techniques (like CoarseDropout) to each model's needs.

12. ✓ Model Builders

Implement three different model architectures: ResNet50, EfficientNetV2B3, and InceptionV3. Each model should include proper fine-tuning strategies with frozen base layers and custom classification heads. Include regularization techniques like dropout and batch normalization.

13. ✓ Model Information Collection

Gather comprehensive model information before training, including parameter counts, layer structures, memory requirements, and compatibility checks. Generate interactive visualizations comparing model architectures and capabilities.

14. ✓ Training Pipeline

Implement intelligent training system with multiple modes: scratch training, resume training, and fine-tuning. Include detailed metrics callbacks, model checkpointing, early stopping, and learning rate scheduling. Support GPU-accelerated training with minimal CPU interference.

15. ✓ Evaluation and Visualization

Create comprehensive evaluation system with multiple metrics including accuracy, precision, recall, F1 scores, Cohen's Kappa, and Matthews correlation coefficient. Generate interactive visualizations including confusion matrices, ROC curves, precision-recall curves, and confidence analysis.

16. ✓ Final Comparison and Main Pipeline

Implement final model comparison with interactive 3D visualizations. The main pipeline should orchestrate all components in a logical flow: dataset analysis → preprocessing → model analysis → baseline testing → intelligent training decisions → training → evaluation → comparison. Include comprehensive reporting and file organization.

17. ✓ Intelligent Training Decisions

Develop decision-making system that analyzes baseline model performance and determines optimal training approach for each model:

- SKIP if accuracy $\geq 98\%$ (excellent performance)
- FINE-TUNE if accuracy $\geq 95\%$ (good performance)
- RESUME if accuracy $\geq 30\%$ (moderate performance)
- SCRATCH if accuracy $< 30\%$ (incompatible/needs retraining)

18. ✓ Memory Management

Implement thorough memory cleanup procedures after each major component. Include GPU memory release, TensorFlow backend clearing, and garbage collection to prevent resource leaks during long training sessions.

19. ✓ Comprehensive Reporting

Generate detailed reports for all stages including dataset statistics, model information, training metrics, evaluation results, and comparisons. Save all reports in organized directory structure with both human-readable and machine-readable formats.

20. ✓ Visual Documentation

Create interactive visual documentation for all major components using Plotly. Include 3D visualizations for class distributions, model comparisons, training progress, feature embeddings, and performance metrics to enhance understanding and analysis.

1. ✓ Installation and Setup

```
# INSTALLATION CHECK
import subprocess
import sys

def install_if_missing(package, pip_name=None):
    """Install package if not available"""
    try:
        __import__(package)
    except ImportError:
        pip_name = pip_name or package
        print(f"Installing {pip_name}...")
        subprocess.check_call([sys.executable, '-m', 'pip', 'install', pip_name])

# Check and install dependencies
install_if_missing('mediapipe')
install_if_missing('cv2', 'opencv-python')
install_if_missing('albumentations')
install_if_missing('sklearn', 'scikit-learn')
install_if_missing('tqdm')
install_if_missing('plotly')
install_if_missing('kaleido') # For saving plotly figures
```

2. ✓ Imports and Configuration

```
# IMPORTS
import os
import shutil
import numpy as np
import pandas as pd
import matplotlib
# Check if running in notebook (Kaggle/Jupyter) - use inline display
import sys
IN_NOTEBOOK = 'ipykernel' in sys.modules or 'IPython' in sys.modules
if not IN_NOTEBOOK:
    matplotlib.use('Agg') # Non-interactive backend only if not in notebook
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import seaborn as sns
from pathlib import Path
import cv2
import mediapipe as mp
from tqdm import tqdm
import json
# Plotly for interactive 3D visualizations
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots
import plotly.io as pio
# Set Plotly renderer for Kaggle/Jupyter notebooks
if 'ipykernel' in sys.modules or 'IPython' in sys.modules:
    pio.renderers.default = 'notebook' # Best for Kaggle
# For inline display in Kaggle notebooks
if IN_NOTEBOOK:
    try:
        from IPython.display import display, Image, HTML, clear_output
        from IPython import get_ipython
        get_ipython().run_line_magic('matplotlib', 'inline')
        print(" Notebook detected - plots will display inline!")
    except:
        pass
from collections import Counter
from datetime import datetime
import gc
import time
import warnings
warnings.filterwarnings('ignore')
# TensorFlow imports with GPU configuration
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, models
from tensorflow.keras.applications import ResNet50, EfficientNetV2B3, InceptionV3
from tensorflow.keras.applications.resnet50 import preprocess_input as resnet_preprocess
from tensorflow.keras.applications.efficientnet_v2 import preprocess_input as
effnet_preprocess
from tensorflow.keras.applications.inception_v3 import preprocess_input as
inception_preprocess
from tensorflow.keras.callbacks import ModelCheckpoint, EarlyStopping, ReduceLROnPlateau
import albumentations as A
# Sklearn imports
from sklearn.model_selection import train_test_split
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
from sklearn.metrics import (
    accuracy_score, classification_report, confusion_matrix,
    precision_score, recall_score, f1_score, top_k_accuracy_score,
    balanced_accuracy_score, cohen_kappa_score, matthews_corrcoef,
    roc_auc_score, log_loss, roc_curve, auc, precision_recall_curve,
    average_precision_score
)
from sklearn.preprocessing import label_binarize
# Set seeds for reproducibility
np.random.seed(42)
tf.random.set_seed(42)
```

3. ✓ GPU Configuration

```
# GPU CONFIGURATION
def configure_gpu():
    """Configure GPU for optimal performance, minimize CPU usage"""
    print("\n" + "=" * 70)
    print("GPU CONFIGURATION")
    print("=" * 70)

    # Force single GPU usage
    os.environ["CUDA_VISIBLE_DEVICES"] = "0"
    os.environ["TF_FORCE_GPU_ALLOW_GROWTH"] = "true"

    # Optimize TensorFlow for GPU
    os.environ["TF_GPU_THREAD_MODE"] = "gpu_private"
    os.environ["TF_GPU_THREAD_COUNT"] = "2"

    # Reduce CPU parallelism to minimize CPU usage
    tf.config.threading.set_inter_op_parallelism_threads(2)
    tf.config.threading.set_intra_op_parallelism_threads(2)

    gpus = tf.config.list_physical_devices('GPU')
    if gpus:
        try:
            # Use only first GPU
            tf.config.set_visible_devices(gpus[0], 'GPU')
            # Enable memory growth to avoid OOM
            tf.config.experimental.set_memory_growth(gpus[0], True)
            print(f" GPU detected: {gpus[0].name}")
            print(f" Memory growth enabled")
            print(f" Using GPU 0 only")

            # Get GPU details
            try:
                gpu_details = tf.config.experimental.get_device_details(gpus[0])
                if gpu_details:
                    print(f" GPU Details: {gpu_details}")
            except:
                pass
            return True
        except RuntimeError as e:
            print(f" GPU configuration error: {e}")
            return False
    else:
        print(" No GPU detected - using CPU (training will be slower)")
        return False

# Configure GPU at import time
HAS_GPU = configure_gpu()
print(f"\n TensorFlow Version: {tf.__version__}")
print(f" GPU Available: {HAS_GPU}")
print(f" Physical Devices: {tf.config.list_physical_devices()}")
```


4. ✓ Dataset Configuration

```
# CONFIGURATION
CONFIG = {
    # ===== PATHS (KAGGLE ENVIRONMENT) =====
    # Dataset paths for Kaggle
    'train_path': '/kaggle/input/aslamerican-sign-language-alphabet-
dataset/ASL_Alphabet_Dataset/asl_alphabet_train',
    'test_path': '/kaggle/input/aslamerican-sign-language-alphabet-
dataset/ASL_Alphabet_Dataset/asl_alphabet_test',
    'cropped_train_path': '/kaggle/working/cropped_train', # MediaPipe cropped training
images
    'cropped_test_path': '/kaggle/working/cropped_test', # MediaPipe cropped test images
    'models_path': '/kaggle/working/models', # Saved models
    'metrics_path': '/kaggle/working/metrics', # Saved metrics and graphs

    # ===== MODEL PATHS FOR RESUME =====
    # Set these to your existing model paths for resume training
    # If None or file doesn't exist, will train from scratch
    'existing_models': {
        'ResNet50': '/kaggle/input/resnet50-best-1-keras/keras/1/2',
        'InceptionV3': '/kaggle/input/inceptionv3-best-
keras/keras/1/1/InceptionV3_best.keras',
        'EfficientNetV2B3': '/kaggle/input/efficientnetv2b3-best-1-
keras/keras/1/1/EfficientNetV2B3_best.keras'
    },

    # ===== DATASET CONFIG =====
    'skip_folders': ['del', 'nothing', 'space'],
    'skip_test_files': ['nothing_test.jpg', 'del_test.jpg', 'space_test.jpg'],
    'images_per_class': 1000, # Set to None for ALL images
    'test_images_per_class_from_train': 100,
    'require_mediapipe_detection': True,
    'use_cached_crops': True, # Skip MediaPipe if crops exist

    # ===== TRAINING CONFIG =====
    'img_size': (224, 224),
    'batch_size': 32,
    'epochs': 35,
    'val_split': 0.15,
    'resume_epochs': 15, # Additional epochs when resuming
    'fine_tune_epochs': 5, # Few epochs for fine-tuning already-good models

    # ===== MODELS TO TRAIN =====
    # Only these models will be trained (in order)
    # Include ALL models you want to train - they will be trained in this order
    'train_models': ['EfficientNetV2B3', 'ResNet50', 'InceptionV3'],

    # ===== MEDIAPIPE CONFIG =====
    'mediapipe_confidence': 0.6,
    'mediapipe_margin': 30,
    'mediapipe_model_complexity': 1, # 0=lite, 1=full
}

# Create directories
for path_key in ['cropped_train_path', 'cropped_test_path', 'models_path', 'metrics_path']:
    os.makedirs(CONFIG[path_key], exist_ok=True)
```


5. ✓ Logging and Visualization Helpers

```
# LOGGING UTILITIES
class Logger:
    """Simple logger that prints to terminal and saves to file"""
    def __init__(self, log_file=None):
        self.log_file = log_file or os.path.join(CONFIG['metrics_path'],
'training_log.txt')
        self.start_time = datetime.now()
        # Clear previous log
        with open(self.log_file, 'w') as f:
            f.write(f"ASL Training Log - {self.start_time.strftime('%Y-%m-%d
%H:%M:%S')}\n")
            f.write("=" * 70 + "\n")

    def log(self, message, also_print=True):
        """Log message to file and optionally print"""
        timestamp = datetime.now().strftime('%H:%M:%S')
        formatted = f"[{timestamp}] {message}"
        if also_print:
            print(message)
        with open(self.log_file, 'a') as f:
            f.write(formatted + "\n")

    def section(self, title):
        """Print section header"""
        header = "\n" + "=" * 70 + f"\n{title}\n" + "=" * 70 + "\n"
        self.log(header)

logger = Logger()
```


6. ✓ 3D Visualization Helpers

```
# 3D BAR HELPER FUNCTION
def create_3d_bar_mesh(x_center, y_center, z_height, bar_width=0.4, bar_depth=0.4,
                       color='blue', opacity=1.0, name='', hovertemplate=''):
    """
    Create a solid 3D bar using Mesh3d with proper triangular faces.
    This creates a proper rectangular prism (cuboid) with all 6 faces rendered correctly.
    Each face is made of 2 triangles = 12 triangles total for a closed solid bar.
    Args:
        x_center: X position of bar center
        y_center: Y position of bar center
        z_height: Height of bar (from 0)
        bar_width: Width in X direction
        bar_depth: Depth in Y direction
        color: Bar color
        opacity: Bar opacity (0-1)
        name: Name for legend
        hovertemplate: Hover text template
    Returns:
        go.Mesh3d trace for the bar
    """
    # Half dimensions
    hw = bar_width / 2 # half width
    hd = bar_depth / 2 # half depth

    # 8 vertices of the cuboid (bar)
    # Bottom face (z=0): vertices 0,1,2,3
    # Top face (z=height): vertices 4,5,6,7
    vertices_x = [
        x_center - hw, x_center + hw, x_center + hw, x_center - hw, # bottom: 0,1,2,3
        x_center - hw, x_center + hw, x_center + hw, x_center - hw # top: 4,5,6,7
    ]
    vertices_y = [
        y_center - hd, y_center - hd, y_center + hd, y_center + hd, # bottom
        y_center - hd, y_center - hd, y_center + hd, y_center + hd # top
    ]
    vertices_z = [
        0, 0, 0, 0, # bottom face at z=0
        z_height, z_height, z_height, z_height # top face at z=height
    ]

    # 12 triangular faces (2 per face × 6 faces)
    # Each face needs 2 triangles defined by vertex indices (i, j, k)
    # The vertices must be in counter-clockwise order when viewed from outside

    # Bottom face (z=0): vertices 0,1,2,3 - looking from below
    # Top face (z=h): vertices 4,5,6,7 - looking from above
    # Front face (y=-): vertices 0,1,5,4
    # Back face (y=+): vertices 2,3,7,6
    # Left face (x=-): vertices 0,3,7,4
    # Right face (x=+): vertices 1,2,6,5

    i_faces = [
        0, 0, # bottom: triangles (0,1,2) and (0,2,3)
        4, 4, # top: triangles (4,6,5) and (4,7,6)
        0, 0, # front: triangles (0,5,1) and (0,4,5)
        2, 2, # back: triangles (2,7,3) and (2,6,7)
        0, 0, # left: triangles (0,3,7) and (0,7,4)
        1, 1 # right: triangles (1,5,6) and (1,6,2)
    ]

    j_faces = [
        1, 2, # bottom
        6, 7, # top
        5, 4, # front
        7, 6, # back
        3, 7, # left
        5, 6 # right
    ]

    k_faces = [
        2, 3, # bottom
        5, 6, # top
    ]
```

```

        1, 5, # front
        3, 7, # back
        7, 4, # left
        6, 2 # right
    ]

    return go.Mesh3d(
        x=vertices_x,
        y=vertices_y,
        z=vertices_z,
        i=i_faces,
        j=j_faces,
        k=k_faces,
        color=color,
        opacity=opacity,
        name=name,
        showlegend=False,
        hovertemplate=hovertemplate,
        flatshading=True, # Better solid appearance
        lighting=dict(
            ambient=0.7,
            diffuse=0.8,
            specular=0.2,
            roughness=0.5,
            fresnel=0.1
        ),
        lightposition=dict(x=100, y=200, z=300)
    )

```

7. ✓ SavedModel Loader

```
# SAVEDMODEL LOADER
def load_model_any_format(model_path, model_name=None):
    """
    Load a model from any format (Keras 3 .keras, H5, or TensorFlow SavedModel).
    Args:
        model_path: Path to the model file/directory
        model_name: Optional model name for logging
    Returns:
        (model, format_type) tuple or (None, error_message)
    """
    name = model_name or os.path.basename(model_path)

    # Helper function to find model files in a directory
    def find_model_file(base_path):
        """Search for model files in directory tree"""
        model_extensions = ['.keras', '.h5', '.hdf5']
        savedmodel_markers = ['saved_model.pb', 'saved_model.pbtxt']

        if not os.path.exists(base_path):
            return None, None

        # If it's a file, return it directly
        if os.path.isfile(base_path):
            return base_path, 'file'

        # Check if this directory is a SavedModel
        for marker in savedmodel_markers:
            if os.path.exists(os.path.join(base_path, marker)):
                return base_path, 'savedmodel'

        # Search for model files recursively (max depth 3)
        for root, dirs, files in os.walk(base_path):
            depth = root[len(base_path):].count(os.sep)
            if depth > 3:
                continue

            # Check for SavedModel markers in subdirs
            for marker in savedmodel_markers:
                if marker in files:
                    return root, 'savedmodel'

            # Check for keras/h5 files
            for f in files:
                for ext in model_extensions:
                    if f.endswith(ext):
                        return os.path.join(root, f), 'file'

        return None, None

    # First, try to find the actual model file/directory
    actual_path, path_type = find_model_file(model_path)
    if actual_path and actual_path != model_path:
        logger.log(f" Found model at: {actual_path}")
        model_path = actual_path

    # Log directory contents for debugging
    if os.path.isdir(model_path):
        try:
            contents = os.listdir(model_path)
            logger.log(f" Directory contents: {contents[:10]}{'...' if len(contents) > 10}
        except:
            pass
    else:
        pass

    # Try standard Keras load first (.keras or .h5)
    try:
        model = keras.models.load_model(model_path)
        logger.log(f" Loaded {name} using keras.models.load_model()")
        return model, 'keras'
    except Exception as e1:
        keras_error = str(e1)

    # If it's a directory, try multiple approaches for SavedModel format
```

```

if os.path.isdir(model_path):
    # Check for saved_model.pb first
    has_savedmodel_pb = os.path.exists(os.path.join(model_path, 'saved_model.pb'))
    has_savedmodel_pbtxt = os.path.exists(os.path.join(model_path,
'saved_model.pbtxt'))

    if not has_savedmodel_pb and not has_savedmodel_pbtxt:
        # Not a valid SavedModel directory - search subdirectories
        logger.log(f" No saved_model.pb found, searching subdirectories...")
        for subdir in os.listdir(model_path):
            subpath = os.path.join(model_path, subdir)
            if os.path.isdir(subpath):
                if os.path.exists(os.path.join(subpath, 'saved_model.pb')):
                    logger.log(f" Found SavedModel in: {subpath}")
                    model_path = subpath
                    has_savedmodel_pb = True
                    break

        # Check for .keras files
        for f in os.listdir(subpath):
            if f.endswith('.keras') or f.endswith('.h5'):
                keras_file = os.path.join(subpath, f)
                logger.log(f" Found Keras file: {keras_file}")
                try:
                    model = keras.models.load_model(keras_file)
                    logger.log(f" Loaded {name} from {keras_file}")
                    return model, 'keras'
                except Exception as e:
                    logger.log(f" Failed to load {keras_file}: {e}")

# Approach 1: Try tf.saved_model.load (TF2 native)
if has_savedmodel_pb or has_savedmodel_pbtxt:
    try:
        logger.log(f" Trying tf.saved_model.load()...")
        imported = tf.saved_model.load(model_path)

        # Check if it has a keras model signature
        if hasattr(imported, 'signatures'):
            signatures = list(imported.signatures.keys())
            logger.log(f" Found signatures: {signatures}")

            # Get the serving function
            if 'serving_default' in signatures:
                serve_fn = imported.signatures['serving_default']

            # Create a wrapper model for inference
            class SavedModelWrapper(keras.Model):
                def __init__(self, serve_fn, **kwargs):
                    super().__init__(**kwargs)
                    self._serve_fn = serve_fn
                    # Try to get output shape from signature
                    output_info = list(serve_fn.structured_outputs.values())[0]
                    self._output_classes = output_info.shape[-1]

                def call(self, inputs):
                    # The serve function expects a dict with input tensors
                    result = self._serve_fn(inputs)
                    # Return the first output value
                    return list(result.values())[0]

            @property
            def output_shape(self):
                return (None, self._output_classes)

            model = SavedModelWrapper(serve_fn, name=f"{name}_TF2")
            # Build the model with sample input
            model.build(input_shape=(None, 224, 224, 3))
            logger.log(f" Loaded {name} using tf.saved_model.load()")
            logger.log(f" Note: This is inference-only - cannot be fine-tuned")
            return model, 'savedmodel'
    except Exception as e2:
        logger.log(f" tf.saved_model.load failed: {e2}")

# Neither worked
return None, f"Cannot load: {keras_error}"

```

8. ✓ Dataset Analysis

```
# DATASET ANALYSIS
def analyze_dataset():
    """Comprehensive dataset analysis with metrics"""
    logger.section("DATASET ANALYSIS")
    metrics = {
        'total_classes': 0,
        'total_train_images': 0,
        'total_test_images': 0,
        'class_distribution': {},
        'image_sizes': [],
        'analysis_time': None
    }
    start_time = time.time()

    # Check if paths exist
    if not os.path.exists(CONFIG['train_path']):
        logger.log(f" Training path not found: {CONFIG['train_path']}")
        logger.log("Please update CONFIG['train_path'] to point to your dataset")
        return metrics

    # Get all class folders
    all_folders = [d for d in os.listdir(CONFIG['train_path'])
                    if os.path.isdir(os.path.join(CONFIG['train_path'], d))]
    class_names = sorted([c for c in all_folders if c not in CONFIG['skip_folders']])
    metrics['total_classes'] = len(class_names)
    metrics['class_names'] = class_names

    logger.log(f"\n Dataset Path: {CONFIG['train_path']}")
    logger.log(f" Total folders found: {len(all_folders)}")
    logger.log(f" Skipped folders: {CONFIG['skip_folders']}")
    logger.log(f" Valid classes: {len(class_names)}")
    logger.log(f" Classes: {class_names}\n")

    # Analyze each class
    logger.log("Class Distribution:")
    logger.log("-" * 50)
    total_images = 0
    class_counts = {}
    for class_name in class_names:
        class_dir = os.path.join(CONFIG['train_path'], class_name)
        images = [f for f in os.listdir(class_dir)
                  if f.lower().endswith(('.jpg', '.jpeg', '.png'))]
        count = len(images)
        class_counts[class_name] = count
        total_images += count

    # Sample image size
    if images and len(metrics['image_sizes']) < 10:
        sample_path = os.path.join(class_dir, images[0])
        img = cv2.imread(sample_path)
        if img is not None:
            metrics['image_sizes'].append(img.shape[:2])
        logger.log(f" {class_name}: {count:}, {images}")

    metrics['total_train_images'] = total_images
    metrics['class_distribution'] = class_counts

    logger.log("-" * 50)
    logger.log(f"\n Total Training Images: {total_images:,}")
    avg_per_class = total_images // len(class_names) if class_names else 0
    logger.log(f" Average per class: {avg_per_class:,}")

    if class_counts:
        min_count = min(class_counts.values())
        max_count = max(class_counts.values())
        logger.log(f" Min per class: {min_count:,}")
        logger.log(f" Max per class: {max_count:,}")

    # Calculate average image size
    if metrics['image_sizes']:
        avg_h = sum(s[0] for s in metrics['image_sizes']) // len(metrics['image_sizes'])
        avg_w = sum(s[1] for s in metrics['image_sizes']) // len(metrics['image_sizes'])
```

```

        logger.log(f" Average image size: {avg_h}x{avg_w}")

# Test set analysis
if os.path.exists(CONFIG['test_path']):
    test_files = [f for f in os.listdir(CONFIG['test_path'])
                  if f.lower().endswith('.jpg') and '_test' in f]
    valid_test = [f for f in test_files if f not in CONFIG['skip_test_files']]
    metrics['total_test_images'] = len(valid_test)
    logger.log(f"\n Test images: {len(valid_test)}")

# Sampling info
if CONFIG['images_per_class']:
    expected_total = CONFIG['images_per_class'] * len(class_names)
    logger.log(f"\n Sampling: {CONFIG['images_per_class']} images per class")
    logger.log(f" Expected total: {expected_total:,} images")
else:
    logger.log(f"\n Using ALL images (~{total_images:,})")

metrics['analysis_time'] = time.time() - start_time
analysis_time = f"{metrics['analysis_time']:.2f}"
logger.log(f"\n Analysis time: {analysis_time}s")

# Save metrics
metrics_file = os.path.join(CONFIG['metrics_path'], 'dataset_metrics.json')
with open(metrics_file, 'w') as f:
    # Convert to JSON-serializable
    save_metrics = {k: v for k, v in metrics.items() if k != 'image_sizes'}
    json.dump(save_metrics, f, indent=2)
logger.log(f"\n Metrics saved to: {metrics_file}")

# Visualize class distribution (single combined visualization)
visualize_class_distribution(class_counts)

# Show sample images from each class (skip 3D distribution - redundant)
visualize_dataset_samples(class_names)

return metrics

def visualize_class_distribution(class_counts):
    """Create INTERACTIVE 3D class distribution visualization with SOLID bars"""
    logger.log("\n Creating interactive 3D class distribution...")
    classes = list(class_counts.keys())
    counts = list(class_counts.values())
    n_classes = len(classes)
    max_count = max(counts) if counts else 1

    # Create 3D bar chart using solid Mesh3d bars
    fig = go.Figure()

    for i, (cls, count) in enumerate(zip(classes, counts)):
        norm = count / max_count if max_count > 0 else 0

        # Color gradient based on count (blue to purple spectrum)
        r = int(50 + 150 * norm)
        g = int(100 + 50 * (1 - norm))
        b = int(200 - 50 * norm)
        color = f'rgb({r}, {g}, {b})'

        # Add solid 3D bar using helper function
        bar = create_3d_bar_mesh(
            x_center=i,
            y_center=0,
            z_height=count,
            bar_width=0.7,
            bar_depth=0.7,
            color=color,
            opacity=0.95,
            name=cls,
            hovertemplate=f'<b>Class {cls}</b><br>Images: {count:,}<extra></extra>'
        )
        fig.add_trace(bar)

    # Add text labels at top of bars
    fig.add_trace(go.Scatter3d(
        x=list(range(n_classes)),

```



```

y=[0] * n_classes,
z=[c + max_count * 0.03 for c in counts],
mode='text',
text=classes,
textfont=dict(size=12, color='black', family='Arial Black'),
hoverinfo='skip',
showlegend=False
))

fig.update_layout(
    title=dict(
        text='ASL Dataset - 3D Class Distribution (Rotate to explore!)',
        font=dict(size=20, color='#333'),
        x=0.5
    ),
    scene=dict(
        xaxis=dict(
            title='Class',
            tickvals=list(range(n_classes)),
            ticktext=classes,
            tickangle=45,
            gridcolor='lightgray'
        ),
        yaxis=dict(
            title='',
            showticklabels=False,
            range=[-1, 1],
            showgrid=False
        ),
        zaxis=dict(
            title='Number of Images',
            gridcolor='lightgray'
        ),
        camera=dict(eye=dict(x=1.5, y=1.5, z=0.8)),
        aspectmode='manual',
        aspectratio=dict(x=2, y=0.5, z=1),
        bgcolor='rgba(250,250,250,0.9)'
    ),
    height=650,
    template='plotly_white',
    margin=dict(l=0, r=0, t=60, b=0)
)

save_path = os.path.join(CONFIG['metrics_path'], 'class_distribution.png')
save_and_show_plotly(fig, save_path, '3D Class Distribution')

def visualize_dataset_samples(class_names):
    """Visualize sample images from each class with interactive Plotly grid"""
    logger.log("\n Creating interactive dataset sample visualization...")
    n_classes = len(class_names)
    n_cols = min(6, n_classes)
    n_rows = (n_classes + n_cols - 1) // n_cols

    # Create Plotly subplots
    fig = make_subplots(
        rows=n_rows, cols=n_cols,
        subplot_titles=[f'Class: {c}' for c in class_names],
        horizontal_spacing=0.02,
        vertical_spacing=0.08
    )

    for idx, class_name in enumerate(class_names):
        row = idx // n_cols + 1
        col = idx % n_cols + 1
        class_dir = os.path.join(CONFIG['train_path'], class_name)
        if os.path.exists(class_dir):
            images = [f for f in os.listdir(class_dir)
                      if f.lower().endswith(('.jpg', '.jpeg', '.png'))]
            if images:
                img_path = os.path.join(class_dir, images[0])
                img = cv2.imread(img_path)
                if img is not None:
                    img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
                    fig.add_trace(
                        go.Image(z=img_rgb, hovertemplate=f'<b>Class: {class_name}</b>

```



```
<extra></extra>'),
        row=row, col=col
    )

fig.update_layout(
    title=dict(
        text=' Sample Images from Each ASL Class (Click to zoom!)',
        font=dict(size=20, color='#333'),
        x=0.5
    ),
    height=200 * n_rows + 100,
    showlegend=False,
    template='plotly_white'
)

# Hide axes
fig.update_xaxes(showticklabels=False, showgrid=False)
fig.update_yaxes(showticklabels=False, showgrid=False)

save_path = os.path.join(CONFIG['metrics_path'], 'dataset_samples.png')
save_and_show_plotly(fig, save_path, 'Dataset Samples')
```

9. ✓ MediaPipe Hand Cropping

```
# MEDIAPIPE HAND CROPPING
class HandCropper:
    """MediaPipe hand detection with smart cropping
    Note: MediaPipe uses CPU by default. We minimize its impact by:
    1. Caching cropped images
    2. Using batch processing
    3. Running during data prep, not training
    """
    def __init__(self):
        self.mp_hands = mp.solutions.hands
        self.hands = self.mp_hands.Hands(
            static_image_mode=True,
            max_num_hands=1,
            min_detection_confidence=CONFIG['mediapipe_confidence'],
            model_complexity=CONFIG['mediapipe_model_complexity']
        )
        self.margin = CONFIG['mediapipe_margin']
        self.stats = {'total': 0, 'success': 0, 'fallback': 0}

    def crop_hand(self, image):
        """Detect and crop hand region"""
        h, w = image.shape[:2]
        self.stats['total'] += 1

        # Add padding for edge detection
        pad = 20
        padded = cv2.copyMakeBorder(image, pad, pad, pad, pad,
                                     cv2.BORDER_CONSTANT, value=[0, 0, 0])

        # Convert BGR to RGB for MediaPipe
        rgb = cv2.cvtColor(padded, cv2.COLOR_BGR2RGB)
        results = self.hands.process(rgb)

        if results.multi_hand_landmarks:
            landmarks = results.multi_hand_landmarks[0]
            h_pad, w_pad = padded.shape[:2]

            x_coords = [int(lm.x * w_pad) for lm in landmarks.landmark]
            y_coords = [int(lm.y * h_pad) for lm in landmarks.landmark]

            x_min = max(0, min(x_coords) - self.margin)
            x_max = min(w_pad, max(x_coords) + self.margin)
            y_min = max(0, min(y_coords) - self.margin)
            y_max = min(h_pad, max(y_coords) + self.margin)

            if x_max > x_min and y_max > y_min:
                cropped = padded[y_min:y_max, x_min:x_max]
                if cropped.shape[0] > 50 and cropped.shape[1] > 50:
                    self.stats['success'] += 1
                    return cropped, True

        # Fallback: center crop
        self.stats['fallback'] += 1
        margin_h, margin_w = int(h * 0.1), int(w * 0.1)
        return image[margin_h:h-margin_h, margin_w:w-margin_w], False

    def get_stats(self):
        """Return detection statistics"""
        total = self.stats['total']
        if total == 0:
            return self.stats
        self.stats['success_rate'] = self.stats['success'] / total * 100
        self.stats['fallback_rate'] = self.stats['fallback'] / total * 100
        return self.stats

    def close(self):
        """Clean up MediaPipe resources"""
        if hasattr(self, 'hands') and self.hands:
            self.hands.close()

def process_dataset_with_mediapipe():
    """Process and cache dataset with MediaPipe hand cropping
```

NOTE: MediaPipe uses TensorFlow Lite which runs on CPU only.
This is expected behavior - GPU is used for model training, not MediaPipe.
We optimize by caching results so this only runs once.

"""

```
logger.section("MEDIAPIPE HAND CROPPING")
logger.log(" NOTE: MediaPipe uses TFLite (CPU-only). This is normal.")
logger.log(" GPU will be used for model training after preprocessing")
logger.log("")
```

```
# Check if cached crops exist
```

```
if CONFIG['use_cached_crops']:
    train_exists = os.path.exists(CONFIG['cropped_train_path'])
    has_classes = False
    if train_exists:
        subdirs = [d for d in os.listdir(CONFIG['cropped_train_path'])
                    if os.path.isdir(os.path.join(CONFIG['cropped_train_path'], d))]
        has_classes = len(subdirs) > 0

    if has_classes:
        class_names = sorted([d for d in subdirs if d not in CONFIG['skip_folders']])
        logger.log(f" Using cached crops from: {CONFIG['cropped_train_path']}")
        logger.log(f" Found {len(class_names)} classes")
```

```
# Count cached images
```

```
total_cached = 0
```

```
for cls in class_names:
```

```
    cls_dir = os.path.join(CONFIG['cropped_train_path'], cls)
```

```
    if os.path.exists(cls_dir):
```

```
        total_cached += len([f for f in os.listdir(cls_dir) if
```

```
f.endswith(('.jpg', '.jpeg', '.png'))])
```

```
    logger.log(f" Total cached images: {total_cached:,}")
```

```
    return class_names
```

```
# Check source exists
```

```
if not os.path.exists(CONFIG['train_path']):
```

```
    logger.log(f" Training path not found: {CONFIG['train_path']}")
```

```
    return []
```

```
cropper = HandCropper()
```

```
# Get valid classes
```

```
all_folders = [d for d in os.listdir(CONFIG['train_path'])
```

```
                if os.path.isdir(os.path.join(CONFIG['train_path'], d))]
```

```
class_names = sorted([c for c in all_folders if c not in CONFIG['skip_folders']])
```

```
logger.log(f"\n Processing {len(class_names)} classes...")
```

```
logger.log(f" Images per class: {CONFIG['images_per_class'] or 'ALL'}")
```

```
# Calculate total work
```

```
total_images_to_process = 0
```

```
for class_name in class_names:
```

```
    src_dir = os.path.join(CONFIG['train_path'], class_name)
```

```
    all_images = [f for f in os.listdir(src_dir) if f.lower().endswith(('.jpg',
```

```
'.jpeg', '.png'))]
```

```
    if CONFIG['images_per_class']:
```

```
        total_images_to_process += min(len(all_images), CONFIG['images_per_class'])
```

```
    else:
```

```
        total_images_to_process += len(all_images)
```

```
logger.log(f" Total images to process: {total_images_to_process:,}")
```

```
est_time = total_images_to_process * 0.03 / 60
```

```
logger.log(f" Estimated time: ~{est_time:.1f} minutes")
```

```
# Process training data with detailed progress
```

```
logger.log("\n Processing TRAINING data...")
```

```
processing_stats = {
```

```
    'total_processed': 0,
```

```
    'successful': 0,
```

```
    'fallback': 0,
```

```
    'failed': 0,
```

```
    'per_class': {}
```

```
}
```

```
start_time = time.time()
```

```
for class_idx, class_name in enumerate(class_names):
```

```
    class_start = time.time()
```

```

src_dir = os.path.join(CONFIG['train_path'], class_name)
dst_dir = os.path.join(CONFIG['cropped_train_path'], class_name)
os.makedirs(dst_dir, exist_ok=True)

all_images = [f for f in os.listdir(src_dir)
               if f.lower().endswith(('.jpg', '.jpeg', '.png'))]

# Sample images if configured
if CONFIG['images_per_class'] and len(all_images) > CONFIG['images_per_class']:
    np.random.seed(42)
    images = list(np.random.choice(all_images, CONFIG['images_per_class'],
replace=False))
else:
    images = all_images

saved_count = 0
for img_name in images:
    img_path = os.path.join(src_dir, img_name)
    img = cv2.imread(img_path)
    if img is None:
        continue

    cropped, success = cropper.crop_hand(img)

    # Skip fallback crops if required
    if CONFIG['require_mediapipe_detection'] and not success:
        continue

    try:
        cropped_resized = cv2.resize(cropped, CONFIG['img_size'])
        save_path = os.path.join(dst_dir, img_name)
        cv2.imwrite(save_path, cropped_resized)
        saved_count += 1
        if success:
            processing_stats['successful'] += 1
        else:
            processing_stats['fallback'] += 1
    except Exception as e:
        processing_stats['failed'] += 1
        continue

processing_stats['total_processed'] += len(images)
processing_stats['per_class'][class_name] = saved_count

# Print per-class progress
class_time = time.time() - class_start
elapsed = time.time() - start_time
remaining_classes = len(class_names) - (class_idx + 1)
eta = (elapsed / (class_idx + 1)) * remaining_classes if class_idx > 0 else 0

print(f" [{class_idx+1:2d}/{len(class_names)}] {class_name}: {saved_count:,} saved
| "
      f"Time: {class_time:.1f}s | ETA: {eta/60:.1f}min", flush=True)

# Also save some to test set
if CONFIG['test_images_per_class_from_train'] > 0:
    remaining = [f for f in all_images if f not in images]
    if remaining:
        n_take = min(CONFIG['test_images_per_class_from_train'], len(remaining))
        np.random.seed(42)
        test_images = list(np.random.choice(remaining, n_take, replace=False))
        dst_test_dir = os.path.join(CONFIG['cropped_test_path'], class_name)
        os.makedirs(dst_test_dir, exist_ok=True)

        for img_name in test_images:
            img_path = os.path.join(src_dir, img_name)
            img = cv2.imread(img_path)
            if img is None:
                continue

            cropped, success = cropper.crop_hand(img)
            if CONFIG['require_mediapipe_detection'] and not success:
                continue

            try:

```

```

        cropped_resized = cv2.resize(cropped, CONFIG['img_size'])
        save_path = os.path.join(dst_test_dir, img_name)
        cv2.imwrite(save_path, cropped_resized)
    except:
        continue

# Print processing summary
total_time = time.time() - start_time
logger.log(f"\n{'='*60}")
logger.log(f" MEDIAPIPE PROCESSING COMPLETE")
logger.log(f"\n{'='*60}")
total_time_min = total_time/60
logger.log(f" Total time: {total_time_min:.1f} minutes")
total_processed = processing_stats['total_processed']
successful = processing_stats['successful']
fallback = processing_stats['fallback']
failed = processing_stats['failed']
processing_speed = total_processed/total_time
logger.log(f" Images processed: {total_processed:,}")
logger.log(f" Successful detections: {successful:,}")
logger.log(f" Fallback crops: {fallback:,}")
logger.log(f" Failed: {failed:,}")
logger.log(f" Processing speed: {processing_speed:.1f} images/sec")

# Per-class summary
logger.log(f"\n Per-Class Results:")
logger.log(f"\n{'-'*40}")
for cls, count in processing_stats['per_class'].items():
    logger.log(f" {cls}: {count:,} images saved")
logger.log(f"\n{'-'*40}")

# Save processing stats
stats_path = os.path.join(CONFIG['metrics_path'], 'mediapipe_processing_stats.json')
with open(stats_path, 'w') as f:
    json.dump(processing_stats, f, indent=2)
logger.log(f"\n Processing stats saved to: {stats_path}")

# Process test data from test folder
if os.path.exists(CONFIG['test_path']):
    logger.log("\n Processing TEST data...")
    test_files = [f for f in os.listdir(CONFIG['test_path'])
                  if f.lower().endswith('.jpg') and '_test' in f]
    test_saved = 0
    for test_file in tqdm(test_files, desc="Test images"):
        if test_file in CONFIG['skip_test_files']:
            continue

        class_name = test_file.split('_test')[0]
        if class_name not in class_names:
            continue

        src_path = os.path.join(CONFIG['test_path'], test_file)
        img = cv2.imread(src_path)
        if img is None:
            continue

        cropped, success = cropper.crop_hand(img)
        if CONFIG['require_mediapipe_detection'] and not success:
            continue

        try:
            cropped_resized = cv2.resize(cropped, CONFIG['img_size'])
            dst_dir = os.path.join(CONFIG['cropped_test_path'], class_name)
            os.makedirs(dst_dir, exist_ok=True)
            save_path = os.path.join(dst_dir, test_file)
            cv2.imwrite(save_path, cropped_resized)
            test_saved += 1
        except:
            continue

    logger.log(f"\n Test images saved: {test_saved}")

# Print statistics
stats = cropper.get_stats()
logger.log(f"\n MediaPipe Detection Statistics:")

```

```
logger.log(f" Total processed: {stats['total']:,}")
success_rate = f"{stats.get('success_rate', 0):.1f}%"
fallback_rate = f"{stats.get('fallback_rate', 0):.1f}%"
logger.log(f" Successful detections: {stats['success']:,} ({success_rate})")
logger.log(f" Fallback crops: {stats['fallback']:,} ({fallback_rate})")

cropper.close()

# Save sample visualization
visualize_cropped_samples(class_names)

# Visualize processing statistics
visualize_mediapipe_stats(processing_stats, class_names)

return class_names
```


10. ✓ Data Loading and Augmentation

```
# DATA LOADING
class AugmentedDataGenerator(keras.utils.Sequence):
    """Custom data generator with model-specific preprocessing"""
    def __init__(self, image_paths, labels, batch_size, model_name,
                 augmentation=None, shuffle=True):
        self.image_paths = image_paths
        self.labels = labels
        self.batch_size = batch_size
        self.model_name = model_name
        self.augmentation = augmentation
        self.shuffle = shuffle
        self.indices = np.arange(len(self.image_paths))

        # Set preprocessing function based on model
        if model_name == 'EfficientNetV2B3':
            self.preprocess_fn = effnet_preprocess
        elif model_name == 'ResNet50':
            self.preprocess_fn = resnet_preprocess
        elif model_name == 'InceptionV3':
            self.preprocess_fn = inception_preprocess
        else:
            self.preprocess_fn = None

        self.on_epoch_end()

    def __len__(self):
        return int(np.ceil(len(self.image_paths) / self.batch_size))

    def __getitem__(self, idx):
        batch_indices = self.indices[idx * self.batch_size:(idx + 1) * self.batch_size]
        batch_paths = [self.image_paths[i] for i in batch_indices]
        batch_labels = [self.labels[i] for i in batch_indices]

        X, y = [], []
        for path, label in zip(batch_paths, batch_labels):
            img = cv2.imread(path)
            if img is None:
                img = np.zeros((CONFIG['img_size'][0], CONFIG['img_size'][1], 3),
                               dtype=np.uint8)
            else:
                img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

            # Apply augmentation
            if self.augmentation:
                try:
                    img = self.augmentation(image=img)['image']
                except:
                    pass

            # Ensure correct size
            img = cv2.resize(img, CONFIG['img_size'])

            # Apply model-specific preprocessing
            if self.preprocess_fn:
                img = self.preprocess_fn(img.astype(np.float32))
            else:
                img = img.astype(np.float32) / 255.0

            X.append(img)
            y.append(label)

        return np.array(X), np.array(y)

    def on_epoch_end(self):
        if self.shuffle:
            np.random.shuffle(self.indices)

def load_dataset(cropped_path, class_names):
    """Load cropped dataset"""
    class_to_idx = {name: idx for idx, name in enumerate(class_names)}
    image_paths, labels = [], []
```



```

for class_name in class_names:
    class_dir = os.path.join(cropped_path, class_name)
    if not os.path.exists(class_dir):
        continue

    images = [f for f in os.listdir(class_dir)
               if f.lower().endswith(('.jpg', '.jpeg', '.png'))]
    for img_name in images:
        image_paths.append(os.path.join(class_dir, img_name))
        labels.append(class_to_idx[class_name])

return np.array(image_paths), np.array(labels)

```

11. ✓ Augmentation Pipelines

```

# AUGMENTATION PIPELINES
def get_augmentation_pipeline(model_name):
    """Model-specific augmentation pipelines"""
    if model_name == 'ResNet50':
        return A.Compose([
            A.Rotate(limit=25, p=0.8),
            A.Affine(scale=(0.85, 1.15), translate_percent=(-0.15, 0.15),
                    rotate=(-20, 20), shear=(-10, 10), p=0.7),
            A.HorizontalFlip(p=0.5),
            A.OneOf([
                A.MotionBlur(blur_limit=5, p=1.0),
                A.GaussianBlur(blur_limit=5, p=1.0),
            ], p=0.4),
            A.RandomBrightnessContrast(brightness_limit=0.25, contrast_limit=0.2, p=0.6),
            A.CLAHE(clip_limit=2.0, p=0.3),
            A.GaussNoise(var_limit=(10.0, 40.0), p=0.25),
            A.CoarseDropout(num_holes_range=(1, 3), hole_height_range=(8, 16),
                           hole_width_range=(8, 16), p=0.25),
        ])
    elif model_name == 'EfficientNetV2B3':
        return A.Compose([
            A.Rotate(limit=30, p=0.9),
            A.Affine(scale=(0.80, 1.2), translate_percent=(-0.12, 0.12),
                    rotate=(-18, 18), p=0.7),
            A.HorizontalFlip(p=0.5),
            A.OneOf([
                A.GaussianBlur(blur_limit=3, p=1.0),
                A.MotionBlur(blur_limit=3, p=1.0),
            ], p=0.35),
            A.RandomBrightnessContrast(brightness_limit=0.25, contrast_limit=0.2, p=0.6),
            A.CLAHE(clip_limit=2.0, p=0.3),
            A.GaussNoise(var_limit=(5.0, 30.0), p=0.2),
            A.CoarseDropout(num_holes_range=(1, 4), hole_height_range=(6, 20),
                           hole_width_range=(6, 20), p=0.25),
        ])
    elif model_name == 'InceptionV3':
        return A.Compose([
            A.Rotate(limit=15, p=0.6),
            A.Affine(scale=(0.9, 1.1), translate_percent=(-0.1, 0.1),
                    rotate=(-10, 10), p=0.5),
            A.RandomBrightnessContrast(brightness_limit=0.15, contrast_limit=0.15, p=0.5),
            A.CLAHE(clip_limit=1.5, p=0.2),
        ])

return None

```

12. ✓ Model Builders

```
# MODEL BUILDERS
def build_resnet50(num_classes):
    """ResNet50 with fine-tuning"""
    base = ResNet50(weights='imagenet', include_top=False,
                      input_shape=(*CONFIG['img_size'], 3))

    # Freeze early layers
    for layer in base.layers[:-30]:
        layer.trainable = False

    model = models.Sequential([
        base,
        layers.GlobalAveragePooling2D(),
        layers.BatchNormalization(),
        layers.Dropout(0.5),
        layers.Dense(512, activation='relu', kernel_regularizer=keras.regularizers.l2(1e-
4)),
        layers.BatchNormalization(),
        layers.Dropout(0.4),
        layers.Dense(256, activation='relu', kernel_regularizer=keras.regularizers.l2(1e-
4)),
        layers.Dropout(0.3),
        layers.Dense(num_classes, activation='softmax')
    ], name='ResNet50_AS1')

    return model

def build_efficientnet(num_classes):
    """EfficientNetV2B3 with fine-tuning"""
    try:
        base = EfficientNetV2B3(weights='imagenet', include_top=False,
                                  input_shape=(*CONFIG['img_size'], 3))
    except:
        base = EfficientNetV2B3(weights=None, include_top=False,
                                  input_shape=(*CONFIG['img_size'], 3))

    # Freeze most of the base model
    for layer in base.layers[:-80]:
        layer.trainable = False

    model = models.Sequential([
        base,
        layers.GlobalAveragePooling2D(),
        layers.BatchNormalization(),
        layers.Dropout(0.5),
        layers.Dense(512, activation='relu', kernel_regularizer=keras.regularizers.l2(1e-
4)),
        layers.BatchNormalization(),
        layers.Dropout(0.35),
        layers.Dense(256, activation='relu', kernel_regularizer=keras.regularizers.l2(1e-
4)),
        layers.Dropout(0.25),
        layers.Dense(num_classes, activation='softmax')
    ], name='EfficientNetV2B3_AS1')

    return model

def build_inception(num_classes):
    """InceptionV3 with fine-tuning"""
    base = InceptionV3(weights='imagenet', include_top=False,
                       input_shape=(*CONFIG['img_size'], 3))

    # Freeze most of the base model
    for layer in base.layers[:-60]:
        layer.trainable = False

    model = models.Sequential([
        base,
        layers.GlobalAveragePooling2D(),
        layers.BatchNormalization(),
        layers.Dropout(0.45),
        layers.Dense(512, activation='relu', kernel_regularizer=keras.regularizers.l2(1e-
```

```
4)),
    layers.BatchNormalization(),
    layers.Dropout(0.35),
    layers.Dense(256, activation='relu', kernel_regularizer=keras.regularizers.l2(1e-
4)),
    layers.Dropout(0.25),
    layers.Dense(num_classes, activation='softmax')
], name='InceptionV3_ASL')

return model

MODEL_BUILDERS = {
    'ResNet50': build_resnet50,
    'EfficientNetV2B3': build_efficientnet,
    'InceptionV3': build_inception
}
```

13. ✓ Model Information Collection

```
# MODEL INFO COLLECTION
def collect_model_info(num_classes):
    """Collect comprehensive information about all models BEFORE training
    This function:
    1. Creates each model architecture
    2. Collects detailed layer info, parameter counts
    3. Tests if existing models can be loaded
    4. Stores all info in a structured format
    5. Saves visualizations of model architectures
    """
    logger.section("MODEL INFORMATION COLLECTION")
    model_info = {}

    for model_name in CONFIG['train_models']:
        logger.log(f"\n{'='*60}")
        logger.log(f" ANALYZING: {model_name}")
        logger.log(f"{'='*60}")

        info = {
            'name': model_name,
            'status': 'unknown',
            'existing_model_path': None,
            'existing_model_found': False,
            'will_resume': False,
            'total_params': 0,
            'trainable_params': 0,
            'non_trainable_params': 0,
            'trainable_ratio': 0,
            'num_layers': 0,
            'base_model_layers': 0,
            'custom_layers': 0,
            'input_shape': None,
            'output_shape': None,
            'layer_summary': [],
            'memory_estimate_mb': 0
        }

        # Check for existing model
        existing_path = CONFIG['existing_models'].get(model_name)
        info['existing_model_path'] = existing_path

        if existing_path and os.path.exists(existing_path):
            info['existing_model_found'] = True
            info['will_resume'] = True
            info['status'] = 'RESUME - Existing model found'
            logger.log(f"\n EXISTING MODEL FOUND: {existing_path}")

            # Try to load and get info from existing model
            try:
                model, format_type = load_model_any_format(existing_path, model_name)
                if model is not None:
                    info['total_params'] = model.count_params()
                    info['trainable_params'] = sum([tf.size(w).numpy() for w in
model.trainable_weights])
                    info['non_trainable_params'] = info['total_params'] -
info['trainable_params']
                    info['num_layers'] = len(model.layers)
                    info['input_shape'] = str(model.input_shape)
                    info['output_shape'] = str(model.output_shape)
                    info['model_format'] = format_type

                    # If it's SavedModel (inference-only), mark for scratch training
                    if format_type == 'savedmodel':
                        info['will_resume'] = False
                        info['status'] = 'SCRATCH - SavedModel (inference-only)'
                        logger.log(f"\n SavedModel format is inference-only, will train
from scratch")

                    del model
                    keras.backend.clear_session()
            else:
                # format_type contains error message
```

```

        raise Exception(format_type)
    except Exception as e:
        logger.log(f"\n Could not load existing model: {e}")
        info['will_resume'] = False
        info['status'] = f'SCRATCH - Load failed: {str(e)}'
    else:
        info['status'] = 'SCRATCH - No existing model'
        if existing_path:
            logger.log(f"\n NO EXISTING MODEL at: {existing_path}")
        else:
            logger.log(f"\n NO EXISTING MODEL configured for {model_name}")
            logger.log(f" Will train from SCRATCH")

# Build fresh model to get architecture info
try:
    model = MODEL_BUILDERS[model_name](num_classes)

    info['total_params'] = model.count_params()
    info['trainable_params'] = sum([tf.size(w).numpy() for w in
model.trainable_weights])
    info['non_trainable_params'] = info['total_params'] - info['trainable_params']
    info['trainable_ratio'] = info['trainable_params'] / info['total_params'] if
info['total_params'] > 0 else 0
    info['num_layers'] = len(model.layers)
    info['input_shape'] = str(model.input_shape)
    info['output_shape'] = str(model.output_shape)

    # Estimate memory (rough: 4 bytes per parameter for float32)
    info['memory_estimate_mb'] = (info['total_params'] * 4) / (1024 * 1024)

    # Count base vs custom layers
    for layer in model.layers:
        if hasattr(layer, 'layers'): # It's a nested model (base)
            info['base_model_layers'] = len(layer.layers)
        else:
            info['custom_layers'] += 1

    # Collect layer summary (last 15 layers)
    for layer in model.layers[-15:]:
        # Safely get output shape (TF 2.18+ compatibility)
        try:
            out_shape = str(layer.output.shape) if hasattr(layer, 'output') and
layer.output is not None else 'N/A'
        except:
            out_shape = 'N/A'

        layer_info = {
            'name': layer.name,
            'type': layer.__class__.__name__,
            'output_shape': out_shape,
            'trainable': layer.trainable,
            'params': layer.count_params()
        }
        info['layer_summary'].append(layer_info)

    # Print detailed info
    logger.log(f"\n Architecture Details:")
    logger.log(f" Input Shape: {info['input_shape']}")
    logger.log(f" Output Shape: {info['output_shape']}")
    logger.log(f" Total Parameters: {info['total_params']:,}")
    logger.log(f" Trainable Params: {info['trainable_params']:,}")
    logger.log(f" Non-trainable Params: {info['non_trainable_params']:,}")
    logger.log(f" Trainable Ratio: {info['trainable_ratio']:.1%}")
    logger.log(f" Total Layers: {info['num_layers']}")
    logger.log(f" Base Model Layers: {info['base_model_layers']}")
    logger.log(f" Custom Layers: {info['custom_layers']}")
    logger.log(f" Est. Memory: {info['memory_estimate_mb']:.1f} MB")

    logger.log(f"\n Layer Summary (last 15):")
    logger.log("-" * 70)
    for li in info['layer_summary']:
        trainable_mark = "✓" if li['trainable'] else "x"
        logger.log(f" [{trainable_mark}] {li['name']:<30} {li['type']:<20}
{li['output_shape']:<20} {li['params']:,} params")
    logger.log("-" * 70)

```

```

        # Cleanup
        del model
        keras.backend.clear_session()
        gc.collect()
    except Exception as e:
        logger.log(f"\n Error building {model_name}: {e}")
        info['status'] = f'ERROR - {str(e)}'

    model_info[model_name] = info

# Save model info to JSON
model_info_path = os.path.join(CONFIG['metrics_path'], 'model_info.json')
# Convert to JSON-serializable format
save_info = {}
for name, info in model_info.items():
    save_info[name] = {k: v for k, v in info.items()}

with open(model_info_path, 'w') as f:
    json.dump(save_info, f, indent=2, default=str)
logger.log(f"\n Model info saved to: {model_info_path}")

# Create comparison visualization
visualize_model_architecture_comparison(model_info)

# Print summary table
logger.log(f"\n{'='*80}")
logger.log(f" MODEL SUMMARY TABLE")
logger.log(f"\n{'='*80}")
logger.log(f"{'Model':<20} {'Status':<25} {'Params':>15} {'Trainable':>15}")
logger.log(f"\n{'-'*80}")
for name, info in model_info.items():
    logger.log(f"{'name':<20} {info['status']:<25} {info['total_params']:>15,}
{info['trainable_params']:>15,}")
    logger.log(f"\n{'='*80}")

return model_info

```


14. ✓ Training Pipeline

```
# TRAINING
class DetailedMetricsCallback(keras.callbacks.Callback):
    """Custom callback to print detailed metrics during training"""
    def __init__(self, model_name, total_epochs):
        super().__init__()
        self.model_name = model_name
        self.total_epochs = total_epochs
        self.best_val_acc = 0
        self.best_epoch = 0
        self.start_time = None
        self.epoch_times = []

    def on_train_begin(self, logs=None):
        self.start_time = time.time()
        print(f"\n{'='*80}")
        print(f" TRAINING STARTED: {self.model_name}")
        print(f"{'='*80}")

    def on_epoch_begin(self, epoch, logs=None):
        self.epoch_start = time.time()
        print(f"\n Epoch {epoch+1}/{self.total_epochs}")

    def on_epoch_end(self, epoch, logs=None):
        epoch_time = time.time() - self.epoch_start
        self.epoch_times.append(epoch_time)

        # Get metrics
        train_acc = logs.get('accuracy', 0)
        val_acc = logs.get('val_accuracy', 0)
        train_loss = logs.get('loss', 0)
        val_loss = logs.get('val_loss', 0)
        lr = float(keras.backend.get_value(self.model.optimizer.learning_rate))

        # Track best
        if val_acc > self.best_val_acc:
            self.best_val_acc = val_acc
            self.best_epoch = epoch + 1
            best_marker = " NEW BEST!"
        else:
            best_marker = ""

        # Calculate ETA
        avg_epoch_time = sum(self.epoch_times) / len(self.epoch_times)
        remaining_epochs = self.total_epochs - (epoch + 1)
        eta_seconds = avg_epoch_time * remaining_epochs
        eta_str = f"{eta_seconds/60:.1f}min" if eta_seconds < 3600 else f"
{eta_seconds/3600:.1f}h"

        # Print detailed metrics
        print(f" {'-'*60}␣")
        print(f" | Train Acc: {train_acc*100:6.2f}% | Val Acc: {val_acc*100:6.2f}% |
{best_marker}")
        print(f" | Train Loss: {train_loss:6.4f} | Val Loss: {val_loss:6.4f} |")
        print(f" | LR: {lr:.2e} | Time: {epoch_time:.1f}s | ETA: {eta_str:<8} |")
        print(f" | Best Val Acc: {self.best_val_acc*100:.2f}% @ Epoch {self.best_epoch} |")
        print(f" {'-'*60}␣")

    def on_train_end(self, logs=None):
        total_time = time.time() - self.start_time
        print(f"\n{'='*80}")
        print(f" TRAINING COMPLETE: {self.model_name}")
        print(f"{'='*80}")
        print(f" Total time: {total_time/60:.1f} minutes")
        print(f" Best val accuracy: {self.best_val_acc*100:.2f}% @ Epoch
{self.best_epoch}")
        avg_epoch_time = sum(self.epoch_times)/len(self.epoch_times) if self.epoch_times
else 0
        print(f" Average epoch time: {avg_epoch_time:.1f} seconds")
        print(f"{'='*80}\n")

def train_model(model_name, train_paths, train_labels, val_paths, val_labels,
                num_classes, class_names, force_scratch=False, training_mode='AUTO'):
```



```

"""Train or resume training for a model with intelligent mode selection
Args:
    training_mode: One of 'AUTO', 'SCRATCH', 'RESUME', 'FINE_TUNE'
    - AUTO: Automatically decide based on existing model
    - SCRATCH: Train new model from scratch
    - RESUME: Resume training from checkpoint (moderate epochs, low LR)
    - FINE_TUNE: Light fine-tuning (few epochs, very low LR)
"""
logger.section(f"TRAINING: {model_name}")
logger.log(f" Training mode: {training_mode}")

# Handle force_scratch legacy parameter
if force_scratch:
    training_mode = 'SCRATCH'

# Check for existing model (unless forced to scratch)
if training_mode == 'SCRATCH':
    logger.log(f"\n FORCED training from SCRATCH (ignoring existing model)")
    existing_model, model_path = None, None
else:
    existing_model, model_path = check_existing_model(model_name)

# Determine training parameters based on mode
if training_mode == 'FINE_TUNE' and existing_model:
    # Fine-tuning: very few epochs, minimal learning rate
    logger.log(f"\n FINE-TUNING from: {model_path}")
    model = existing_model
    epochs = CONFIG.get('fine_tune_epochs', 5) # Very few epochs
    initial_lr = 5e-6 # Very low LR for fine-tuning
    is_resume = True
    logger.log(f" • Fine-tune epochs: {epochs}")
    logger.log(f" • Fine-tune LR: {initial_lr}")
elif training_mode == 'RESUME' and existing_model:
    # Resume: moderate epochs, low learning rate
    logger.log(f"\n RESUMING training from: {model_path}")
    model = existing_model
    epochs = CONFIG['resume_epochs']
    initial_lr = 1e-5 # Low LR for continued training
    is_resume = True
elif existing_model and training_mode == 'AUTO':
    # Auto mode with existing model - treat as resume
    logger.log(f"\n AUTO mode: RESUMING from: {model_path}")
    model = existing_model
    epochs = CONFIG['resume_epochs']
    initial_lr = 1e-5
    is_resume = True
else:
    # Scratch training (new model or forced)
    logger.log(f"\n Training from SCRATCH")
    if training_mode not in ['SCRATCH', 'AUTO']:
        logger.log(f"\n No existing model found - falling back to SCRATCH")
    if training_mode == 'AUTO':
        logger.log(f"\n No existing model found at:
{CONFIG['existing_models'].get(model_name, 'N/A')}")
    model = MODEL_BUILDERS[model_name](num_classes)
    epochs = CONFIG['epochs']
    initial_lr = 1e-3
    is_resume = False

# Compile model
optimizer = keras.optimizers.Adam(learning_rate=initial_lr)
model.compile(
    optimizer=optimizer,
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

# Print detailed model info
total_params = model.count_params()
trainable_params = sum([tf.size(w).numpy() for w in model.trainable_weights])
non_trainable_params = total_params - trainable_params
logger.log(f"\n{'=' * 60}")
logger.log(f" MODEL ARCHITECTURE: {model.name}")
logger.log(f"{'=' * 60}")
logger.log(f" Total parameters: {total_params:,}")

```

```

logger.log(f" Trainable parameters: {trainable_params:,}")
logger.log(f" Non-trainable params: {non_trainable_params:,}")
trainable_ratio = trainable_params/total_params*100 if total_params > 0 else 0
logger.log(f" Trainable ratio: {trainable_ratio:.1f}%")
logger.log(f"{'=' * 60}")
logger.log(f" Epochs: {epochs}")
logger.log(f" Initial LR: {initial_lr}")
logger.log(f" Batch size: {CONFIG['batch_size']}")
logger.log(f" Training samples: {len(train_paths)}")
logger.log(f" Validation samples: {len(val_paths)}")
steps_per_epoch = len(train_paths) // CONFIG['batch_size']
logger.log(f" Steps per epoch: {steps_per_epoch}")
logger.log(f"{'=' * 60}")

# Print layer summary to terminal
logger.log("\n Layer Summary (last 10):")
logger.log("-" * 60)
for i, layer in enumerate(model.layers[-10:]): # Last 10 layers
    trainable_str = "✓" if layer.trainable else "x"
    # Safely get output shape (TF 2.18+ compatibility)
    try:
        out_shape = layer.output.shape if hasattr(layer, 'output') and layer.output is
not None else 'N/A'
    except:
        out_shape = 'N/A'
    logger.log(f" [{trainable_str}] {layer.name}: {out_shape}")
logger.log("-" * 60)

# Create data generators
aug_pipeline = get_augmentation_pipeline(model_name)

# Preview augmentation
visualize_augmentation_preview(train_paths, train_labels, aug_pipeline, model_name)

train_gen = AugmentedDataGenerator(
    train_paths, train_labels, CONFIG['batch_size'], model_name,
    augmentation=aug_pipeline, shuffle=True
)
val_gen = AugmentedDataGenerator(
    val_paths, val_labels, CONFIG['batch_size'], model_name,
    augmentation=None, shuffle=False
)

# Callbacks
callbacks = [
    DetailedMetricsCallback(model_name, epochs), # Custom detailed metrics
    ModelCheckpoint(
        os.path.join(CONFIG['models_path'], f'{model_name}_best.keras'),
        save_best_only=True, monitor='val_accuracy', mode='max', verbose=1
    ),
    EarlyStopping(
        monitor='val_accuracy', patience=8, restore_best_weights=True, verbose=1
    ),
    ReduceLROnPlateau(
        monitor='val_loss', factor=0.3, patience=4, min_lr=1e-7, verbose=1
    ),
    keras.callbacks.TerminateOnNaN()
]

# Train
logger.log(f"\n Starting training...")
logger.log(f" Using GPU: {HAS_GPU}")
start_time = time.time()

try:
    history = model.fit(
        train_gen,
        validation_data=val_gen,
        epochs=epochs,
        callbacks=callbacks,
        verbose=1
    )
except Exception as e:
    logger.log(f"\n Training failed: {e}")
    # Save crash model

```

```
crash_path = os.path.join(CONFIG['models_path'], f'{model_name}_crash.keras')
try:
    model.save(crash_path)
    logger.log(f"\n Crash model saved to: {crash_path}")
except:
    pass
raise

training_time = time.time() - start_time

# Save final model
final_path = os.path.join(CONFIG['models_path'], f'{model_name}_final.keras')
model.save(final_path)
logger.log(f"\n Final model saved to: {final_path}")
logger.log(f" Training time: {training_time/60:.1f} minutes")

# Plot training history
plot_training_history(history, model_name)

return model, history
```

15. ✓ Evaluation and Visualization

```
# EVALUATION
def evaluate_model(model, model_name, test_paths, test_labels, class_names):
    """Comprehensive model evaluation with ALL metrics"""
    logger.section(f"EVALUATING: {model_name}")

    # Set preprocessing function
    if model_name == 'EfficientNetV2B3':
        preprocess_fn = effnet_preprocess
    elif model_name == 'ResNet50':
        preprocess_fn = resnet_preprocess
    elif model_name == 'InceptionV3':
        preprocess_fn = inception_preprocess
    else:
        preprocess_fn = None

    logger.log(f" Using preprocessing: {preprocess_fn.__name__ if preprocess_fn else
'None'}")

    # Load ALL test data
    X_test, y_test = [], []
    logger.log(f" Loading ALL {len(test_paths)} test samples...")
    for path, label in zip(test_paths, test_labels):
        img = cv2.imread(path)
        if img is None:
            continue
        img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
        img = cv2.resize(img, CONFIG['img_size'])
        if preprocess_fn:
            img = preprocess_fn(img.astype(np.float32))
        else:
            img = img.astype(np.float32) / 255.0
        X_test.append(img)
        y_test.append(label)

    X_test = np.array(X_test)
    y_test = np.array(y_test)

    logger.log(f" Test samples: {len(X_test)}")
    logger.log(f" Data shape: {X_test.shape}")
    logger.log(f" Data range: [{X_test.min():.2f}, {X_test.max():.2f}]")

    if len(X_test) == 0:
        logger.log("\n No test samples - skipping evaluation")
        return None

    # Predictions
    y_pred_probs = model.predict(X_test, verbose=0)
    y_pred = np.argmax(y_pred_probs, axis=1)

    # COMPREHENSIVE METRICS
    # Basic metrics
    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred, average='weighted', zero_division=0)
    recall = recall_score(y_test, y_pred, average='weighted', zero_division=0)
    f1 = f1_score(y_test, y_pred, average='weighted', zero_division=0)

    # Advanced metrics
    balanced_acc = balanced_accuracy_score(y_test, y_pred)
    cohen_kappa = cohen_kappa_score(y_test, y_pred)
    try:
        mcc = matthews_corrcoef(y_test, y_pred)
    except:
        mcc = 0.0

    # Multi-class ROC-AUC (one-vs-rest)
    try:
        roc_auc = roc_auc_score(y_test, y_pred_probs, multi_class='ovr')
    except:
        roc_auc = None

    # Log loss
    try:
```

```

        logloss = log_loss(y_test, y_pred_probs)
    except:
        logloss = None

# Per-class metrics (ALL classes)
precision_per_class = precision_score(y_test, y_pred, average=None, zero_division=0)
recall_per_class = recall_score(y_test, y_pred, average=None, zero_division=0)
f1_per_class = f1_score(y_test, y_pred, average=None, zero_division=0)

# Macro averages
precision_macro = precision_score(y_test, y_pred, average='macro', zero_division=0)
recall_macro = recall_score(y_test, y_pred, average='macro', zero_division=0)
f1_macro = f1_score(y_test, y_pred, average='macro', zero_division=0)

# Top-K accuracy
try:
    top3_acc = top_k_accuracy_score(y_test, y_pred_probs, k=3)
    top5_acc = top_k_accuracy_score(y_test, y_pred_probs, k=5)
except:
    top3_acc = None
    top5_acc = None

# Confidence analysis
pred_confidences = np.max(y_pred_probs, axis=1)
avg_confidence = np.mean(pred_confidences)
correct_mask = y_pred == y_test
avg_conf_correct = np.mean(pred_confidences[correct_mask]) if np.any(correct_mask) else
0
avg_conf_wrong = np.mean(pred_confidences[~correct_mask]) if np.any(~correct_mask) else
0

# Confusion matrix
cm = confusion_matrix(y_test, y_pred)
per_class_acc = cm.diagonal() / cm.sum(axis=1) if cm.sum() > 0 else
np.zeros(len(class_names))

# Most confused pairs
cm_no_diag = cm.copy()
np.fill_diagonal(cm_no_diag, 0)
confused_pairs = []
for i in range(len(class_names)):
    for j in range(len(class_names)):
        if cm_no_diag[i, j] > 0:
            confused_pairs.append({
                'true': class_names[i],
                'pred': class_names[j],
                'count': int(cm_no_diag[i, j])
            })
confused_pairs.sort(key=lambda x: x['count'], reverse=True)

# DISPLAY COMPREHENSIVE RESULTS
logger.log(f"\n{'='*70}")
logger.log(f" COMPREHENSIVE EVALUATION RESULTS for {model_name}")
logger.log(f"{'='*70}")
logger.log(f"\n OVERALL METRICS:")
logger.log(f" {'-'*60}┐")
logger.log(f" | Accuracy: {accuracy:.4f} ({accuracy*100:.2f}%) |")
logger.log(f" | Balanced Accuracy: {balanced_acc:.4f} ({balanced_acc*100:.2f}%) |")
logger.log(f" | Precision (weighted): {precision:.4f} |")
logger.log(f" | Recall (weighted): {recall:.4f} |")
logger.log(f" | F1-Score (weighted): {f1:.4f} |")
logger.log(f" | F1-Score (macro): {f1_macro:.4f} |")
logger.log(f" | Cohen's Kappa: {cohen_kappa:.4f} |")
logger.log(f" | Matthews CC: {mcc:.4f} |")
if roc_auc:
    logger.log(f" | ROC-AUC (weighted): {roc_auc:.4f} |")
if logloss:
    logger.log(f" | Log Loss: {logloss:.4f} |")
if top3_acc:
    logger.log(f" | Top-3 Accuracy: {top3_acc:.4f} ({top3_acc*100:.2f}%) |")
if top5_acc:
    logger.log(f" | Top-5 Accuracy: {top5_acc:.4f} ({top5_acc*100:.2f}%) |")
logger.log(f" {'-'*60}└")

logger.log(f"\n CONFIDENCE ANALYSIS:")

```

```

logger.log(f" • Average Confidence: {avg_confidence:.4f}")
logger.log(f" • Conf (Correct Preds): {avg_conf_correct:.4f}")
logger.log(f" • Conf (Wrong Preds): {avg_conf_wrong:.4f}")
confidence_gap = avg_conf_correct - avg_conf_wrong
calibration = "(excellent calibration)" if confidence_gap > 0.2 else "(good
calibration)"
logger.log(f" • Confidence Gap: {confidence_gap:.4f} {calibration}")

# Classification report (shows ALL classes)
logger.log(f"\n CLASSIFICATION REPORT (ALL {len(class_names)} CLASSES):")
report = classification_report(y_test, y_pred, target_names=class_names, digits=4)
logger.log(report)

# Per-class metrics table
logger.log(f"\n PER-CLASS METRICS:")
logger.log(f" {'Class':<8} {'Accuracy':>10} {'Precision':>10} {'Recall':>10} {'F1':>10}
{'Support':>8}")
logger.log(f" {'-'*56}")
for i, cls_name in enumerate(class_names):
    if i < len(per_class_acc):
        support = int(cm.sum(axis=1)[i]) if i < len(cm) else 0
        logger.log(f" {cls_name:<8} {per_class_acc[i]:>10.4f}
{precision_per_class[i]:>10.4f} "
                    f"{recall_per_class[i]:>10.4f} {f1_per_class[i]:>10.4f}
{support:>8}")

# Most confused pairs
logger.log(f"\n MOST CONFUSED PAIRS (Top 10):")
for pair in confused_pairs[:10]:
    logger.log(f" {pair['true']} → {pair['pred']}: {pair['count']} mistakes")

# Save detailed report
report_path = os.path.join(CONFIG['metrics_path'],
f'{model_name}_classification_report.txt')
with open(report_path, 'w') as f:
    f.write(f"{'='*70}\n")
    f.write(f"Model: {model_name}\n")
    f.write(f"{'='*70}\n\n")
    f.write(f"OVERALL METRICS:\n")
    f.write(f" Accuracy: {accuracy:.4f}\n")
    f.write(f" Balanced Accuracy: {balanced_acc:.4f}\n")
    f.write(f" Precision (weighted): {precision:.4f}\n")
    f.write(f" Recall (weighted): {recall:.4f}\n")
    f.write(f" F1-Score (weighted): {f1:.4f}\n")
    f.write(f" F1-Score (macro): {f1_macro:.4f}\n")
    f.write(f" Cohen's Kappa: {cohen_kappa:.4f}\n")
    f.write(f" Matthews CC: {mcc:.4f}\n")
    if roc_auc:
        f.write(f" ROC-AUC: {roc_auc:.4f}\n")
    if logloss:
        f.write(f" Log Loss: {logloss:.4f}\n")
    if top3_acc:
        f.write(f" Top-3 Accuracy: {top3_acc:.4f}\n")
    if top5_acc:
        f.write(f" Top-5 Accuracy: {top5_acc:.4f}\n\n")

    f.write(f"CLASSIFICATION REPORT:\n")
    f.write(f"{report}\n\n")
    f.write(f"MOST CONFUSED PAIRS:\n")
    for pair in confused_pairs[:20]:
        f.write(f" {pair['true']} → {pair['pred']}: {pair['count']}\n")

logger.log(f"\n Detailed report saved to: {report_path}")

# 3D CONFUSION MATRIX VISUALIZATIONS
# Restore original cm (diagonal was modified for confusion analysis)
cm = confusion_matrix(y_test, y_pred)
n_classes = len(class_names)

# Create 3D Surface confusion matrix
logger.log(f"\n Creating 3D confusion matrix for {model_name}...")
fig_cm = make_subplots(
    rows=1, cols=2,
    specs=[[{"type": "surface"}, {"type": "surface"}]],
    subplot_titles=['Raw Confusion Matrix', 'Normalized Confusion Matrix'],

```



```

        horizontal_spacing=0.05
    )

    # 3D Surface for raw confusion matrix
    fig_cm.add_trace(
        go.Surface(
            z=cm,
            x=list(range(n_classes)),
            y=list(range(n_classes)),
            colorscale='Blues',
            showscale=True,
            colorbar=dict(title='Count', x=0.45, len=0.8),
            hovertemplate='True: %{y}<br>Pred: %{x}<br>Count: %{z}<extra></extra>',
            contours=dict(
                z=dict(show=True, usecolormap=True, highlightcolor="white", project_x=True)
            )
        ),
        row=1, col=1
    )

    # Normalized confusion matrix
    cm_normalized = cm.astype('float') / cm.sum(axis=1)[:, np.newaxis]
    cm_normalized = np.nan_to_num(cm_normalized)

    fig_cm.add_trace(
        go.Surface(
            z=cm_normalized * 100,
            x=list(range(n_classes)),
            y=list(range(n_classes)),
            colorscale='RdYlGn',
            showscale=True,
            colorbar=dict(title='%', x=1.0, len=0.8),
            hovertemplate='True: %{y}<br>Pred: %{x}<br>Rate: %{z:.1f}%<extra></extra>',
            contours=dict(
                z=dict(show=True, usecolormap=True, highlightcolor="white", project_x=True)
            )
        ),
        row=1, col=2
    )

    # Update both scenes
    for col in [1, 2]:
        fig_cm.update_scenes(
            dict(
                xaxis=dict(title='Predicted', tickvals=list(range(n_classes)),
                    ticktext=class_names),
                yaxis=dict(title='True', tickvals=list(range(n_classes)),
                    ticktext=class_names),
                zaxis=dict(title='Count' if col == 1 else 'Rate %'),
                camera=dict(eye=dict(x=1.5, y=-1.5, z=1.0))
            ),
            row=1, col=col
        )

    fig_cm.update_layout(
        title=dict(
            text=f' {model_name} - 3D Confusion Matrix (Rotate to explore!)',
            font=dict(size=18, color='#333'),
            x=0.5
        ),
        height=700,
        template='plotly_white'
    )

    cm_path = os.path.join(CONFIG['metrics_path'], f'{model_name}_confusion_matrix.png')
    save_and_show_plotly(fig_cm, cm_path, f'{model_name} 3D Confusion Matrix')

    # Return comprehensive metrics
    return {
        'accuracy': accuracy,
        'balanced_accuracy': balanced_acc,
        'precision': precision,
        'recall': recall,
        'f1_score': f1,
        'f1_macro': f1_macro,
    }

```



```
'cohen_kappa': cohen_kappa,  
'matthews_corrcoef': mcc,  
'roc_auc': roc_auc,  
'log_loss': logloss,  
'top3_accuracy': top3_acc,  
'top5_accuracy': top5_acc,  
'avg_confidence': avg_confidence,  
'avg_conf_correct': avg_conf_correct,  
'avg_conf_wrong': avg_conf_wrong  
}
```

16. ✓ Final Comparison and Main Pipeline

```
# FINAL COMPARISON
def plot_model_comparison(results):
    """Plot final model comparison with SOLID 3D bars"""
    logger.section("MODEL COMPARISON")

    # Filter valid results
    valid_results = {k: v for k, v in results.items() if v is not None}
    if not valid_results:
        logger.log("\n No valid results to compare")
        return

    # Create comparison dataframe
    df = pd.DataFrame(valid_results).T
    df.index.name = 'Model'
    logger.log("\n Final Results:")
    logger.log(df.to_string())

    # Find best model
    best_model = df['accuracy'].idxmax()
    best_acc = df.loc[best_model, 'accuracy']
    logger.log(f"\n Best Model: {best_model} ({best_acc:.4f})")

    # Save results
    results_path = os.path.join(CONFIG['metrics_path'], 'final_results.csv')
    df.to_csv(results_path)
    logger.log(f"\n Results saved to: {results_path}")

    # Create 3D model comparison visualization with SOLID bars
    models = list(valid_results.keys())
    n_models = len(models)
    metrics = ['accuracy', 'precision', 'recall', 'f1_score', 'cohen_kappa']

    fig = go.Figure()

    # Create SOLID 3D bars for each model and metric
    metric_colors = ['#2ecc71', '#3498db', '#e74c3c', '#9b59b6', '#f39c12']
    for m_idx, metric in enumerate(metrics):
        values = [v.get(metric, 0) or 0 for v in valid_results.values()]
        for i, (model, val) in enumerate(zip(models, values)):
            # Create solid 3D bar with proper position
            x_center = i + (m_idx - 2) * 0.18 # Spread metrics within model group
            bar_width = 0.15
            bar_depth = 0.4

            bar = create_3d_bar_mesh(
                x_center=x_center,
                y_center=0,
                z_height=val,
                bar_width=bar_width,
                bar_depth=bar_depth,
                color=metric_colors[m_idx],
                opacity=0.95,
                name=metric.replace('_', ' ').title() if i == 0 else '',
                hovertemplate=f'<b>{model}</b><br>{metric}: {val:.4f}<extra></extra>'
            )
            # Set legend group manually
            bar.update(showlegend=(i == 0), legendgroup=metric)
            fig.add_trace(bar)

    # Add text labels
    fig.add_trace(go.Scatter3d(
        x=list(range(n_models)),
        y=[0] * n_models,
        z=[1.08] * n_models,
        mode='text',
        text=models,
        textfont=dict(size=12, color='black', family='Arial Black'),
        hoverinfo='skip',
        showlegend=False
    ))

    fig.update_layout(
```

```

        title=dict(
            text=' 3D Model Comparison (Rotate to explore!)',
            font=dict(size=20, color='#333'),
            x=0.5
        ),
        scene=dict(
            xaxis=dict(title='Model', tickvals=list(range(n_models)), ticktext=models),
            yaxis=dict(title='', showticklabels=False, range=[-0.8, 0.8]),
            zaxis=dict(title='Score', range=[0, 1.15]),
            camera=dict(eye=dict(x=1.5, y=1.5, z=0.8)),
            bgcolor='rgba(250,250,250,0.9)'
        ),
        height=650,
        template='plotly_white',
        legend=dict(
            title=dict(text='Metrics'),
            yanchor="top",
            y=0.99,
            xanchor="left",
            x=1.02
        )
    )

comparison_path = os.path.join(CONFIG['metrics_path'], 'model_comparison.png')
save_and_show_plotly(fig, comparison_path, '3D Model Comparison')

```

MAIN PIPELINE

```

def main():
    """Main training pipeline
    FLOW:
    1. Collect and store all dataset information
    2. Process dataset with MediaPipe
    3. Collect all model information (architecture, params, status)
    4. Test existing models BEFORE training (baseline metrics)
    5. Train models (resume or from scratch)
    6. Evaluate trained models
    7. Compare results (baseline vs trained)
    """
    start_time = time.time()
    print("\n" + "=" * 80)
    print(" ASL ALPHABET CLASSIFICATION - UNIFIED TRAINING PIPELINE")
    print("=" * 80)
    print(f" Start time: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}")
    print(f" GPU Available: {HAS_GPU}")
    print(f" TensorFlow: {tf.__version__}")
    print("=" * 80 + "\n")

    # PHASE 1: DATASET INFO
    logger.section("PHASE 1: DATASET INFORMATION")
    logger.log("Collecting comprehensive dataset information...")
    metrics = analyze_dataset()
    if metrics['total_classes'] == 0:
        logger.log("\n No valid classes found. Please check CONFIG paths.")
        return

    # PHASE 2: MEDIAPIPE PROCESSING
    logger.section("PHASE 2: MEDIAPIPE PROCESSING")
    class_names = process_dataset_with_mediapipe()
    if len(class_names) == 0:
        logger.log("\n No classes after processing. Please check dataset.")
        return

    num_classes = len(class_names)

    # Save class names
    class_names_path = os.path.join(CONFIG['models_path'], 'class_names.json')
    with open(class_names_path, 'w') as f:
        json.dump(class_names, f, indent=2)
    logger.log(f"\n Class names saved to: {class_names_path}")

    # PHASE 3: MODEL INFO COLLECTION
    logger.section("PHASE 3: MODEL INFORMATION")
    logger.log("Collecting comprehensive model information...")
    model_info = collect_model_info(num_classes)

```

```

# PHASE 4: LOAD DATASET
logger.section("PHASE 4: LOADING DATASET")
train_paths, train_labels = load_dataset(CONFIG['cropped_train_path'], class_names)
test_paths, test_labels = load_dataset(CONFIG['cropped_test_path'], class_names)
logger.log(f"\n Training samples: {len(train_paths)}")
logger.log(f" Test samples: {len(test_paths)}")
logger.log(f" Classes: {num_classes}")

if len(train_paths) == 0:
    logger.log("\n No training data found.")
    return

# PHASE 5: BASELINE TESTING
if len(test_paths) > 0:
    logger.section("PHASE 5: PRE-TRAINING BASELINE TEST")
    logger.log("Testing existing models to get baseline metrics...")
    baseline_results = test_models_before_training(
        test_paths, test_labels, class_names, model_info
    )
else:
    logger.log("\n No test data available for baseline testing")
    baseline_results = {}

# PHASE 6: TRAIN/VAL SPLIT
logger.section("PHASE 6: PREPARING TRAINING DATA")

# Split train/val
try:
    train_paths, val_paths, train_labels, val_labels = train_test_split(
        train_paths, train_labels,
        test_size=CONFIG['val_split'],
        random_state=42,
        stratify=train_labels
    )
except Exception as e:
    logger.log(f"\n Stratified split failed: {e}. Using random split.")
    split_idx = int(len(train_paths) * (1 - CONFIG['val_split']))
    train_paths, val_paths = train_paths[:split_idx], train_paths[split_idx:]
    train_labels, val_labels = train_labels[:split_idx], train_labels[split_idx:]

logger.log(f"\n Training samples: {len(train_paths)}")
logger.log(f" Validation samples: {len(val_paths)}")
logger.log(f" Test samples: {len(test_paths)}")

# PHASE 7: INTELLIGENT TRAINING DECISIONS
logger.section("PHASE 7: INTELLIGENT TRAINING ANALYSIS")
results = {}

# ===== INTELLIGENT TRAINING DECISION THRESHOLDS =====
SKIP_THRESHOLD = 0.98 # If accuracy >= 98%, skip training (model is excellent)
GOOD_THRESHOLD = 0.95 # If accuracy >= 95%, model is good but can be fine-tuned
MODERATE_THRESHOLD = 0.30 # If accuracy >= 30%, resume training
# Below 30% = retrain from scratch (model incompatible)

# Analyze models and create training plan
training_plan = {}
for model_name in CONFIG['train_models']:
    baseline = baseline_results.get(model_name, {})
    action, reason, details = analyze_model_decision(model_name, baseline)
    training_plan[model_name] = {
        'action': action,
        'reason': reason,
        'details': details,
        'baseline': baseline
    }

# Display training plan
display_intelligent_training_summary(training_plan)

# Train or skip models based on decisions
for model_name in CONFIG['train_models']:
    plan = training_plan[model_name]
    action = plan['action']

    if action == 'SKIP':

```

```

logger.log(f"\n SKIPPING {model_name} - Already excellent!")
baseline = plan['baseline']
# Use baseline results as final results
results[model_name] = {
    'accuracy': baseline.get('accuracy', 0),
    'balanced_accuracy': baseline.get('balanced_accuracy', 0),
    'precision': baseline.get('precision', 0),
    'recall': baseline.get('recall', 0),
    'f1_score': baseline.get('f1_score', 0),
    'f1_macro': baseline.get('f1_macro', 0),
    'cohen_kappa': baseline.get('cohen_kappa', 0),
    'matthews_corrcoef': baseline.get('matthews_corrcoef', 0),
    'roc_auc': baseline.get('roc_auc', None),
    'log_loss': baseline.get('log_loss', None),
    'top3_accuracy': baseline.get('top3_accuracy', None),
    'top5_accuracy': baseline.get('top5_accuracy', None),
    'avg_confidence': baseline.get('avg_confidence', 0),
    'skipped_training': True,
    'reason': 'Excellent baseline performance'
}
else:
    # Train the model
    try:
        model, history = train_model(
            model_name, train_paths, train_labels,
            val_paths, val_labels, num_classes, class_names,
            training_mode=action # Pass the intelligent decision
        )

        # Evaluate the trained model
        model_results = evaluate_model(
            model, model_name, test_paths, test_labels, class_names
        )
        results[model_name] = model_results

        # Clear memory
        del model
        keras.backend.clear_session()
        gc.collect()
        time.sleep(3)
    except Exception as e:
        logger.log(f"\n {model_name} failed: {e}")
        import traceback
        logger.log(traceback.format_exc())
        results[model_name] = None

# PHASE 8: FINAL COMPARISON
logger.section("PHASE 8: FINAL COMPARISON")
plot_model_comparison(results)

# SUMMARY
total_time = time.time() - start_time
logger.section("TRAINING COMPLETE")
logger.log(f"\n Total time: {total_time/60:.1f} minutes ({total_time/3600:.2f} hours)")
logger.log(f" Models saved to: {CONFIG['models_path']}")
logger.log(f" Metrics saved to: {CONFIG['metrics_path']}")
logger.log(f" Log saved to: {logger.log_file}")

# Print summary of all saved files
logger.log(f"\n OUTPUT FILES SUM# ASL Alphabet Classification - TODO LIST

## 1. ✓ Installation and Setup
```python
INSTALLATION CHECK
import subprocess
import sys

def install_if_missing(package, pip_name=None):
 """Install package if not available"""
 try:
 __import__(package)
 except ImportError:
 pip_name = pip_name or package
 print(f"Installing {pip_name}...")
 subprocess.check_call([sys.executable, '-m', 'pip', 'install', pip_name])

```

```
Check and install dependencies
install_if_missing('mediapipe')
install_if_missing('cv2', 'opencv-python')
install_if_missing('alumentations')
install_if_missing('sklearn', 'scikit-learn')
install_if_missing('tqdm')
install_if_missing('plotly')
install_if_missing('kaleido') # For saving plotly figures
```



## 2. ✓ Imports and Configuration

---

```
IMPORTS
import os
import shutil
import numpy as np
import pandas as pd
import matplotlib
Check if running in notebook (Kaggle/Jupyter) - use inline display
import sys
IN_NOTEBOOK = 'ipykernel' in sys.modules or 'IPython' in sys.modules
if not IN_NOTEBOOK:
 matplotlib.use('Agg') # Non-interactive backend only if not in notebook
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import seaborn as sns
from pathlib import Path
import cv2
import mediapipe as mp
from tqdm import tqdm
import json
Plotly for interactive 3D visualizations
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots
import plotly.io as pio
Set Plotly renderer for Kaggle/Jupyter notebooks
if 'ipykernel' in sys.modules or 'IPython' in sys.modules:
 pio.renderers.default = 'notebook' # Best for Kaggle
For inline display in Kaggle notebooks
if IN_NOTEBOOK:
 try:
 from IPython.display import display, Image, HTML, clear_output
 from IPython import get_ipython
 get_ipython().run_line_magic('matplotlib', 'inline')
 print(" Notebook detected - plots will display inline!")
 except:
 pass
from collections import Counter
from datetime import datetime
import gc
import time
import warnings
warnings.filterwarnings('ignore')
TensorFlow imports with GPU configuration
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, models
from tensorflow.keras.applications import ResNet50, EfficientNetV2B3, InceptionV3
from tensorflow.keras.applications.resnet50 import preprocess_input as resnet_preprocess
from tensorflow.keras.applications.efficientnet_v2 import preprocess_input as
effnet_preprocess
from tensorflow.keras.applications.inception_v3 import preprocess_input as
inception_preprocess
from tensorflow.keras.callbacks import ModelCheckpoint, EarlyStopping, ReduceLROnPlateau
import albumentations as A
Sklearn imports
from sklearn.model_selection import train_test_split
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
from sklearn.metrics import (
 accuracy_score, classification_report, confusion_matrix,
 precision_score, recall_score, f1_score, top_k_accuracy_score,
 balanced_accuracy_score, cohen_kappa_score, matthews_corrcoef,
 roc_auc_score, log_loss, roc_curve, auc, precision_recall_curve,
 average_precision_score
)
from sklearn.preprocessing import label_binarize
Set seeds for reproducibility
np.random.seed(42)
tf.random.set_seed(42)
```



### 3. ✓ GPU Configuration

---

```
GPU CONFIGURATION
def configure_gpu():
 """Configure GPU for optimal performance, minimize CPU usage"""
 print("\n" + "=" * 70)
 print("GPU CONFIGURATION")
 print("=" * 70)

 # Force single GPU usage
 os.environ["CUDA_VISIBLE_DEVICES"] = "0"
 os.environ["TF_FORCE_GPU_ALLOW_GROWTH"] = "true"

 # Optimize TensorFlow for GPU
 os.environ["TF_GPU_THREAD_MODE"] = "gpu_private"
 os.environ["TF_GPU_THREAD_COUNT"] = "2"

 # Reduce CPU parallelism to minimize CPU usage
 tf.config.threading.set_inter_op_parallelism_threads(2)
 tf.config.threading.set_intra_op_parallelism_threads(2)

 gpus = tf.config.list_physical_devices('GPU')
 if gpus:
 try:
 # Use only first GPU
 tf.config.set_visible_devices(gpus[0], 'GPU')
 # Enable memory growth to avoid OOM
 tf.config.experimental.set_memory_growth(gpus[0], True)
 print(f" GPU detected: {gpus[0].name}")
 print(f" Memory growth enabled")
 print(f" Using GPU 0 only")

 # Get GPU details
 try:
 gpu_details = tf.config.experimental.get_device_details(gpus[0])
 if gpu_details:
 print(f" GPU Details: {gpu_details}")
 except:
 pass
 return True
 except RuntimeError as e:
 print(f" GPU configuration error: {e}")
 return False
 else:
 print(" No GPU detected - using CPU (training will be slower)")
 return False

Configure GPU at import time
HAS_GPU = configure_gpu()
print(f"\n TensorFlow Version: {tf.__version__}")
print(f" GPU Available: {HAS_GPU}")
print(f" Physical Devices: {tf.config.list_physical_devices()}")
```

## 4. ✓ Dataset Configuration

---

```
CONFIGURATION
CONFIG = {
 # PATHS (KAGGLE ENVIRONMENT)
 # Dataset paths for Kaggle
 'train_path': '/kaggle/input/aslamerican-sign-language-alphabet-
dataset/ASL_Alphabet_Dataset/asl_alphabet_train',
 'test_path': '/kaggle/input/aslamerican-sign-language-alphabet-
dataset/ASL_Alphabet_Dataset/asl_alphabet_test',
 'cropped_train_path': '/kaggle/working/cropped_train', # MediaPipe cropped training
images
 'cropped_test_path': '/kaggle/working/cropped_test', # MediaPipe cropped test images
 'models_path': '/kaggle/working/models', # Saved models
 'metrics_path': '/kaggle/working/metrics', # Saved metrics and graphs

 # MODEL PATHS FOR RESUME
 # Set these to your existing model paths for resume training
 # If None or file doesn't exist, will train from scratch
 'existing_models': {
 'ResNet50': '/kaggle/input/resnet50-best-1-keras/keras/1/2',
 'InceptionV3': '/kaggle/input/inceptionv3-best-
keras/keras/1/1/InceptionV3_best.keras',
 'EfficientNetV2B3': '/kaggle/input/efficientnetv2b3-best-1-
keras/keras/1/1/EfficientNetV2B3_best.keras'
 },

 # DATASET CONFIG
 'skip_folders': ['del', 'nothing', 'space'],
 'skip_test_files': ['nothing_test.jpg', 'del_test.jpg', 'space_test.jpg'],
 'images_per_class': 1000, # Set to None for ALL images
 'test_images_per_class_from_train': 100,
 'require_mediapipe_detection': True,
 'use_cached_crops': True, # Skip MediaPipe if crops exist

 # TRAINING CONFIG
 'img_size': (224, 224),
 'batch_size': 32,
 'epochs': 35,
 'val_split': 0.15,
 'resume_epochs': 15, # Additional epochs when resuming
 'fine_tune_epochs': 5, # Few epochs for fine-tuning already-good models

 # MODELS TO TRAIN
 # Only these models will be trained (in order)
 # Include ALL models you want to train - they will be trained in this order
 'train_models': ['EfficientNetV2B3', 'ResNet50', 'InceptionV3'],

 # MEDIAPIPE CONFIG
 'mediapipe_confidence': 0.6,
 'mediapipe_margin': 30,
 'mediapipe_model_complexity': 1, # 0=lite, 1=full
}

Create directories
for path_key in ['cropped_train_path', 'cropped_test_path', 'models_path', 'metrics_path']:
 os.makedirs(CONFIG[path_key], exist_ok=True)
```

## 5. ✓ Logging and Visualization Helpers

---

```
LOGGING UTILITIES
class Logger:
 """Simple logger that prints to terminal and saves to file"""
 def __init__(self, log_file=None):
 self.log_file = log_file or os.path.join(CONFIG['metrics_path'],
'training_log.txt')
 self.start_time = datetime.now()
 # Clear previous log
 with open(self.log_file, 'w') as f:
 f.write(f"ASL Training Log - {self.start_time.strftime('%Y-%m-%d
%H:%M:%S')}\n")
 f.write("=" * 70 + "\n")

 def log(self, message, also_print=True):
 """Log message to file and optionally print"""
 timestamp = datetime.now().strftime('%H:%M:%S')
 formatted = f"[{timestamp}] {message}"
 if also_print:
 print(message)
 with open(self.log_file, 'a') as f:
 f.write(formatted + "\n")

 def section(self, title):
 """Print section header"""
 header = "\n" + "=" * 70 + f"\n{title}\n" + "=" * 70 + "\n"
 self.log(header)

logger = Logger()
```

## 6. ✓ 3D Visualization Helpers

---

```
3D BAR HELPER FUNCTION
def create_3d_bar_mesh(x_center, y_center, z_height, bar_width=0.4, bar_depth=0.4,
 color='blue', opacity=1.0, name='', hovertemplate=''):
 """
 Create a solid 3D bar using Mesh3d with proper triangular faces.
 This creates a proper rectangular prism (cuboid) with all 6 faces rendered correctly.
 Each face is made of 2 triangles = 12 triangles total for a closed solid bar.
 Args:
 x_center: X position of bar center
 y_center: Y position of bar center
 z_height: Height of bar (from 0)
 bar_width: Width in X direction
 bar_depth: Depth in Y direction
 color: Bar color
 opacity: Bar opacity (0-1)
 name: Name for legend
 hovertemplate: Hover text template
 Returns:
 go.Mesh3d trace for the bar
 """

 # Half dimensions
 hw = bar_width / 2 # half width
 hd = bar_depth / 2 # half depth

 # 8 vertices of the cuboid (bar)
 # Bottom face (z=0): vertices 0,1,2,3
 # Top face (z=height): vertices 4,5,6,7
 vertices_x = [
 x_center - hw, x_center + hw, x_center + hw, x_center - hw, # bottom: 0,1,2,3
 x_center - hw, x_center + hw, x_center + hw, x_center - hw # top: 4,5,6,7
]
 vertices_y = [
 y_center - hd, y_center - hd, y_center + hd, y_center + hd, # bottom
 y_center - hd, y_center - hd, y_center + hd, y_center + hd # top
]
 vertices_z = [
 0, 0, 0, 0, # bottom face at z=0
 z_height, z_height, z_height, z_height # top face at z=height
]

 # 12 triangular faces (2 per face × 6 faces)
 # Each face needs 2 triangles defined by vertex indices (i, j, k)
 # The vertices must be in counter-clockwise order when viewed from outside

 # Bottom face (z=0): vertices 0,1,2,3 - looking from below
 # Top face (z=h): vertices 4,5,6,7 - looking from above
 # Front face (y=-): vertices 0,1,5,4
 # Back face (y=+): vertices 2,3,7,6
 # Left face (x=-): vertices 0,3,7,4
 # Right face (x=+): vertices 1,2,6,5

 i_faces = [
 0, 0, # bottom: triangles (0,1,2) and (0,2,3)
 4, 4, # top: triangles (4,6,5) and (4,7,6)
 0, 0, # front: triangles (0,5,1) and (0,4,5)
 2, 2, # back: triangles (2,7,3) and (2,6,7)
 0, 0, # left: triangles (0,3,7) and (0,7,4)
 1, 1 # right: triangles (1,5,6) and (1,6,2)
]

 j_faces = [
 1, 2, # bottom
 6, 7, # top
 5, 4, # front
 7, 6, # back
 3, 7, # left
 5, 6 # right
]

 k_faces = [
 2, 3, # bottom
 5, 6, # top
]
```

```
1, 5, # front
3, 7, # back
7, 4, # left
6, 2 # right
]

return go.Mesh3d(
 x=vertices_x,
 y=vertices_y,
 z=vertices_z,
 i=i_faces,
 j=j_faces,
 k=k_faces,
 color=color,
 opacity=opacity,
 name=name,
 showlegend=False,
 hovertemplate=hovertemplate,
 flatshading=True, # Better solid appearance
 lighting=dict(
 ambient=0.7,
 diffuse=0.8,
 specular=0.2,
 roughness=0.5,
 fresnel=0.1
),
 lightposition=dict(x=100, y=200, z=300)
)
```

## 7. ✓ SavedModel Loader

---

```
SAVEDMODEL LOADER
def load_model_any_format(model_path, model_name=None):
 """
 Load a model from any format (Keras 3 .keras, H5, or TensorFlow SavedModel).
 Args:
 model_path: Path to the model file/directory
 model_name: Optional model name for logging
 Returns:
 (model, format_type) tuple or (None, error_message)
 """
 name = model_name or os.path.basename(model_path)

 # Helper function to find model files in a directory
 def find_model_file(base_path):
 """Search for model files in directory tree"""
 model_extensions = ['.keras', '.h5', '.hdf5']
 savedmodel_markers = ['saved_model.pb', 'saved_model.pbtxt']

 if not os.path.exists(base_path):
 return None, None

 # If it's a file, return it directly
 if os.path.isfile(base_path):
 return base_path, 'file'

 # Check if this directory is a SavedModel
 for marker in savedmodel_markers:
 if os.path.exists(os.path.join(base_path, marker)):
 return base_path, 'savedmodel'

 # Search for model files recursively (max depth 3)
 for root, dirs, files in os.walk(base_path):
 depth = root[len(base_path):].count(os.sep)
 if depth > 3:
 continue

 # Check for SavedModel markers in subdirs
 for marker in savedmodel_markers:
 if marker in files:
 return root, 'savedmodel'

 # Check for keras/h5 files
 for f in files:
 for ext in model_extensions:
 if f.endswith(ext):
 return os.path.join(root, f), 'file'

 return None, None

 # First, try to find the actual model file/directory
 actual_path, path_type = find_model_file(model_path)
 if actual_path and actual_path != model_path:
 logger.log(f" Found model at: {actual_path}")
 model_path = actual_path

 # Log directory contents for debugging
 if os.path.isdir(model_path):
 try:
 contents = os.listdir(model_path)
 logger.log(f" Directory contents: {contents[:10]}{'...' if len(contents) > 10}")
 except:
 pass
 else:
 pass

 # Try standard Keras load first (.keras or .h5)
 try:
 model = keras.models.load_model(model_path)
 logger.log(f" Loaded {name} using keras.models.load_model()")
 return model, 'keras'
 except Exception as e1:
 keras_error = str(e1)

 # If it's a directory, try multiple approaches for SavedModel format
```



```

if os.path.isdir(model_path):
 # Check for saved_model.pb first
 has_savedmodel_pb = os.path.exists(os.path.join(model_path, 'saved_model.pb'))
 has_savedmodel_pbtxt = os.path.exists(os.path.join(model_path,
'saved_model.pbtxt'))

 if not has_savedmodel_pb and not has_savedmodel_pbtxt:
 # Not a valid SavedModel directory - search subdirectories
 logger.log(f" No saved_model.pb found, searching subdirectories...")
 for subdir in os.listdir(model_path):
 subpath = os.path.join(model_path, subdir)
 if os.path.isdir(subpath):
 if os.path.exists(os.path.join(subpath, 'saved_model.pb')):
 logger.log(f" Found SavedModel in: {subpath}")
 model_path = subpath
 has_savedmodel_pb = True
 break

 # Check for .keras files
 for f in os.listdir(subpath):
 if f.endswith('.keras') or f.endswith('.h5'):
 keras_file = os.path.join(subpath, f)
 logger.log(f" Found Keras file: {keras_file}")
 try:
 model = keras.models.load_model(keras_file)
 logger.log(f" Loaded {name} from {keras_file}")
 return model, 'keras'
 except Exception as e:
 logger.log(f" Failed to load {keras_file}: {e}")

Approach 1: Try tf.saved_model.load (TF2 native)
if has_savedmodel_pb or has_savedmodel_pbtxt:
 try:
 logger.log(f" Trying tf.saved_model.load()...")
 imported = tf.saved_model.load(model_path)

 # Check if it has a keras model signature
 if hasattr(imported, 'signatures'):
 signatures = list(imported.signatures.keys())
 logger.log(f" Found signatures: {signatures}")

 # Get the serving function
 if 'serving_default' in signatures:
 serve_fn = imported.signatures['serving_default']

 # Create a wrapper model for inference
 class SavedModelWrapper(keras.Model):
 def __init__(self, serve_fn, **kwargs):
 super().__init__(**kwargs)
 self._serve_fn = serve_fn
 # Try to get output shape from signature
 output_info = list(serve_fn.structured_outputs.values())[0]
 self._output_classes = output_info.shape[-1]

 def call(self, inputs):
 # The serve function expects a dict with input tensors
 result = self._serve_fn(inputs)
 # Return the first output value
 return list(result.values())[0]

 @property
 def output_shape(self):
 return (None, self._output_classes)

 model = SavedModelWrapper(serve_fn, name=f"{name}_TF2")
 # Build the model with sample input
 model.build(input_shape=(None, 224, 224, 3))
 logger.log(f" Loaded {name} using tf.saved_model.load()")
 logger.log(f" Note: This is inference-only - cannot be fine-tuned")
 return model, 'savedmodel'
 except Exception as e2:
 logger.log(f" tf.saved_model.load failed: {e2}")

Neither worked
return None, f"Cannot load: {keras_error}"

```



## 8. ✓ Dataset Analysis

---

```
DATASET ANALYSIS
def analyze_dataset():
 """Comprehensive dataset analysis with metrics"""
 logger.section("DATASET ANALYSIS")
 metrics = {
 'total_classes': 0,
 'total_train_images': 0,
 'total_test_images': 0,
 'class_distribution': {},
 'image_sizes': [],
 'analysis_time': None
 }
 start_time = time.time()

 # Check if paths exist
 if not os.path.exists(CONFIG['train_path']):
 logger.log(f" Training path not found: {CONFIG['train_path']}")
 logger.log("Please update CONFIG['train_path'] to point to your dataset")
 return metrics

 # Get all class folders
 all_folders = [d for d in os.listdir(CONFIG['train_path'])
 if os.path.isdir(os.path.join(CONFIG['train_path'], d))]
 class_names = sorted([c for c in all_folders if c not in CONFIG['skip_folders']])
 metrics['total_classes'] = len(class_names)
 metrics['class_names'] = class_names

 logger.log(f"\n Dataset Path: {CONFIG['train_path']}")
 logger.log(f" Total folders found: {len(all_folders)}")
 logger.log(f" Skipped folders: {CONFIG['skip_folders']}")
 logger.log(f" Valid classes: {len(class_names)}")
 logger.log(f" Classes: {class_names}\n")

 # Analyze each class
 logger.log("Class Distribution:")
 logger.log("-" * 50)
 total_images = 0
 class_counts = {}
 for class_name in class_names:
 class_dir = os.path.join(CONFIG['train_path'], class_name)
 images = [f for f in os.listdir(class_dir)
 if f.lower().endswith(('.jpg', '.jpeg', '.png'))]
 count = len(images)
 class_counts[class_name] = count
 total_images += count

 # Sample image size
 if images and len(metrics['image_sizes']) < 10:
 sample_path = os.path.join(class_dir, images[0])
 img = cv2.imread(sample_path)
 if img is not None:
 metrics['image_sizes'].append(img.shape[:2])
 logger.log(f" {class_name}: {count:}, {images}")

 metrics['total_train_images'] = total_images
 metrics['class_distribution'] = class_counts

 logger.log("-" * 50)
 logger.log(f"\n Total Training Images: {total_images:,}")
 avg_per_class = total_images // len(class_names) if class_names else 0
 logger.log(f" Average per class: {avg_per_class:,}")

 if class_counts:
 min_count = min(class_counts.values())
 max_count = max(class_counts.values())
 logger.log(f" Min per class: {min_count:,}")
 logger.log(f" Max per class: {max_count:,}")

 # Calculate average image size
 if metrics['image_sizes']:
 avg_h = sum(s[0] for s in metrics['image_sizes']) // len(metrics['image_sizes'])
 avg_w = sum(s[1] for s in metrics['image_sizes']) // len(metrics['image_sizes'])
```

```

 logger.log(f" Average image size: {avg_h}x{avg_w}")

Test set analysis
if os.path.exists(CONFIG['test_path']):
 test_files = [f for f in os.listdir(CONFIG['test_path'])
 if f.lower().endswith('.jpg') and '_test' in f]
 valid_test = [f for f in test_files if f not in CONFIG['skip_test_files']]
 metrics['total_test_images'] = len(valid_test)
 logger.log(f"\n Test images: {len(valid_test)}")

Sampling info
if CONFIG['images_per_class']:
 expected_total = CONFIG['images_per_class'] * len(class_names)
 logger.log(f"\n Sampling: {CONFIG['images_per_class']} images per class")
 logger.log(f" Expected total: {expected_total:,} images")
else:
 logger.log(f"\n Using ALL images (~{total_images:,})")

metrics['analysis_time'] = time.time() - start_time
analysis_time = f"{metrics['analysis_time']:.2f}"
logger.log(f"\n Analysis time: {analysis_time}s")

Save metrics
metrics_file = os.path.join(CONFIG['metrics_path'], 'dataset_metrics.json')
with open(metrics_file, 'w') as f:
 # Convert to JSON-serializable
 save_metrics = {k: v for k, v in metrics.items() if k != 'image_sizes'}
 json.dump(save_metrics, f, indent=2)
logger.log(f"\n Metrics saved to: {metrics_file}")

Visualize class distribution (single combined visualization)
visualize_class_distribution(class_counts)

Show sample images from each class (skip 3D distribution - redundant)
visualize_dataset_samples(class_names)

return metrics

def visualize_class_distribution(class_counts):
 """Create INTERACTIVE 3D class distribution visualization with SOLID bars"""
 logger.log("\n Creating interactive 3D class distribution...")
 classes = list(class_counts.keys())
 counts = list(class_counts.values())
 n_classes = len(classes)
 max_count = max(counts) if counts else 1

 # Create 3D bar chart using solid Mesh3d bars
 fig = go.Figure()

 for i, (cls, count) in enumerate(zip(classes, counts)):
 norm = count / max_count if max_count > 0 else 0

 # Color gradient based on count (blue to purple spectrum)
 r = int(50 + 150 * norm)
 g = int(100 + 50 * (1 - norm))
 b = int(200 - 50 * norm)
 color = f'rgb({r}, {g}, {b})'

 # Add solid 3D bar using helper function
 bar = create_3d_bar_mesh(
 x_center=i,
 y_center=0,
 z_height=count,
 bar_width=0.7,
 bar_depth=0.7,
 color=color,
 opacity=0.95,
 name=cls,
 hovertemplate=f'Class {cls}
Images: {count:,}<extra></extra>'
)
 fig.add_trace(bar)

 # Add text labels at top of bars
 fig.add_trace(go.Scatter3d(
 x=list(range(n_classes)),

```

```

y=[0] * n_classes,
z=[c + max_count * 0.03 for c in counts],
mode='text',
text=classes,
textfont=dict(size=12, color='black', family='Arial Black'),
hoverinfo='skip',
showlegend=False
))

fig.update_layout(
 title=dict(
 text='ASL Dataset - 3D Class Distribution (Rotate to explore!)',
 font=dict(size=20, color='#333'),
 x=0.5
),
 scene=dict(
 xaxis=dict(
 title='Class',
 tickvals=list(range(n_classes)),
 ticktext=classes,
 tickangle=45,
 gridcolor='lightgray'
),
 yaxis=dict(
 title='',
 showticklabels=False,
 range=[-1, 1],
 showgrid=False
),
 zaxis=dict(
 title='Number of Images',
 gridcolor='lightgray'
),
 camera=dict(eye=dict(x=1.5, y=1.5, z=0.8)),
 aspectmode='manual',
 aspectratio=dict(x=2, y=0.5, z=1),
 bgcolor='rgba(250,250,250,0.9)'
),
 height=650,
 template='plotly_white',
 margin=dict(l=0, r=0, t=60, b=0)
)

save_path = os.path.join(CONFIG['metrics_path'], 'class_distribution.png')
save_and_show_plotly(fig, save_path, '3D Class Distribution')

def visualize_dataset_samples(class_names):
 """Visualize sample images from each class with interactive Plotly grid"""
 logger.log("\n Creating interactive dataset sample visualization...")
 n_classes = len(class_names)
 n_cols = min(6, n_classes)
 n_rows = (n_classes + n_cols - 1) // n_cols

 # Create Plotly subplots
 fig = make_subplots(
 rows=n_rows, cols=n_cols,
 subplot_titles=[f'Class: {c}' for c in class_names],
 horizontal_spacing=0.02,
 vertical_spacing=0.08
)

 for idx, class_name in enumerate(class_names):
 row = idx // n_cols + 1
 col = idx % n_cols + 1
 class_dir = os.path.join(CONFIG['train_path'], class_name)
 if os.path.exists(class_dir):
 images = [f for f in os.listdir(class_dir)
 if f.lower().endswith(('.jpg', '.jpeg', '.png'))]
 if images:
 img_path = os.path.join(class_dir, images[0])
 img = cv2.imread(img_path)
 if img is not None:
 img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
 fig.add_trace(
 go.Image(z=img_rgb, hovertemplate=f'Class: {class_name}'

```

```
<extra></extra>'),
 row=row, col=col
)

fig.update_layout(
 title=dict(
 text=' Sample Images from Each ASL Class (Click to zoom!)',
 font=dict(size=20, color='#333'),
 x=0.5
),
 height=200 * n_rows + 100,
 showlegend=False,
 template='plotly_white'
)

Hide axes
fig.update_xaxes(showticklabels=False, showgrid=False)
fig.update_yaxes(showticklabels=False, showgrid=False)

save_path = os.path.join(CONFIG['metrics_path'], 'dataset_samples.png')
save_and_show_plotly(fig, save_path, 'Dataset Samples')
```

## 9. ✓ MediaPipe Hand Cropping

---

```
MEDIAPIPE HAND CROPPING
class HandCropper:
 """MediaPipe hand detection with smart cropping
 Note: MediaPipe uses CPU by default. We minimize its impact by:
 1. Caching cropped images
 2. Using batch processing
 3. Running during data prep, not training
 """
 def __init__(self):
 self.mp_hands = mp.solutions.hands
 self.hands = self.mp_hands.Hands(
 static_image_mode=True,
 max_num_hands=1,
 min_detection_confidence=CONFIG['mediapipe_confidence'],
 model_complexity=CONFIG['mediapipe_model_complexity']
)
 self.margin = CONFIG['mediapipe_margin']
 self.stats = {'total': 0, 'success': 0, 'fallback': 0}

 def crop_hand(self, image):
 """Detect and crop hand region"""
 h, w = image.shape[:2]
 self.stats['total'] += 1

 # Add padding for edge detection
 pad = 20
 padded = cv2.copyMakeBorder(image, pad, pad, pad, pad,
 cv2.BORDER_CONSTANT, value=[0, 0, 0])

 # Convert BGR to RGB for MediaPipe
 rgb = cv2.cvtColor(padded, cv2.COLOR_BGR2RGB)
 results = self.hands.process(rgb)

 if results.multi_hand_landmarks:
 landmarks = results.multi_hand_landmarks[0]
 h_pad, w_pad = padded.shape[:2]

 x_coords = [int(lm.x * w_pad) for lm in landmarks.landmark]
 y_coords = [int(lm.y * h_pad) for lm in landmarks.landmark]

 x_min = max(0, min(x_coords) - self.margin)
 x_max = min(w_pad, max(x_coords) + self.margin)
 y_min = max(0, min(y_coords) - self.margin)
 y_max = min(h_pad, max(y_coords) + self.margin)

 if x_max > x_min and y_max > y_min:
 cropped = padded[y_min:y_max, x_min:x_max]
 if cropped.shape[0] > 50 and cropped.shape[1] > 50:
 self.stats['success'] += 1
 return cropped, True

 # Fallback: center crop
 self.stats['fallback'] += 1
 margin_h, margin_w = int(h * 0.1), int(w * 0.1)
 return image[margin_h:h-margin_h, margin_w:w-margin_w], False

 def get_stats(self):
 """Return detection statistics"""
 total = self.stats['total']
 if total == 0:
 return self.stats
 self.stats['success_rate'] = self.stats['success'] / total * 100
 self.stats['fallback_rate'] = self.stats['fallback'] / total * 100
 return self.stats

 def close(self):
 """Clean up MediaPipe resources"""
 if hasattr(self, 'hands') and self.hands:
 self.hands.close()

def process_dataset_with_mediapipe():
 """Process and cache dataset with MediaPipe hand cropping
```

NOTE: MediaPipe uses TensorFlow Lite which runs on CPU only.  
This is expected behavior - GPU is used for model training, not MediaPipe.  
We optimize by caching results so this only runs once.  
"""

```
logger.section("MEDIAPIPE HAND CROPPING")
logger.log(" NOTE: MediaPipe uses TFLite (CPU-only). This is normal.")
logger.log(" GPU will be used for model training after preprocessing")
logger.log("")
```

```
Check if cached crops exist
```

```
if CONFIG['use_cached_crops']:
 train_exists = os.path.exists(CONFIG['cropped_train_path'])
 has_classes = False
 if train_exists:
 subdirs = [d for d in os.listdir(CONFIG['cropped_train_path'])
 if os.path.isdir(os.path.join(CONFIG['cropped_train_path'], d))]
 has_classes = len(subdirs) > 0

 if has_classes:
 class_names = sorted([d for d in subdirs if d not in CONFIG['skip_folders']])
 logger.log(f" Using cached crops from: {CONFIG['cropped_train_path']}")
 logger.log(f" Found {len(class_names)} classes")
```

```
Count cached images
```

```
total_cached = 0
```

```
for cls in class_names:
```

```
 cls_dir = os.path.join(CONFIG['cropped_train_path'], cls)
```

```
 if os.path.exists(cls_dir):
```

```
 total_cached += len([f for f in os.listdir(cls_dir) if
```

```
f.endswith(('.jpg', '.jpeg', '.png'))])
```

```
 logger.log(f" Total cached images: {total_cached:,}")
```

```
 return class_names
```

```
Check source exists
```

```
if not os.path.exists(CONFIG['train_path']):
```

```
 logger.log(f" Training path not found: {CONFIG['train_path']}")
```

```
 return []
```

```
cropper = HandCropper()
```

```
Get valid classes
```

```
all_folders = [d for d in os.listdir(CONFIG['train_path'])
```

```
 if os.path.isdir(os.path.join(CONFIG['train_path'], d))]
```

```
class_names = sorted([c for c in all_folders if c not in CONFIG['skip_folders']])
```

```
logger.log(f"\n Processing {len(class_names)} classes...")
```

```
logger.log(f" Images per class: {CONFIG['images_per_class'] or 'ALL'}")
```

```
Calculate total work
```

```
total_images_to_process = 0
```

```
for class_name in class_names:
```

```
 src_dir = os.path.join(CONFIG['train_path'], class_name)
```

```
 all_images = [f for f in os.listdir(src_dir) if f.lower().endswith(('.jpg',
```

```
'.jpeg', '.png'))]
```

```
 if CONFIG['images_per_class']:
```

```
 total_images_to_process += min(len(all_images), CONFIG['images_per_class'])
```

```
 else:
```

```
 total_images_to_process += len(all_images)
```

```
logger.log(f" Total images to process: {total_images_to_process:,}")
```

```
est_time = total_images_to_process * 0.03 / 60
```

```
logger.log(f" Estimated time: ~{est_time:.1f} minutes")
```

```
Process training data with detailed progress
```

```
logger.log("\n Processing TRAINING data...")
```

```
processing_stats = {
```

```
 'total_processed': 0,
```

```
 'successful': 0,
```

```
 'fallback': 0,
```

```
 'failed': 0,
```

```
 'per_class': {}
```

```
}
```

```
start_time = time.time()
```

```
for class_idx, class_name in enumerate(class_names):
```

```
 class_start = time.time()
```



```

src_dir = os.path.join(CONFIG['train_path'], class_name)
dst_dir = os.path.join(CONFIG['cropped_train_path'], class_name)
os.makedirs(dst_dir, exist_ok=True)

all_images = [f for f in os.listdir(src_dir)
 if f.lower().endswith(('.jpg', '.jpeg', '.png'))]

Sample images if configured
if CONFIG['images_per_class'] and len(all_images) > CONFIG['images_per_class']:
 np.random.seed(42)
 images = list(np.random.choice(all_images, CONFIG['images_per_class'],
replace=False))
else:
 images = all_images

saved_count = 0
for img_name in images:
 img_path = os.path.join(src_dir, img_name)
 img = cv2.imread(img_path)
 if img is None:
 continue

 cropped, success = cropper.crop_hand(img)

 # Skip fallback crops if required
 if CONFIG['require_mediapipe_detection'] and not success:
 continue

 try:
 cropped_resized = cv2.resize(cropped, CONFIG['img_size'])
 save_path = os.path.join(dst_dir, img_name)
 cv2.imwrite(save_path, cropped_resized)
 saved_count += 1
 if success:
 processing_stats['successful'] += 1
 else:
 processing_stats['fallback'] += 1
 except Exception as e:
 processing_stats['failed'] += 1
 continue

processing_stats['total_processed'] += len(images)
processing_stats['per_class'][class_name] = saved_count

Print per-class progress
class_time = time.time() - class_start
elapsed = time.time() - start_time
remaining_classes = len(class_names) - (class_idx + 1)
eta = (elapsed / (class_idx + 1)) * remaining_classes if class_idx > 0 else 0

print(f" [{class_idx+1:2d}/{len(class_names)}] {class_name}: {saved_count:,} saved
| "
 f"Time: {class_time:.1f}s | ETA: {eta/60:.1f}min", flush=True)

Also save some to test set
if CONFIG['test_images_per_class_from_train'] > 0:
 remaining = [f for f in all_images if f not in images]
 if remaining:
 n_take = min(CONFIG['test_images_per_class_from_train'], len(remaining))
 np.random.seed(42)
 test_images = list(np.random.choice(remaining, n_take, replace=False))
 dst_test_dir = os.path.join(CONFIG['cropped_test_path'], class_name)
 os.makedirs(dst_test_dir, exist_ok=True)

 for img_name in test_images:
 img_path = os.path.join(src_dir, img_name)
 img = cv2.imread(img_path)
 if img is None:
 continue

 cropped, success = cropper.crop_hand(img)
 if CONFIG['require_mediapipe_detection'] and not success:
 continue

 try:

```



```

 cropped_resized = cv2.resize(cropped, CONFIG['img_size'])
 save_path = os.path.join(dst_test_dir, img_name)
 cv2.imwrite(save_path, cropped_resized)
 except:
 continue

Print processing summary
total_time = time.time() - start_time
logger.log(f"\n{'='*60}")
logger.log(f" MEDIAPIPE PROCESSING COMPLETE")
logger.log(f"\n{'='*60}")
total_time_min = total_time/60
logger.log(f" Total time: {total_time_min:.1f} minutes")
total_processed = processing_stats['total_processed']
successful = processing_stats['successful']
fallback = processing_stats['fallback']
failed = processing_stats['failed']
processing_speed = total_processed/total_time
logger.log(f" Images processed: {total_processed:,}")
logger.log(f" Successful detections: {successful:,}")
logger.log(f" Fallback crops: {fallback:,}")
logger.log(f" Failed: {failed:,}")
logger.log(f" Processing speed: {processing_speed:.1f} images/sec")

Per-class summary
logger.log(f"\n Per-Class Results:")
logger.log(f"\n{'-'*40}")
for cls, count in processing_stats['per_class'].items():
 logger.log(f" {cls}: {count:,} images saved")
logger.log(f"\n{'-'*40}")

Save processing stats
stats_path = os.path.join(CONFIG['metrics_path'], 'mediapipe_processing_stats.json')
with open(stats_path, 'w') as f:
 json.dump(processing_stats, f, indent=2)
logger.log(f"\n Processing stats saved to: {stats_path}")

Process test data from test folder
if os.path.exists(CONFIG['test_path']):
 logger.log("\n Processing TEST data...")
 test_files = [f for f in os.listdir(CONFIG['test_path'])
 if f.lower().endswith('.jpg') and '_test' in f]
 test_saved = 0
 for test_file in tqdm(test_files, desc="Test images"):
 if test_file in CONFIG['skip_test_files']:
 continue

 class_name = test_file.split('_test')[0]
 if class_name not in class_names:
 continue

 src_path = os.path.join(CONFIG['test_path'], test_file)
 img = cv2.imread(src_path)
 if img is None:
 continue

 cropped, success = cropper.crop_hand(img)
 if CONFIG['require_mediapipe_detection'] and not success:
 continue

 try:
 cropped_resized = cv2.resize(cropped, CONFIG['img_size'])
 dst_dir = os.path.join(CONFIG['cropped_test_path'], class_name)
 os.makedirs(dst_dir, exist_ok=True)
 save_path = os.path.join(dst_dir, test_file)
 cv2.imwrite(save_path, cropped_resized)
 test_saved += 1
 except:
 continue

 logger.log(f"\n Test images saved: {test_saved}")

Print statistics
stats = cropper.get_stats()
logger.log(f"\n MediaPipe Detection Statistics:")

```

```
logger.log(f" Total processed: {stats['total']:,}")
success_rate = f"{stats.get('success_rate', 0):.1f}%"
fallback_rate = f"{stats.get('fallback_rate', 0):.1f}%"
logger.log(f" Successful detections: {stats['success']:,} ({success_rate})")
logger.log(f" Fallback crops: {stats['fallback']:,} ({fallback_rate})")

cropper.close()

Save sample visualization
visualize_cropped_samples(class_names)

Visualize processing statistics
visualize_mediapipe_stats(processing_stats, class_names)

return class_names
```

## 10. ✓ Data Loading and Augmentation

---

```
DATA LOADING
class AugmentedDataGenerator(keras.utils.Sequence):
 """Custom data generator with model-specific preprocessing"""
 def __init__(self, image_paths, labels, batch_size, model_name,
 augmentation=None, shuffle=True):
 self.image_paths = image_paths
 self.labels = labels
 self.batch_size = batch_size
 self.model_name = model_name
 self.augmentation = augmentation
 self.shuffle = shuffle
 self.indices = np.arange(len(self.image_paths))

 # Set preprocessing function based on model
 if model_name == 'EfficientNetV2B3':
 self.preprocess_fn = effnet_preprocess
 elif model_name == 'ResNet50':
 self.preprocess_fn = resnet_preprocess
 elif model_name == 'InceptionV3':
 self.preprocess_fn = inception_preprocess
 else:
 self.preprocess_fn = None

 self.on_epoch_end()

 def __len__(self):
 return int(np.ceil(len(self.image_paths) / self.batch_size))

 def __getitem__(self, idx):
 batch_indices = self.indices[idx * self.batch_size:(idx + 1) * self.batch_size]
 batch_paths = [self.image_paths[i] for i in batch_indices]
 batch_labels = [self.labels[i] for i in batch_indices]

 X, y = [], []
 for path, label in zip(batch_paths, batch_labels):
 img = cv2.imread(path)
 if img is None:
 img = np.zeros((CONFIG['img_size'][0], CONFIG['img_size'][1], 3),
 dtype=np.uint8)
 else:
 img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

 # Apply augmentation
 if self.augmentation:
 try:
 img = self.augmentation(image=img)['image']
 except:
 pass

 # Ensure correct size
 img = cv2.resize(img, CONFIG['img_size'])

 # Apply model-specific preprocessing
 if self.preprocess_fn:
 img = self.preprocess_fn(img.astype(np.float32))
 else:
 img = img.astype(np.float32) / 255.0

 X.append(img)
 y.append(label)

 return np.array(X), np.array(y)

 def on_epoch_end(self):
 if self.shuffle:
 np.random.shuffle(self.indices)

def load_dataset(cropped_path, class_names):
 """Load cropped dataset"""
 class_to_idx = {name: idx for idx, name in enumerate(class_names)}
 image_paths, labels = [], []
```

```

for class_name in class_names:
 class_dir = os.path.join(cropped_path, class_name)
 if not os.path.exists(class_dir):
 continue

 images = [f for f in os.listdir(class_dir)
 if f.lower().endswith(('.jpg', '.jpeg', '.png'))]
 for img_name in images:
 image_paths.append(os.path.join(class_dir, img_name))
 labels.append(class_to_idx[class_name])

return np.array(image_paths), np.array(labels)

```

## 11. ✓ Augmentation Pipelines

---

```

AUGMENTATION PIPELINES
def get_augmentation_pipeline(model_name):
 """Model-specific augmentation pipelines"""
 if model_name == 'ResNet50':
 return A.Compose([
 A.Rotate(limit=25, p=0.8),
 A.Affine(scale=(0.85, 1.15), translate_percent=(-0.15, 0.15),
 rotate=(-20, 20), shear=(-10, 10), p=0.7),
 A.HorizontalFlip(p=0.5),
 A.OneOf([
 A.MotionBlur(blur_limit=5, p=1.0),
 A.GaussianBlur(blur_limit=5, p=1.0),
], p=0.4),
 A.RandomBrightnessContrast(brightness_limit=0.25, contrast_limit=0.2, p=0.6),
 A.CLAHE(clip_limit=2.0, p=0.3),
 A.GaussNoise(var_limit=(10.0, 40.0), p=0.25),
 A.CoarseDropout(num_holes_range=(1, 3), hole_height_range=(8, 16),
 hole_width_range=(8, 16), p=0.25),
])
 elif model_name == 'EfficientNetV2B3':
 return A.Compose([
 A.Rotate(limit=30, p=0.9),
 A.Affine(scale=(0.80, 1.2), translate_percent=(-0.12, 0.12),
 rotate=(-18, 18), p=0.7),
 A.HorizontalFlip(p=0.5),
 A.OneOf([
 A.GaussianBlur(blur_limit=3, p=1.0),
 A.MotionBlur(blur_limit=3, p=1.0),
], p=0.35),
 A.RandomBrightnessContrast(brightness_limit=0.25, contrast_limit=0.2, p=0.6),
 A.CLAHE(clip_limit=2.0, p=0.3),
 A.GaussNoise(var_limit=(5.0, 30.0), p=0.2),
 A.CoarseDropout(num_holes_range=(1, 4), hole_height_range=(6, 20),
 hole_width_range=(6, 20), p=0.25),
])
 elif model_name == 'InceptionV3':
 return A.Compose([
 A.Rotate(limit=15, p=0.6),
 A.Affine(scale=(0.9, 1.1), translate_percent=(-0.1, 0.1),
 rotate=(-10, 10), p=0.5),
 A.RandomBrightnessContrast(brightness_limit=0.15, contrast_limit=0.15, p=0.5),
 A.CLAHE(clip_limit=1.5, p=0.2),
])

return None

```

## 12. ✓ Model Builders

---

```
MODEL BUILDERS
def build_resnet50(num_classes):
 """ResNet50 with fine-tuning"""
 base = ResNet50(weights='imagenet', include_top=False,
 input_shape=(*CONFIG['img_size'], 3))

 # Freeze early layers
 for layer in base.layers[:-30]:
 layer.trainable = False

 model = models.Sequential([
 base,
 layers.GlobalAveragePooling2D(),
 layers.BatchNormalization(),
 layers.Dropout(0.5),
 layers.Dense(512, activation='relu', kernel_regularizer=keras.regularizers.l2(1e-
4)),
 layers.BatchNormalization(),
 layers.Dropout(0.4),
 layers.Dense(256, activation='relu', kernel_regularizer=keras.regularizers.l2(1e-
4)),
 layers.Dropout(0.3),
 layers.Dense(num_classes, activation='softmax')
], name='ResNet50_AS1')

 return model

def build_efficientnet(num_classes):
 """EfficientNetV2B3 with fine-tuning"""
 try:
 base = EfficientNetV2B3(weights='imagenet', include_top=False,
 input_shape=(*CONFIG['img_size'], 3))
 except:
 base = EfficientNetV2B3(weights=None, include_top=False,
 input_shape=(*CONFIG['img_size'], 3))

 # Freeze most of the base model
 for layer in base.layers[:-80]:
 layer.trainable = False

 model = models.Sequential([
 base,
 layers.GlobalAveragePooling2D(),
 layers.BatchNormalization(),
 layers.Dropout(0.5),
 layers.Dense(512, activation='relu', kernel_regularizer=keras.regularizers.l2(1e-
4)),
 layers.BatchNormalization(),
 layers.Dropout(0.35),
 layers.Dense(256, activation='relu', kernel_regularizer=keras.regularizers.l2(1e-
4)),
 layers.Dropout(0.25),
 layers.Dense(num_classes, activation='softmax')
], name='EfficientNetV2B3_AS1')

 return model

def build_inception(num_classes):
 """InceptionV3 with fine-tuning"""
 base = InceptionV3(weights='imagenet', include_top=False,
 input_shape=(*CONFIG['img_size'], 3))

 # Freeze most of the base model
 for layer in base.layers[:-60]:
 layer.trainable = False

 model = models.Sequential([
 base,
 layers.GlobalAveragePooling2D(),
 layers.BatchNormalization(),
 layers.Dropout(0.45),
 layers.Dense(512, activation='relu', kernel_regularizer=keras.regularizers.l2(1e-
```

```
4)),
 layers.BatchNormalization(),
 layers.Dropout(0.35),
 layers.Dense(256, activation='relu', kernel_regularizer=keras.regularizers.l2(1e-
4)),
 layers.Dropout(0.25),
 layers.Dense(num_classes, activation='softmax')
], name='InceptionV3_ASL')

return model

MODEL_BUILDERS = {
 'ResNet50': build_resnet50,
 'EfficientNetV2B3': build_efficientnet,
 'InceptionV3': build_inception
}
```



## 13. ✓ Model Information Collection

---

```
MODEL INFO COLLECTION
def collect_model_info(num_classes):
 """Collect comprehensive information about all models BEFORE training
 This function:
 1. Creates each model architecture
 2. Collects detailed layer info, parameter counts
 3. Tests if existing models can be loaded
 4. Stores all info in a structured format
 5. Saves visualizations of model architectures
 """
 logger.section("MODEL INFORMATION COLLECTION")
 model_info = {}

 for model_name in CONFIG['train_models']:
 logger.log(f"\n{'='*60}")
 logger.log(f"ANALYZING: {model_name}")
 logger.log(f"{'='*60}")

 info = {
 'name': model_name,
 'status': 'unknown',
 'existing_model_path': None,
 'existing_model_found': False,
 'will_resume': False,
 'total_params': 0,
 'trainable_params': 0,
 'non_trainable_params': 0,
 'trainable_ratio': 0,
 'num_layers': 0,
 'base_model_layers': 0,
 'custom_layers': 0,
 'input_shape': None,
 'output_shape': None,
 'layer_summary': [],
 'memory_estimate_mb': 0
 }

 # Check for existing model
 existing_path = CONFIG['existing_models'].get(model_name)
 info['existing_model_path'] = existing_path

 if existing_path and os.path.exists(existing_path):
 info['existing_model_found'] = True
 info['will_resume'] = True
 info['status'] = 'RESUME - Existing model found'
 logger.log(f"\nEXISTING MODEL FOUND: {existing_path}")

 # Try to load and get info from existing model
 try:
 model, format_type = load_model_any_format(existing_path, model_name)
 if model is not None:
 info['total_params'] = model.count_params()
 info['trainable_params'] = sum([tf.size(w).numpy() for w in
model.trainable_weights])
 info['non_trainable_params'] = info['total_params'] -
info['trainable_params']
 info['num_layers'] = len(model.layers)
 info['input_shape'] = str(model.input_shape)
 info['output_shape'] = str(model.output_shape)
 info['model_format'] = format_type

 # If it's SavedModel (inference-only), mark for scratch training
 if format_type == 'savedmodel':
 info['will_resume'] = False
 info['status'] = 'SCRATCH - SavedModel (inference-only)'
 logger.log(f"\nSavedModel format is inference-only, will train
from scratch")

 del model
 keras.backend.clear_session()
 else:
 # format_type contains error message
```



```

 raise Exception(format_type)
 except Exception as e:
 logger.log(f"\n Could not load existing model: {e}")
 info['will_resume'] = False
 info['status'] = f'SCRATCH - Load failed: {str(e)}'
 else:
 info['status'] = 'SCRATCH - No existing model'
 if existing_path:
 logger.log(f"\n NO EXISTING MODEL at: {existing_path}")
 else:
 logger.log(f"\n NO EXISTING MODEL configured for {model_name}")
 logger.log(f" Will train from SCRATCH")

Build fresh model to get architecture info
try:
 model = MODEL_BUILDERS[model_name](num_classes)

 info['total_params'] = model.count_params()
 info['trainable_params'] = sum([tf.size(w).numpy() for w in
model.trainable_weights])
 info['non_trainable_params'] = info['total_params'] - info['trainable_params']
 info['trainable_ratio'] = info['trainable_params'] / info['total_params'] if
info['total_params'] > 0 else 0
 info['num_layers'] = len(model.layers)
 info['input_shape'] = str(model.input_shape)
 info['output_shape'] = str(model.output_shape)

 # Estimate memory (rough: 4 bytes per parameter for float32)
 info['memory_estimate_mb'] = (info['total_params'] * 4) / (1024 * 1024)

 # Count base vs custom layers
 for layer in model.layers:
 if hasattr(layer, 'layers'): # It's a nested model (base)
 info['base_model_layers'] = len(layer.layers)
 else:
 info['custom_layers'] += 1

 # Collect layer summary (last 15 layers)
 for layer in model.layers[-15:]:
 # Safely get output shape (TF 2.18+ compatibility)
 try:
 out_shape = str(layer.output.shape) if hasattr(layer, 'output') and
layer.output is not None else 'N/A'
 except:
 out_shape = 'N/A'

 layer_info = {
 'name': layer.name,
 'type': layer.__class__.__name__,
 'output_shape': out_shape,
 'trainable': layer.trainable,
 'params': layer.count_params()
 }
 info['layer_summary'].append(layer_info)

 # Print detailed info
 logger.log(f"\n Architecture Details:")
 logger.log(f" Input Shape: {info['input_shape']}")
 logger.log(f" Output Shape: {info['output_shape']}")
 logger.log(f" Total Parameters: {info['total_params']:,}")
 logger.log(f" Trainable Params: {info['trainable_params']:,}")
 logger.log(f" Non-trainable Params: {info['non_trainable_params']:,}")
 logger.log(f" Trainable Ratio: {info['trainable_ratio']:.1%}")
 logger.log(f" Total Layers: {info['num_layers']}")
 logger.log(f" Base Model Layers: {info['base_model_layers']}")
 logger.log(f" Custom Layers: {info['custom_layers']}")
 logger.log(f" Est. Memory: {info['memory_estimate_mb']:.1f} MB")

 logger.log(f"\n Layer Summary (last 15):")
 logger.log("-" * 70)
 for li in info['layer_summary']:
 trainable_mark = "✓" if li['trainable'] else "x"
 logger.log(f" [{trainable_mark}] {li['name']:<30} {li['type']:<20}
{li['output_shape']:<20} {li['params']:,} params")
 logger.log("-" * 70)

```

```

 # Cleanup
 del model
 keras.backend.clear_session()
 gc.collect()
 except Exception as e:
 logger.log(f"\n Error building {model_name}: {e}")
 info['status'] = f'ERROR - {str(e)}'

 model_info[model_name] = info

Save model info to JSON
model_info_path = os.path.join(CONFIG['metrics_path'], 'model_info.json')
Convert to JSON-serializable format
save_info = {}
for name, info in model_info.items():
 save_info[name] = {k: v for k, v in info.items()}

with open(model_info_path, 'w') as f:
 json.dump(save_info, f, indent=2, default=str)
logger.log(f"\n Model info saved to: {model_info_path}")

Create comparison visualization
visualize_model_architecture_comparison(model_info)

Print summary table
logger.log(f"\n{'='*80}")
logger.log(f" MODEL SUMMARY TABLE")
logger.log(f"\n{'='*80}")
logger.log(f"{'Model':<20} {'Status':<25} {'Params':>15} {'Trainable':>15}")
logger.log(f"\n{'-'*80}")
for name, info in model_info.items():
 logger.log(f"{name:<20} {info['status']:<25} {info['total_params']:>15,}
{info['trainable_params']:>15,}")
 logger.log(f"\n{'='*80}")

return model_info

```

## 14. ✓ Training Pipeline

---

```
TRAINING
class DetailedMetricsCallback(keras.callbacks.Callback):
 """Custom callback to print detailed metrics during training"""
 def __init__(self, model_name, total_epochs):
 super().__init__()
 self.model_name = model_name
 self.total_epochs = total_epochs
 self.best_val_acc = 0
 self.best_epoch = 0
 self.start_time = None
 self.epoch_times = []

 def on_train_begin(self, logs=None):
 self.start_time = time.time()
 print(f"\n{'='*80}")
 print(f" TRAINING STARTED: {self.model_name}")
 print(f"{'='*80}")

 def on_epoch_begin(self, epoch, logs=None):
 self.epoch_start = time.time()
 print(f"\n Epoch {epoch+1}/{self.total_epochs}")

 def on_epoch_end(self, epoch, logs=None):
 epoch_time = time.time() - self.epoch_start
 self.epoch_times.append(epoch_time)

 # Get metrics
 train_acc = logs.get('accuracy', 0)
 val_acc = logs.get('val_accuracy', 0)
 train_loss = logs.get('loss', 0)
 val_loss = logs.get('val_loss', 0)
 lr = float(keras.backend.get_value(self.model.optimizer.learning_rate))

 # Track best
 if val_acc > self.best_val_acc:
 self.best_val_acc = val_acc
 self.best_epoch = epoch + 1
 best_marker = " NEW BEST!"
 else:
 best_marker = ""

 # Calculate ETA
 avg_epoch_time = sum(self.epoch_times) / len(self.epoch_times)
 remaining_epochs = self.total_epochs - (epoch + 1)
 eta_seconds = avg_epoch_time * remaining_epochs
 eta_str = f"{eta_seconds/60:.1f}min" if eta_seconds < 3600 else f"
{eta_seconds/3600:.1f}h"

 # Print detailed metrics
 print(f" {'-'*60}␣")
 print(f" | Train Acc: {train_acc*100:6.2f}% | Val Acc: {val_acc*100:6.2f}% |
{best_marker}")
 print(f" | Train Loss: {train_loss:6.4f} | Val Loss: {val_loss:6.4f} |")
 print(f" | LR: {lr:.2e} | Time: {epoch_time:.1f}s | ETA: {eta_str:<8} |")
 print(f" | Best Val Acc: {self.best_val_acc*100:.2f}% @ Epoch {self.best_epoch} |")
 print(f" {'-'*60}␣")

 def on_train_end(self, logs=None):
 total_time = time.time() - self.start_time
 print(f"\n{'='*80}")
 print(f" TRAINING COMPLETE: {self.model_name}")
 print(f"{'='*80}")
 print(f" Total time: {total_time/60:.1f} minutes")
 print(f" Best val accuracy: {self.best_val_acc*100:.2f}% @ Epoch
{self.best_epoch}")
 avg_epoch_time = sum(self.epoch_times)/len(self.epoch_times) if self.epoch_times
else 0
 print(f" Average epoch time: {avg_epoch_time:.1f} seconds")
 print(f"{'='*80}\n")

def train_model(model_name, train_paths, train_labels, val_paths, val_labels,
 num_classes, class_names, force_scratch=False, training_mode='AUTO'):
```

```

"""Train or resume training for a model with intelligent mode selection
Args:
 training_mode: One of 'AUTO', 'SCRATCH', 'RESUME', 'FINE_TUNE'
 - AUTO: Automatically decide based on existing model
 - SCRATCH: Train new model from scratch
 - RESUME: Resume training from checkpoint (moderate epochs, low LR)
 - FINE_TUNE: Light fine-tuning (few epochs, very low LR)
"""
logger.section(f"TRAINING: {model_name}")
logger.log(f" Training mode: {training_mode}")

Handle force_scratch legacy parameter
if force_scratch:
 training_mode = 'SCRATCH'

Check for existing model (unless forced to scratch)
if training_mode == 'SCRATCH':
 logger.log(f"\n FORCED training from SCRATCH (ignoring existing model)")
 existing_model, model_path = None, None
else:
 existing_model, model_path = check_existing_model(model_name)

Determine training parameters based on mode
if training_mode == 'FINE_TUNE' and existing_model:
 # Fine-tuning: very few epochs, minimal learning rate
 logger.log(f"\n FINE-TUNING from: {model_path}")
 model = existing_model
 epochs = CONFIG.get('fine_tune_epochs', 5) # Very few epochs
 initial_lr = 5e-6 # Very low LR for fine-tuning
 is_resume = True
 logger.log(f" • Fine-tune epochs: {epochs}")
 logger.log(f" • Fine-tune LR: {initial_lr}")
elif training_mode == 'RESUME' and existing_model:
 # Resume: moderate epochs, low learning rate
 logger.log(f"\n RESUMING training from: {model_path}")
 model = existing_model
 epochs = CONFIG['resume_epochs']
 initial_lr = 1e-5 # Low LR for continued training
 is_resume = True
elif existing_model and training_mode == 'AUTO':
 # Auto mode with existing model - treat as resume
 logger.log(f"\n AUTO mode: RESUMING from: {model_path}")
 model = existing_model
 epochs = CONFIG['resume_epochs']
 initial_lr = 1e-5
 is_resume = True
else:
 # Scratch training (new model or forced)
 logger.log(f"\n Training from SCRATCH")
 if training_mode not in ['SCRATCH', 'AUTO']:
 logger.log(f"\n No existing model found - falling back to SCRATCH")
 if training_mode == 'AUTO':
 logger.log(f"\n No existing model found at:
{CONFIG['existing_models'].get(model_name, 'N/A')}")
 model = MODEL_BUILDERS[model_name](num_classes)
 epochs = CONFIG['epochs']
 initial_lr = 1e-3
 is_resume = False

Compile model
optimizer = keras.optimizers.Adam(learning_rate=initial_lr)
model.compile(
 optimizer=optimizer,
 loss='sparse_categorical_crossentropy',
 metrics=['accuracy']
)

Print detailed model info
total_params = model.count_params()
trainable_params = sum([tf.size(w).numpy() for w in model.trainable_weights])
non_trainable_params = total_params - trainable_params
logger.log(f"\n{'=' * 60}")
logger.log(f" MODEL ARCHITECTURE: {model.name}")
logger.log(f"{'=' * 60}")
logger.log(f" Total parameters: {total_params:,}")

```

```

logger.log(f" Trainable parameters: {trainable_params:,}")
logger.log(f" Non-trainable params: {non_trainable_params:,}")
trainable_ratio = trainable_params/total_params*100 if total_params > 0 else 0
logger.log(f" Trainable ratio: {trainable_ratio:.1f}%")
logger.log(f"{'=' * 60}")
logger.log(f" Epochs: {epochs}")
logger.log(f" Initial LR: {initial_lr}")
logger.log(f" Batch size: {CONFIG['batch_size']}")
logger.log(f" Training samples: {len(train_paths)}")
logger.log(f" Validation samples: {len(val_paths)}")
steps_per_epoch = len(train_paths) // CONFIG['batch_size']
logger.log(f" Steps per epoch: {steps_per_epoch}")
logger.log(f"{'=' * 60}")

Print layer summary to terminal
logger.log("\n Layer Summary (last 10):")
logger.log("-" * 60)
for i, layer in enumerate(model.layers[-10:]): # Last 10 layers
 trainable_str = "✓" if layer.trainable else "x"
 # Safely get output shape (TF 2.18+ compatibility)
 try:
 out_shape = layer.output.shape if hasattr(layer, 'output') and layer.output is
not None else 'N/A'
 except:
 out_shape = 'N/A'
 logger.log(f" [{trainable_str}] {layer.name}: {out_shape}")
logger.log("-" * 60)

Create data generators
aug_pipeline = get_augmentation_pipeline(model_name)

Preview augmentation
visualize_augmentation_preview(train_paths, train_labels, aug_pipeline, model_name)

train_gen = AugmentedDataGenerator(
 train_paths, train_labels, CONFIG['batch_size'], model_name,
 augmentation=aug_pipeline, shuffle=True
)
val_gen = AugmentedDataGenerator(
 val_paths, val_labels, CONFIG['batch_size'], model_name,
 augmentation=None, shuffle=False
)

Callbacks
callbacks = [
 DetailedMetricsCallback(model_name, epochs), # Custom detailed metrics
 ModelCheckpoint(
 os.path.join(CONFIG['models_path'], f'{model_name}_best.keras'),
 save_best_only=True, monitor='val_accuracy', mode='max', verbose=1
),
 EarlyStopping(
 monitor='val_accuracy', patience=8, restore_best_weights=True, verbose=1
),
 ReduceLROnPlateau(
 monitor='val_loss', factor=0.3, patience=4, min_lr=1e-7, verbose=1
),
 keras.callbacks.TerminateOnNaN()
]

Train
logger.log(f"\n Starting training...")
logger.log(f" Using GPU: {HAS_GPU}")
start_time = time.time()

try:
 history = model.fit(
 train_gen,
 validation_data=val_gen,
 epochs=epochs,
 callbacks=callbacks,
 verbose=1
)
except Exception as e:
 logger.log(f"\n Training failed: {e}")
 # Save crash model

```

```
crash_path = os.path.join(CONFIG['models_path'], f'{model_name}_crash.keras')
try:
 model.save(crash_path)
 logger.log(f"\n Crash model saved to: {crash_path}")
except:
 pass
raise

training_time = time.time() - start_time

Save final model
final_path = os.path.join(CONFIG['models_path'], f'{model_name}_final.keras')
model.save(final_path)
logger.log(f"\n Final model saved to: {final_path}")
logger.log(f" Training time: {training_time/60:.1f} minutes")

Plot training history
plot_training_history(history, model_name)

return model, history
```



## 15. ✓ Evaluation and Visualization

---

```
EVALUATION
def evaluate_model(model, model_name, test_paths, test_labels, class_names):
 """Comprehensive model evaluation with ALL metrics"""
 logger.section(f"EVALUATING: {model_name}")

 # Set preprocessing function
 if model_name == 'EfficientNetV2B3':
 preprocess_fn = effnet_preprocess
 elif model_name == 'ResNet50':
 preprocess_fn = resnet_preprocess
 elif model_name == 'InceptionV3':
 preprocess_fn = inception_preprocess
 else:
 preprocess_fn = None

 logger.log(f" Using preprocessing: {preprocess_fn.__name__ if preprocess_fn else
'None'}")

 # Load ALL test data
 X_test, y_test = [], []
 logger.log(f" Loading ALL {len(test_paths)} test samples...")
 for path, label in zip(test_paths, test_labels):
 img = cv2.imread(path)
 if img is None:
 continue
 img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
 img = cv2.resize(img, CONFIG['img_size'])
 if preprocess_fn:
 img = preprocess_fn(img.astype(np.float32))
 else:
 img = img.astype(np.float32) / 255.0
 X_test.append(img)
 y_test.append(label)

 X_test = np.array(X_test)
 y_test = np.array(y_test)

 logger.log(f" Test samples: {len(X_test)}")
 logger.log(f" Data shape: {X_test.shape}")
 logger.log(f" Data range: [{X_test.min():.2f}, {X_test.max():.2f}]")

 if len(X_test) == 0:
 logger.log("\n No test samples - skipping evaluation")
 return None

 # Predictions
 y_pred_probs = model.predict(X_test, verbose=0)
 y_pred = np.argmax(y_pred_probs, axis=1)

 # COMPREHENSIVE METRICS
 # Basic metrics
 accuracy = accuracy_score(y_test, y_pred)
 precision = precision_score(y_test, y_pred, average='weighted', zero_division=0)
 recall = recall_score(y_test, y_pred, average='weighted', zero_division=0)
 f1 = f1_score(y_test, y_pred, average='weighted', zero_division=0)

 # Advanced metrics
 balanced_acc = balanced_accuracy_score(y_test, y_pred)
 cohen_kappa = cohen_kappa_score(y_test, y_pred)
 try:
 mcc = matthews_corrcoef(y_test, y_pred)
 except:
 mcc = 0.0

 # Multi-class ROC-AUC (one-vs-rest)
 try:
 roc_auc = roc_auc_score(y_test, y_pred_probs, multi_class='ovr')
 except:
 roc_auc = None

 # Log loss
 try:
```



```

 logloss = log_loss(y_test, y_pred_probs)
 except:
 logloss = None

Per-class metrics (ALL classes)
precision_per_class = precision_score(y_test, y_pred, average=None, zero_division=0)
recall_per_class = recall_score(y_test, y_pred, average=None, zero_division=0)
f1_per_class = f1_score(y_test, y_pred, average=None, zero_division=0)

Macro averages
precision_macro = precision_score(y_test, y_pred, average='macro', zero_division=0)
recall_macro = recall_score(y_test, y_pred, average='macro', zero_division=0)
f1_macro = f1_score(y_test, y_pred, average='macro', zero_division=0)

Top-K accuracy
try:
 top3_acc = top_k_accuracy_score(y_test, y_pred_probs, k=3)
 top5_acc = top_k_accuracy_score(y_test, y_pred_probs, k=5)
except:
 top3_acc = None
 top5_acc = None

Confidence analysis
pred_confidences = np.max(y_pred_probs, axis=1)
avg_confidence = np.mean(pred_confidences)
correct_mask = y_pred == y_test
avg_conf_correct = np.mean(pred_confidences[correct_mask]) if np.any(correct_mask) else
0
avg_conf_wrong = np.mean(pred_confidences[~correct_mask]) if np.any(~correct_mask) else
0

Confusion matrix
cm = confusion_matrix(y_test, y_pred)
per_class_acc = cm.diagonal() / cm.sum(axis=1) if cm.sum() > 0 else
np.zeros(len(class_names))

Most confused pairs
cm_no_diag = cm.copy()
np.fill_diagonal(cm_no_diag, 0)
confused_pairs = []
for i in range(len(class_names)):
 for j in range(len(class_names)):
 if cm_no_diag[i, j] > 0:
 confused_pairs.append({
 'true': class_names[i],
 'pred': class_names[j],
 'count': int(cm_no_diag[i, j])
 })
confused_pairs.sort(key=lambda x: x['count'], reverse=True)

DISPLAY COMPREHENSIVE RESULTS
logger.log(f"\n{'='*70}")
logger.log(f" COMPREHENSIVE EVALUATION RESULTS for {model_name}")
logger.log(f"{'='*70}")
logger.log(f"\n OVERALL METRICS:")
logger.log(f" {'-'*60} ")
logger.log(f" | Accuracy: {accuracy:.4f} ({accuracy*100:.2f}%) |")
logger.log(f" | Balanced Accuracy: {balanced_acc:.4f} ({balanced_acc*100:.2f}%) |")
logger.log(f" | Precision (weighted): {precision:.4f} |")
logger.log(f" | Recall (weighted): {recall:.4f} |")
logger.log(f" | F1-Score (weighted): {f1:.4f} |")
logger.log(f" | F1-Score (macro): {f1_macro:.4f} |")
logger.log(f" | Cohen's Kappa: {cohen_kappa:.4f} |")
logger.log(f" | Matthews CC: {mcc:.4f} |")
if roc_auc:
 logger.log(f" | ROC-AUC (weighted): {roc_auc:.4f} |")
if logloss:
 logger.log(f" | Log Loss: {logloss:.4f} |")
if top3_acc:
 logger.log(f" | Top-3 Accuracy: {top3_acc:.4f} ({top3_acc*100:.2f}%) |")
if top5_acc:
 logger.log(f" | Top-5 Accuracy: {top5_acc:.4f} ({top5_acc*100:.2f}%) |")
logger.log(f" {'-'*60} ")

logger.log(f"\n CONFIDENCE ANALYSIS:")

```

```

logger.log(f" • Average Confidence: {avg_confidence:.4f}")
logger.log(f" • Conf (Correct Preds): {avg_conf_correct:.4f}")
logger.log(f" • Conf (Wrong Preds): {avg_conf_wrong:.4f}")
confidence_gap = avg_conf_correct - avg_conf_wrong
calibration = "(excellent calibration)" if confidence_gap > 0.2 else "(good
calibration)"
logger.log(f" • Confidence Gap: {confidence_gap:.4f} {calibration}")

Classification report (shows ALL classes)
logger.log(f"\n CLASSIFICATION REPORT (ALL {len(class_names)} CLASSES):")
report = classification_report(y_test, y_pred, target_names=class_names, digits=4)
logger.log(report)

Per-class metrics table
logger.log(f"\n PER-CLASS METRICS:")
logger.log(f" {'Class':<8} {'Accuracy':>10} {'Precision':>10} {'Recall':>10} {'F1':>10}
{'Support':>8}")
logger.log(f" {'-'*56}")
for i, cls_name in enumerate(class_names):
 if i < len(per_class_acc):
 support = int(cm.sum(axis=1)[i]) if i < len(cm) else 0
 logger.log(f" {cls_name:<8} {per_class_acc[i]:>10.4f}
{precision_per_class[i]:>10.4f} "
 f"{recall_per_class[i]:>10.4f} {f1_per_class[i]:>10.4f}
{support:>8}")

Most confused pairs
logger.log(f"\n MOST CONFUSED PAIRS (Top 10):")
for pair in confused_pairs[:10]:
 logger.log(f" {pair['true']} → {pair['pred']}: {pair['count']} mistakes")

Save detailed report
report_path = os.path.join(CONFIG['metrics_path'],
f'{model_name}_classification_report.txt')
with open(report_path, 'w') as f:
 f.write(f"{'='*70}\n")
 f.write(f"Model: {model_name}\n")
 f.write(f"{'='*70}\n\n")
 f.write(f"OVERALL METRICS:\n")
 f.write(f" Accuracy: {accuracy:.4f}\n")
 f.write(f" Balanced Accuracy: {balanced_acc:.4f}\n")
 f.write(f" Precision (weighted): {precision:.4f}\n")
 f.write(f" Recall (weighted): {recall:.4f}\n")
 f.write(f" F1-Score (weighted): {f1:.4f}\n")
 f.write(f" F1-Score (macro): {f1_macro:.4f}\n")
 f.write(f" Cohen's Kappa: {cohen_kappa:.4f}\n")
 f.write(f" Matthews CC: {mcc:.4f}\n")
 if roc_auc:
 f.write(f" ROC-AUC: {roc_auc:.4f}\n")
 if logloss:
 f.write(f" Log Loss: {logloss:.4f}\n")
 if top3_acc:
 f.write(f" Top-3 Accuracy: {top3_acc:.4f}\n")
 if top5_acc:
 f.write(f" Top-5 Accuracy: {top5_acc:.4f}\n\n")

 f.write(f"CLASSIFICATION REPORT:\n")
 f.write(f"{report}\n\n")
 f.write(f"MOST CONFUSED PAIRS:\n")
 for pair in confused_pairs[:20]:
 f.write(f" {pair['true']} → {pair['pred']}: {pair['count']}\n")

logger.log(f"\n Detailed report saved to: {report_path}")

3D CONFUSION MATRIX VISUALIZATIONS
Restore original cm (diagonal was modified for confusion analysis)
cm = confusion_matrix(y_test, y_pred)
n_classes = len(class_names)

Create 3D Surface confusion matrix
logger.log(f"\n Creating 3D confusion matrix for {model_name}...")
fig_cm = make_subplots(
 rows=1, cols=2,
 specs=[[{"type": "surface"}, {"type": "surface"}]],
 subplot_titles=['Raw Confusion Matrix', 'Normalized Confusion Matrix'],

```

```

 horizontal_spacing=0.05
)

3D Surface for raw confusion matrix
fig_cm.add_trace(
 go.Surface(
 z=cm,
 x=list(range(n_classes)),
 y=list(range(n_classes)),
 colorscale='Blues',
 showscale=True,
 colorbar=dict(title='Count', x=0.45, len=0.8),
 hovertemplate='True: %{y}
Pred: %{x}
Count: %{z}<extra></extra>',
 contours=dict(
 z=dict(show=True, usecolormap=True, highlightcolor="white", project_x=True)
)
),
 row=1, col=1
)

Normalized confusion matrix
cm_normalized = cm.astype('float') / cm.sum(axis=1)[:, np.newaxis]
cm_normalized = np.nan_to_num(cm_normalized)

fig_cm.add_trace(
 go.Surface(
 z=cm_normalized * 100,
 x=list(range(n_classes)),
 y=list(range(n_classes)),
 colorscale='RdYlGn',
 showscale=True,
 colorbar=dict(title='%', x=1.0, len=0.8),
 hovertemplate='True: %{y}
Pred: %{x}
Rate: %{z:.1f}%<extra></extra>',
 contours=dict(
 z=dict(show=True, usecolormap=True, highlightcolor="white", project_x=True)
)
),
 row=1, col=2
)

Update both scenes
for col in [1, 2]:
 fig_cm.update_scenes(
 dict(
 xaxis=dict(title='Predicted', tickvals=list(range(n_classes)),
 ticktext=class_names),
 yaxis=dict(title='True', tickvals=list(range(n_classes)),
 ticktext=class_names),
 zaxis=dict(title='Count' if col == 1 else 'Rate %'),
 camera=dict(eye=dict(x=1.5, y=-1.5, z=1.0))
),
 row=1, col=col
)

fig_cm.update_layout(
 title=dict(
 text=f' {model_name} - 3D Confusion Matrix (Rotate to explore!)',
 font=dict(size=18, color='#333'),
 x=0.5
),
 height=700,
 template='plotly_white'
)

cm_path = os.path.join(CONFIG['metrics_path'], f'{model_name}_confusion_matrix.png')
save_and_show_plotly(fig_cm, cm_path, f'{model_name} 3D Confusion Matrix')

Return comprehensive metrics
return {
 'accuracy': accuracy,
 'balanced_accuracy': balanced_acc,
 'precision': precision,
 'recall': recall,
 'f1_score': f1,
 'f1_macro': f1_macro,

```

```

 'cohen_kappa': cohen_kappa,
 'matthews_corrcoef': mcc,
 'roc_auc': roc_auc,
 'log_loss': logloss,
 'top3_accuracy': top3_acc,
 'top5_accuracy': top5_acc,
 'avg_confidence': avg_confidence,
 'avg_conf_correct': avg_conf_correct,
 'avg_conf_wrong': avg_conf_wrong
 }

```

## 16. ✓ Final Comparison and Main Pipeline

---

```

FINAL COMPARISON
def plot_model_comparison(results):
 """Plot final model comparison with SOLID 3D bars"""
 logger.section("MODEL COMPARISON")

 # Filter valid results
ASL Alphabet Classification - TODO LIST

1. ✓ Installation and Setup
```python
# INSTALLATION CHECK
import subprocess
import sys

def install_if_missing(package, pip_name=None):
    """Install package if not available"""
    try:
        __import__(package)
    except ImportError:
        pip_name = pip_name or package
        print(f"Installing {pip_name}...")
        subprocess.check_call([sys.executable, '-m', 'pip', 'install', pip_name])

# Check and install dependencies
install_if_missing('mediapipe')
install_if_missing('cv2', 'opencv-python')
install_if_missing('alumentations')
install_if_missing('sklearn', 'scikit-learn')
install_if_missing('tqdm')
install_if_missing('plotly')
install_if_missing('kaleido') # For saving plotly figures

```

2. ✓ Imports and Configuration

```
# IMPORTS
import os
import shutil
import numpy as np
import pandas as pd
import matplotlib
# Check if running in notebook (Kaggle/Jupyter) - use inline display
import sys
IN_NOTEBOOK = 'ipykernel' in sys.modules or 'IPython' in sys.modules
if not IN_NOTEBOOK:
    matplotlib.use('Agg') # Non-interactive backend only if not in notebook
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import seaborn as sns
from pathlib import Path
import cv2
import mediapipe as mp
from tqdm import tqdm
import json
# Plotly for interactive 3D visualizations
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots
import plotly.io as pio
# Set Plotly renderer for Kaggle/Jupyter notebooks
if 'ipykernel' in sys.modules or 'IPython' in sys.modules:
    pio.renderers.default = 'notebook' # Best for Kaggle
# For inline display in Kaggle notebooks
if IN_NOTEBOOK:
    try:
        from IPython.display import display, Image, HTML, clear_output
        from IPython import get_ipython
        get_ipython().run_line_magic('matplotlib', 'inline')
        print(" Notebook detected - plots will display inline!")
    except:
        pass
from collections import Counter
from datetime import datetime
import gc
import time
import warnings
warnings.filterwarnings('ignore')
# TensorFlow imports with GPU configuration
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, models
from tensorflow.keras.applications import ResNet50, EfficientNetV2B3, InceptionV3
from tensorflow.keras.applications.resnet50 import preprocess_input as resnet_preprocess
from tensorflow.keras.applications.efficientnet_v2 import preprocess_input as
effnet_preprocess
from tensorflow.keras.applications.inception_v3 import preprocess_input as
inception_preprocess
from tensorflow.keras.callbacks import ModelCheckpoint, EarlyStopping, ReduceLROnPlateau
import albumentations as A
# Sklearn imports
from sklearn.model_selection import train_test_split
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
from sklearn.metrics import (
    accuracy_score, classification_report, confusion_matrix,
    precision_score, recall_score, f1_score, top_k_accuracy_score,
    balanced_accuracy_score, cohen_kappa_score, matthews_corrcoef,
    roc_auc_score, log_loss, roc_curve, auc, precision_recall_curve,
    average_precision_score
)
from sklearn.preprocessing import label_binarize
# Set seeds for reproducibility
np.random.seed(42)
tf.random.set_seed(42)
```

3. ✓ GPU Configuration

```
# GPU CONFIGURATION
def configure_gpu():
    """Configure GPU for optimal performance, minimize CPU usage"""
    print("\n" + "=" * 70)
    print("GPU CONFIGURATION")
    print("=" * 70)

    # Force single GPU usage
    os.environ["CUDA_VISIBLE_DEVICES"] = "0"
    os.environ["TF_FORCE_GPU_ALLOW_GROWTH"] = "true"

    # Optimize TensorFlow for GPU
    os.environ["TF_GPU_THREAD_MODE"] = "gpu_private"
    os.environ["TF_GPU_THREAD_COUNT"] = "2"

    # Reduce CPU parallelism to minimize CPU usage
    tf.config.threading.set_inter_op_parallelism_threads(2)
    tf.config.threading.set_intra_op_parallelism_threads(2)

    gpus = tf.config.list_physical_devices('GPU')
    if gpus:
        try:
            # Use only first GPU
            tf.config.set_visible_devices(gpus[0], 'GPU')
            # Enable memory growth to avoid OOM
            tf.config.experimental.set_memory_growth(gpus[0], True)
            print(f" GPU detected: {gpus[0].name}")
            print(f" Memory growth enabled")
            print(f" Using GPU 0 only")

            # Get GPU details
            try:
                gpu_details = tf.config.experimental.get_device_details(gpus[0])
                if gpu_details:
                    print(f" GPU Details: {gpu_details}")
            except:
                pass
            return True
        except RuntimeError as e:
            print(f" GPU configuration error: {e}")
            return False
    else:
        print(" No GPU detected - using CPU (training will be slower)")
        return False

# Configure GPU at import time
HAS_GPU = configure_gpu()
print(f"\n TensorFlow Version: {tf.__version__}")
print(f" GPU Available: {HAS_GPU}")
print(f" Physical Devices: {tf.config.list_physical_devices()}")
```


4. ✓ Dataset Configuration

```
# CONFIGURATION
CONFIG = {
    # PATHS (KAGGLE ENVIRONMENT)
    # Dataset paths for Kaggle
    'train_path': '/kaggle/input/aslamerican-sign-language-alphabet-
dataset/ASL_Alphabet_Dataset/asl_alphabet_train',
    'test_path': '/kaggle/input/aslamerican-sign-language-alphabet-
dataset/ASL_Alphabet_Dataset/asl_alphabet_test',
    'cropped_train_path': '/kaggle/working/cropped_train', # MediaPipe cropped training
images
    'cropped_test_path': '/kaggle/working/cropped_test', # MediaPipe cropped test images
    'models_path': '/kaggle/working/models', # Saved models
    'metrics_path': '/kaggle/working/metrics', # Saved metrics and graphs

    # MODEL PATHS FOR RESUME
    # Set these to your existing model paths for resume training
    # If None or file doesn't exist, will train from scratch
    'existing_models': {
        'ResNet50': '/kaggle/input/resnet50-best-1-keras/keras/1/2',
        'InceptionV3': '/kaggle/input/inceptionv3-best-
keras/keras/1/1/InceptionV3_best.keras',
        'EfficientNetV2B3': '/kaggle/input/efficientnetv2b3-best-1-
keras/keras/1/1/EfficientNetV2B3_best.keras'
    },

    # DATASET CONFIG
    'skip_folders': ['del', 'nothing', 'space'],
    'skip_test_files': ['nothing_test.jpg', 'del_test.jpg', 'space_test.jpg'],
    'images_per_class': 1000, # Set to None for ALL images
    'test_images_per_class_from_train': 100,
    'require_mediapipe_detection': True,
    'use_cached_crops': True, # Skip MediaPipe if crops exist

    # TRAINING CONFIG
    'img_size': (224, 224),
    'batch_size': 32,
    'epochs': 35,
    'val_split': 0.15,
    'resume_epochs': 15, # Additional epochs when resuming
    'fine_tune_epochs': 5, # Few epochs for fine-tuning already-good models

    # MODELS TO TRAIN
    # Only these models will be trained (in order)
    # Include ALL models you want to train - they will be trained in this order
    'train_models': ['EfficientNetV2B3', 'ResNet50', 'InceptionV3'],

    # MEDIAPIPE CONFIG
    'mediapipe_confidence': 0.6,
    'mediapipe_margin': 30,
    'mediapipe_model_complexity': 1, # 0=lite, 1=full
}

# Create directories
for path_key in ['cropped_train_path', 'cropped_test_path', 'models_path', 'metrics_path']:
    os.makedirs(CONFIG[path_key], exist_ok=True)
```


5. ✓ Logging and Visualization Helpers

```
# LOGGING UTILITIES
class Logger:
    """Simple logger that prints to terminal and saves to file"""
    def __init__(self, log_file=None):
        self.log_file = log_file or os.path.join(CONFIG['metrics_path'],
'training_log.txt')
        self.start_time = datetime.now()
        # Clear previous log
        with open(self.log_file, 'w') as f:
            f.write(f"ASL Training Log - {self.start_time.strftime('%Y-%m-%d
%H:%M:%S')}\n")
            f.write("=" * 70 + "\n")

    def log(self, message, also_print=True):
        """Log message to file and optionally print"""
        timestamp = datetime.now().strftime('%H:%M:%S')
        formatted = f"[{timestamp}] {message}"
        if also_print:
            print(message)
        with open(self.log_file, 'a') as f:
            f.write(formatted + "\n")

    def section(self, title):
        """Print section header"""
        header = "\n" + "=" * 70 + f"\n{title}\n" + "=" * 70 + "\n"
        self.log(header)

logger = Logger()
```

6. ✓ 3D Visualization Helpers

```
# 3D BAR HELPER FUNCTION
def create_3d_bar_mesh(x_center, y_center, z_height, bar_width=0.4, bar_depth=0.4,
                       color='blue', opacity=1.0, name='', hovertemplate=''):
    """
    Create a solid 3D bar using Mesh3d with proper triangular faces.
    This creates a proper rectangular prism (cuboid) with all 6 faces rendered correctly.
    Each face is made of 2 triangles = 12 triangles total for a closed solid bar.
    Args:
        x_center: X position of bar center
        y_center: Y position of bar center
        z_height: Height of bar (from 0)
        bar_width: Width in X direction
        bar_depth: Depth in Y direction
        color: Bar color
        opacity: Bar opacity (0-1)
        name: Name for legend
        hovertemplate: Hover text template
    Returns:
        go.Mesh3d trace for the bar
    """

    # Half dimensions
    hw = bar_width / 2 # half width
    hd = bar_depth / 2 # half depth

    # 8 vertices of the cuboid (bar)
    # Bottom face (z=0): vertices 0,1,2,3
    # Top face (z=height): vertices 4,5,6,7
    vertices_x = [
        x_center - hw, x_center + hw, x_center + hw, x_center - hw, # bottom: 0,1,2,3
        x_center - hw, x_center + hw, x_center + hw, x_center - hw # top: 4,5,6,7
    ]
    vertices_y = [
        y_center - hd, y_center - hd, y_center + hd, y_center + hd, # bottom
        y_center - hd, y_center - hd, y_center + hd, y_center + hd # top
    ]
    vertices_z = [
        0, 0, 0, 0, # bottom face at z=0
        z_height, z_height, z_height, z_height # top face at z=height
    ]

    # 12 triangular faces (2 per face × 6 faces)
    # Each face needs 2 triangles defined by vertex indices (i, j, k)
    # The vertices must be in counter-clockwise order when viewed from outside

    # Bottom face (z=0): vertices 0,1,2,3 - looking from below
    # Top face (z=h): vertices 4,5,6,7 - looking from above
    # Front face (y=-): vertices 0,1,5,4
    # Back face (y=+): vertices 2,3,7,6
    # Left face (x=-): vertices 0,3,7,4
    # Right face (x=+): vertices 1,2,6,5

    i_faces = [
        0, 0, # bottom: triangles (0,1,2) and (0,2,3)
        4, 4, # top: triangles (4,6,5) and (4,7,6)
        0, 0, # front: triangles (0,5,1) and (0,4,5)
        2, 2, # back: triangles (2,7,3) and (2,6,7)
        0, 0, # left: triangles (0,3,7) and (0,7,4)
        1, 1 # right: triangles (1,5,6) and (1,6,2)
    ]

    j_faces = [
        1, 2, # bottom
        6, 7, # top
        5, 4, # front
        7, 6, # back
        3, 7, # left
        5, 6 # right
    ]

    k_faces = [
        2, 3, # bottom
        5, 6, # top
    ]
```

```
1, 5, # front
3, 7, # back
7, 4, # left
6, 2  # right
]

return go.Mesh3d(
    x=vertices_x,
    y=vertices_y,
    z=vertices_z,
    i=i_faces,
    j=j_faces,
    k=k_faces,
    color=color,
    opacity=opacity,
    name=name,
    showlegend=False,
    hovertemplate=hovertemplate,
    flatshading=True, # Better solid appearance
    lighting=dict(
        ambient=0.7,
        diffuse=0.8,
        specular=0.2,
        roughness=0.5,
        fresnel=0.1
    ),
    lightposition=dict(x=100, y=200, z=300)
)
```

7. ✓ SavedModel Loader

```
# SAVEDMODEL LOADER
def load_model_any_format(model_path, model_name=None):
    """
    Load a model from any format (Keras 3 .keras, H5, or TensorFlow SavedModel).
    Args:
        model_path: Path to the model file/directory
        model_name: Optional model name for logging
    Returns:
        (model, format_type) tuple or (None, error_message)
    """
    name = model_name or os.path.basename(model_path)

    # Helper function to find model files in a directory
    def find_model_file(base_path):
        """Search for model files in directory tree"""
        model_extensions = ['.keras', '.h5', '.hdf5']
        savedmodel_markers = ['saved_model.pb', 'saved_model.pbtxt']

        if not os.path.exists(base_path):
            return None, None

        # If it's a file, return it directly
        if os.path.isfile(base_path):
            return base_path, 'file'

        # Check if this directory is a SavedModel
        for marker in savedmodel_markers:
            if os.path.exists(os.path.join(base_path, marker)):
                return base_path, 'savedmodel'

        # Search for model files recursively (max depth 3)
        for root, dirs, files in os.walk(base_path):
            depth = root[len(base_path):].count(os.sep)
            if depth > 3:
                continue

            # Check for SavedModel markers in subdirs
            for marker in savedmodel_markers:
                if marker in files:
                    return root, 'savedmodel'

            # Check for keras/h5 files
            for f in files:
                for ext in model_extensions:
                    if f.endswith(ext):
                        return os.path.join(root, f), 'file'

        return None, None

    # First, try to find the actual model file/directory
    actual_path, path_type = find_model_file(model_path)
    if actual_path and actual_path != model_path:
        logger.log(f" Found model at: {actual_path}")
        model_path = actual_path

    # Log directory contents for debugging
    if os.path.isdir(model_path):
        try:
            contents = os.listdir(model_path)
            logger.log(f" Directory contents: {contents[:10]}{'...' if len(contents) > 10}
        except:
            pass
    else: ''})

    # Try standard Keras load first (.keras or .h5)
    try:
        model = keras.models.load_model(model_path)
        logger.log(f" Loaded {name} using keras.models.load_model()")
        return model, 'keras'
    except Exception as e1:
        keras_error = str(e1)

    # If it's a directory, try multiple approaches for SavedModel format
```

```

if os.path.isdir(model_path):
    # Check for saved_model.pb first
    has_savedmodel_pb = os.path.exists(os.path.join(model_path, 'saved_model.pb'))
    has_savedmodel_pbtxt = os.path.exists(os.path.join(model_path,
'saved_model.pbtxt'))

    if not has_savedmodel_pb and not has_savedmodel_pbtxt:
        # Not a valid SavedModel directory - search subdirectories
        logger.log(f" No saved_model.pb found, searching subdirectories...")
        for subdir in os.listdir(model_path):
            subpath = os.path.join(model_path, subdir)
            if os.path.isdir(subpath):
                if os.path.exists(os.path.join(subpath, 'saved_model.pb')):
                    logger.log(f" Found SavedModel in: {subpath}")
                    model_path = subpath
                    has_savedmodel_pb = True
                    break

        # Check for .keras files
        for f in os.listdir(subpath):
            if f.endswith('.keras') or f.endswith('.h5'):
                keras_file = os.path.join(subpath, f)
                logger.log(f" Found Keras file: {keras_file}")
                try:
                    model = keras.models.load_model(keras_file)
                    logger.log(f" Loaded {name} from {keras_file}")
                    return model, 'keras'
                except Exception as e:
                    logger.log(f" Failed to load {keras_file}: {e}")

# Approach 1: Try tf.saved_model.load (TF2 native)
if has_savedmodel_pb or has_savedmodel_pbtxt:
    try:
        logger.log(f" Trying tf.saved_model.load()...")
        imported = tf.saved_model.load(model_path)

        # Check if it has a keras model signature
        if hasattr(imported, 'signatures'):
            signatures = list(imported.signatures.keys())
            logger.log(f" Found signatures: {signatures}")

            # Get the serving function
            if 'serving_default' in signatures:
                serve_fn = imported.signatures['serving_default']

            # Create a wrapper model for inference
            class SavedModelWrapper(keras.Model):
                def __init__(self, serve_fn, **kwargs):
                    super().__init__(**kwargs)
                    self._serve_fn = serve_fn
                    # Try to get output shape from signature
                    output_info = list(serve_fn.structured_outputs.values())[0]
                    self._output_classes = output_info.shape[-1]

                def call(self, inputs):
                    # The serve function expects a dict with input tensors
                    result = self._serve_fn(inputs)
                    # Return the first output value
                    return list(result.values())[0]

            @property
            def output_shape(self):
                return (None, self._output_classes)

            model = SavedModelWrapper(serve_fn, name=f"{name}_TF2")
            # Build the model with sample input
            model.build(input_shape=(None, 224, 224, 3))
            logger.log(f" Loaded {name} using tf.saved_model.load()")
            logger.log(f" Note: This is inference-only - cannot be fine-tuned")
            return model, 'savedmodel'
    except Exception as e2:
        logger.log(f" tf.saved_model.load failed: {e2}")

# Neither worked
return None, f"Cannot load: {keras_error}"

```

8. ✓ Dataset Analysis

```
# DATASET ANALYSIS
def analyze_dataset():
    """Comprehensive dataset analysis with metrics"""
    logger.section("DATASET ANALYSIS")
    metrics = {
        'total_classes': 0,
        'total_train_images': 0,
        'total_test_images': 0,
        'class_distribution': {},
        'image_sizes': [],
        'analysis_time': None
    }
    start_time = time.time()

    # Check if paths exist
    if not os.path.exists(CONFIG['train_path']):
        logger.log(f" Training path not found: {CONFIG['train_path']}")
        logger.log("Please update CONFIG['train_path'] to point to your dataset")
        return metrics

    # Get all class folders
    all_folders = [d for d in os.listdir(CONFIG['train_path'])
                   if os.path.isdir(os.path.join(CONFIG['train_path'], d))]
    class_names = sorted([c for c in all_folders if c not in CONFIG['skip_folders']])
    metrics['total_classes'] = len(class_names)
    metrics['class_names'] = class_names

    logger.log(f"\n Dataset Path: {CONFIG['train_path']}")
    logger.log(f" Total folders found: {len(all_folders)}")
    logger.log(f" Skipped folders: {CONFIG['skip_folders']}")
    logger.log(f" Valid classes: {len(class_names)}")
    logger.log(f" Classes: {class_names}\n")

    # Analyze each class
    logger.log("Class Distribution:")
    logger.log("-" * 50)
    total_images = 0
    class_counts = {}
    for class_name in class_names:
        class_dir = os.path.join(CONFIG['train_path'], class_name)
        images = [f for f in os.listdir(class_dir)
                  if f.lower().endswith(('.jpg', '.jpeg', '.png'))]
        count = len(images)
        class_counts[class_name] = count
        total_images += count

    # Sample image size
    if images and len(metrics['image_sizes']) < 10:
        sample_path = os.path.join(class_dir, images[0])
        img = cv2.imread(sample_path)
        if img is not None:
            metrics['image_sizes'].append(img.shape[:2])
        logger.log(f" {class_name}: {count:}, {images}")

    metrics['total_train_images'] = total_images
    metrics['class_distribution'] = class_counts

    logger.log("-" * 50)
    logger.log(f"\n Total Training Images: {total_images:,}")
    avg_per_class = total_images // len(class_names) if class_names else 0
    logger.log(f" Average per class: {avg_per_class:,}")

    if class_counts:
        min_count = min(class_counts.values())
        max_count = max(class_counts.values())
        logger.log(f" Min per class: {min_count:,}")
        logger.log(f" Max per class: {max_count:,}")

    # Calculate average image size
    if metrics['image_sizes']:
        avg_h = sum(s[0] for s in metrics['image_sizes']) // len(metrics['image_sizes'])
        avg_w = sum(s[1] for s in metrics['image_sizes']) // len(metrics['image_sizes'])
```



```

        logger.log(f" Average image size: {avg_h}x{avg_w}")

# Test set analysis
if os.path.exists(CONFIG['test_path']):
    test_files = [f for f in os.listdir(CONFIG['test_path'])
                  if f.lower().endswith('.jpg') and '_test' in f]
    valid_test = [f for f in test_files if f not in CONFIG['skip_test_files']]
    metrics['total_test_images'] = len(valid_test)
    logger.log(f"\n Test images: {len(valid_test)}")

# Sampling info
if CONFIG['images_per_class']:
    expected_total = CONFIG['images_per_class'] * len(class_names)
    logger.log(f"\n Sampling: {CONFIG['images_per_class']} images per class")
    logger.log(f" Expected total: {expected_total:,} images")
else:
    logger.log(f"\n Using ALL images (~{total_images:,})")

metrics['analysis_time'] = time.time() - start_time
analysis_time = f"{metrics['analysis_time']:.2f}"
logger.log(f"\n Analysis time: {analysis_time}s")

# Save metrics
metrics_file = os.path.join(CONFIG['metrics_path'], 'dataset_metrics.json')
with open(metrics_file, 'w') as f:
    # Convert to JSON-serializable
    save_metrics = {k: v for k, v in metrics.items() if k != 'image_sizes'}
    json.dump(save_metrics, f, indent=2)
logger.log(f"\n Metrics saved to: {metrics_file}")

# Visualize class distribution (single combined visualization)
visualize_class_distribution(class_counts)

# Show sample images from each class (skip 3D distribution - redundant)
visualize_dataset_samples(class_names)

return metrics

def visualize_class_distribution(class_counts):
    """Create INTERACTIVE 3D class distribution visualization with SOLID bars"""
    logger.log("\n Creating interactive 3D class distribution...")
    classes = list(class_counts.keys())
    counts = list(class_counts.values())
    n_classes = len(classes)
    max_count = max(counts) if counts else 1

    # Create 3D bar chart using solid Mesh3d bars
    fig = go.Figure()

    for i, (cls, count) in enumerate(zip(classes, counts)):
        norm = count / max_count if max_count > 0 else 0

        # Color gradient based on count (blue to purple spectrum)
        r = int(50 + 150 * norm)
        g = int(100 + 50 * (1 - norm))
        b = int(200 - 50 * norm)
        color = f'rgb({r}, {g}, {b})'

        # Add solid 3D bar using helper function
        bar = create_3d_bar_mesh(
            x_center=i,
            y_center=0,
            z_height=count,
            bar_width=0.7,
            bar_depth=0.7,
            color=color,
            opacity=0.95,
            name=cls,
            hovertemplate=f'<b>Class {cls}</b><br>Images: {count:,}<extra></extra>'
        )
        fig.add_trace(bar)

    # Add text labels at top of bars
    fig.add_trace(go.Scatter3d(
        x=list(range(n_classes)),

```



```

y=[0] * n_classes,
z=[c + max_count * 0.03 for c in counts],
mode='text',
text=classes,
textfont=dict(size=12, color='black', family='Arial Black'),
hoverinfo='skip',
showlegend=False
))

fig.update_layout(
    title=dict(
        text='ASL Dataset - 3D Class Distribution (Rotate to explore!)',
        font=dict(size=20, color='#333'),
        x=0.5
    ),
    scene=dict(
        xaxis=dict(
            title='Class',
            tickvals=list(range(n_classes)),
            ticktext=classes,
            tickangle=45,
            gridcolor='lightgray'
        ),
        yaxis=dict(
            title='',
            showticklabels=False,
            range=[-1, 1],
            showgrid=False
        ),
        zaxis=dict(
            title='Number of Images',
            gridcolor='lightgray'
        ),
        camera=dict(eye=dict(x=1.5, y=1.5, z=0.8)),
        aspectmode='manual',
        aspectratio=dict(x=2, y=0.5, z=1),
        bgcolor='rgba(250,250,250,0.9)'
    ),
    height=650,
    template='plotly_white',
    margin=dict(l=0, r=0, t=60, b=0)
)

save_path = os.path.join(CONFIG['metrics_path'], 'class_distribution.png')
save_and_show_plotly(fig, save_path, '3D Class Distribution')

def visualize_dataset_samples(class_names):
    """Visualize sample images from each class with interactive Plotly grid"""
    logger.log("\n Creating interactive dataset sample visualization...")
    n_classes = len(class_names)
    n_cols = min(6, n_classes)
    n_rows = (n_classes + n_cols - 1) // n_cols

    # Create Plotly subplots
    fig = make_subplots(
        rows=n_rows, cols=n_cols,
        subplot_titles=[f'Class: {c}' for c in class_names],
        horizontal_spacing=0.02,
        vertical_spacing=0.08
    )

    for idx, class_name in enumerate(class_names):
        row = idx // n_cols + 1
        col = idx % n_cols + 1
        class_dir = os.path.join(CONFIG['train_path'], class_name)
        if os.path.exists(class_dir):
            images = [f for f in os.listdir(class_dir)
                      if f.lower().endswith(('.jpg', '.jpeg', '.png'))]
            if images:
                img_path = os.path.join(class_dir, images[0])
                img = cv2.imread(img_path)
                if img is not None:
                    img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
                    fig.add_trace(
                        go.Image(z=img_rgb, hovertemplate=f'<b>Class: {class_name}</b>'

```

```
<extra></extra>'),
        row=row, col=col
    )

fig.update_layout(
    title=dict(
        text=' Sample Images from Each ASL Class (Click to zoom!)',
        font=dict(size=20, color='#333'),
        x=0.5
    ),
    height=200 * n_rows + 100,
    showlegend=False,
    template='plotly_white'
)

# Hide axes
fig.update_xaxes(showticklabels=False, showgrid=False)
fig.update_yaxes(showticklabels=False, showgrid=False)

save_path = os.path.join(CONFIG['metrics_path'], 'dataset_samples.png')
save_and_show_plotly(fig, save_path, 'Dataset Samples')
```

9. ✓ MediaPipe Hand Cropping

```
# MEDIAPIPE HAND CROPPING
class HandCropper:
    """MediaPipe hand detection with smart cropping
    Note: MediaPipe uses CPU by default. We minimize its impact by:
    1. Caching cropped images
    2. Using batch processing
    3. Running during data prep, not training
    """
    def __init__(self):
        self.mp_hands = mp.solutions.hands
        self.hands = self.mp_hands.Hands(
            static_image_mode=True,
            max_num_hands=1,
            min_detection_confidence=CONFIG['mediapipe_confidence'],
            model_complexity=CONFIG['mediapipe_model_complexity']
        )
        self.margin = CONFIG['mediapipe_margin']
        self.stats = {'total': 0, 'success': 0, 'fallback': 0}

    def crop_hand(self, image):
        """Detect and crop hand region"""
        h, w = image.shape[:2]
        self.stats['total'] += 1

        # Add padding for edge detection
        pad = 20
        padded = cv2.copyMakeBorder(image, pad, pad, pad, pad,
                                     cv2.BORDER_CONSTANT, value=[0, 0, 0])

        # Convert BGR to RGB for MediaPipe
        rgb = cv2.cvtColor(padded, cv2.COLOR_BGR2RGB)
        results = self.hands.process(rgb)

        if results.multi_hand_landmarks:
            landmarks = results.multi_hand_landmarks[0]
            h_pad, w_pad = padded.shape[:2]

            x_coords = [int(lm.x * w_pad) for lm in landmarks.landmark]
            y_coords = [int(lm.y * h_pad) for lm in landmarks.landmark]

            x_min = max(0, min(x_coords) - self.margin)
            x_max = min(w_pad, max(x_coords) + self.margin)
            y_min = max(0, min(y_coords) - self.margin)
            y_max = min(h_pad, max(y_coords) + self.margin)

            if x_max > x_min and y_max > y_min:
                cropped = padded[y_min:y_max, x_min:x_max]
                if cropped.shape[0] > 50 and cropped.shape[1] > 50:
                    self.stats['success'] += 1
                    return cropped, True

        # Fallback: center crop
        self.stats['fallback'] += 1
        margin_h, margin_w = int(h * 0.1), int(w * 0.1)
        return image[margin_h:h-margin_h, margin_w:w-margin_w], False

    def get_stats(self):
        """Return detection statistics"""
        total = self.stats['total']
        if total == 0:
            return self.stats
        self.stats['success_rate'] = self.stats['success'] / total * 100
        self.stats['fallback_rate'] = self.stats['fallback'] / total * 100
        return self.stats

    def close(self):
        """Clean up MediaPipe resources"""
        if hasattr(self, 'hands') and self.hands:
            self.hands.close()

def process_dataset_with_mediapipe():
    """Process and cache dataset with MediaPipe hand cropping
```

NOTE: MediaPipe uses TensorFlow Lite which runs on CPU only.
This is expected behavior - GPU is used for model training, not MediaPipe.
We optimize by caching results so this only runs once.
"""

```
logger.section("MEDIAPIPE HAND CROPPING")
logger.log(" NOTE: MediaPipe uses TFLite (CPU-only). This is normal.")
logger.log(" GPU will be used for model training after preprocessing")
logger.log("")
```

```
# Check if cached crops exist
```

```
if CONFIG['use_cached_crops']:
    train_exists = os.path.exists(CONFIG['cropped_train_path'])
    has_classes = False
    if train_exists:
        subdirs = [d for d in os.listdir(CONFIG['cropped_train_path'])
                    if os.path.isdir(os.path.join(CONFIG['cropped_train_path'], d))]
        has_classes = len(subdirs) > 0

    if has_classes:
        class_names = sorted([d for d in subdirs if d not in CONFIG['skip_folders']])
        logger.log(f" Using cached crops from: {CONFIG['cropped_train_path']}")
        logger.log(f" Found {len(class_names)} classes")
```

```
# Count cached images
```

```
total_cached = 0
```

```
for cls in class_names:
```

```
    cls_dir = os.path.join(CONFIG['cropped_train_path'], cls)
```

```
    if os.path.exists(cls_dir):
```

```
        total_cached += len([f for f in os.listdir(cls_dir) if
```

```
f.endswith(('.jpg', '.jpeg', '.png'))])
```

```
    logger.log(f" Total cached images: {total_cached:,}")
```

```
    return class_names
```

```
# Check source exists
```

```
if not os.path.exists(CONFIG['train_path']):
```

```
    logger.log(f" Training path not found: {CONFIG['train_path']}")
```

```
    return []
```

```
cropper = HandCropper()
```

```
# Get valid classes
```

```
all_folders = [d for d in os.listdir(CONFIG['train_path'])
```

```
                if os.path.isdir(os.path.join(CONFIG['train_path'], d))]
```

```
class_names = sorted([c for c in all_folders if c not in CONFIG['skip_folders']])
```

```
logger.log(f"\n Processing {len(class_names)} classes...")
```

```
logger.log(f" Images per class: {CONFIG['images_per_class'] or 'ALL'}")
```

```
# Calculate total work
```

```
total_images_to_process = 0
```

```
for class_name in class_names:
```

```
    src_dir = os.path.join(CONFIG['train_path'], class_name)
```

```
    all_images = [f for f in os.listdir(src_dir) if f.lower().endswith(('.jpg',
```

```
'.jpeg', '.png'))]
```

```
    if CONFIG['images_per_class']:
```

```
        total_images_to_process += min(len(all_images), CONFIG['images_per_class'])
```

```
    else:
```

```
        total_images_to_process += len(all_images)
```

```
logger.log(f" Total images to process: {total_images_to_process:,}")
```

```
est_time = total_images_to_process * 0.03 / 60
```

```
logger.log(f" Estimated time: ~{est_time:.1f} minutes")
```

```
# Process training data with detailed progress
```

```
logger.log("\n Processing TRAINING data...")
```

```
processing_stats = {
```

```
    'total_processed': 0,
```

```
    'successful': 0,
```

```
    'fallback': 0,
```

```
    'failed': 0,
```

```
    'per_class': {}
```

```
}
```

```
start_time = time.time()
```

```
for class_idx, class_name in enumerate(class_names):
```

```
    class_start = time.time()
```

```

src_dir = os.path.join(CONFIG['train_path'], class_name)
dst_dir = os.path.join(CONFIG['cropped_train_path'], class_name)
os.makedirs(dst_dir, exist_ok=True)

all_images = [f for f in os.listdir(src_dir)
               if f.lower().endswith(('.jpg', '.jpeg', '.png'))]

# Sample images if configured
if CONFIG['images_per_class'] and len(all_images) > CONFIG['images_per_class']:
    np.random.seed(42)
    images = list(np.random.choice(all_images, CONFIG['images_per_class'],
replace=False))
else:
    images = all_images

saved_count = 0
for img_name in images:
    img_path = os.path.join(src_dir, img_name)
    img = cv2.imread(img_path)
    if img is None:
        continue

    cropped, success = cropper.crop_hand(img)

    # Skip fallback crops if required
    if CONFIG['require_mediapipe_detection'] and not success:
        continue

    try:
        cropped_resized = cv2.resize(cropped, CONFIG['img_size'])
        save_path = os.path.join(dst_dir, img_name)
        cv2.imwrite(save_path, cropped_resized)
        saved_count += 1
        if success:
            processing_stats['successful'] += 1
        else:
            processing_stats['fallback'] += 1
    except Exception as e:
        processing_stats['failed'] += 1
        continue

processing_stats['total_processed'] += len(images)
processing_stats['per_class'][class_name] = saved_count

# Print per-class progress
class_time = time.time() - class_start
elapsed = time.time() - start_time
remaining_classes = len(class_names) - (class_idx + 1)
eta = (elapsed / (class_idx + 1)) * remaining_classes if class_idx > 0 else 0

print(f" [{class_idx+1:2d}/{len(class_names)}] {class_name}: {saved_count:,} saved
| "
      f"Time: {class_time:.1f}s | ETA: {eta/60:.1f}min", flush=True)

# Also save some to test set
if CONFIG['test_images_per_class_from_train'] > 0:
    remaining = [f for f in all_images if f not in images]
    if remaining:
        n_take = min(CONFIG['test_images_per_class_from_train'], len(remaining))
        np.random.seed(42)
        test_images = list(np.random.choice(remaining, n_take, replace=False))
        dst_test_dir = os.path.join(CONFIG['cropped_test_path'], class_name)
        os.makedirs(dst_test_dir, exist_ok=True)

        for img_name in test_images:
            img_path = os.path.join(src_dir, img_name)
            img = cv2.imread(img_path)
            if img is None:
                continue

            cropped, success = cropper.crop_hand(img)
            if CONFIG['require_mediapipe_detection'] and not success:
                continue

            try:

```

```

        cropped_resized = cv2.resize(cropped, CONFIG['img_size'])
        save_path = os.path.join(dst_test_dir, img_name)
        cv2.imwrite(save_path, cropped_resized)
    except:
        continue

# Print processing summary
total_time = time.time() - start_time
logger.log(f"\n{'='*60}")
logger.log(f" MEDIAPIPE PROCESSING COMPLETE")
logger.log(f"\n{'='*60}")
total_time_min = total_time/60
logger.log(f" Total time: {total_time_min:.1f} minutes")
total_processed = processing_stats['total_processed']
successful = processing_stats['successful']
fallback = processing_stats['fallback']
failed = processing_stats['failed']
processing_speed = total_processed/total_time
logger.log(f" Images processed: {total_processed:,}")
logger.log(f" Successful detections: {successful:,}")
logger.log(f" Fallback crops: {fallback:,}")
logger.log(f" Failed: {failed:,}")
logger.log(f" Processing speed: {processing_speed:.1f} images/sec")

# Per-class summary
logger.log(f"\n Per-Class Results:")
logger.log(f"\n{'-'*40}")
for cls, count in processing_stats['per_class'].items():
    logger.log(f" {cls}: {count:,} images saved")
logger.log(f"\n{'-'*40}")

# Save processing stats
stats_path = os.path.join(CONFIG['metrics_path'], 'mediapipe_processing_stats.json')
with open(stats_path, 'w') as f:
    json.dump(processing_stats, f, indent=2)
logger.log(f"\n Processing stats saved to: {stats_path}")

# Process test data from test folder
if os.path.exists(CONFIG['test_path']):
    logger.log("\n Processing TEST data...")
    test_files = [f for f in os.listdir(CONFIG['test_path'])
                  if f.lower().endswith('.jpg') and '_test' in f]
    test_saved = 0
    for test_file in tqdm(test_files, desc="Test images"):
        if test_file in CONFIG['skip_test_files']:
            continue

        class_name = test_file.split('_test')[0]
        if class_name not in class_names:
            continue

        src_path = os.path.join(CONFIG['test_path'], test_file)
        img = cv2.imread(src_path)
        if img is None:
            continue

        cropped, success = cropper.crop_hand(img)
        if CONFIG['require_mediapipe_detection'] and not success:
            continue

        try:
            cropped_resized = cv2.resize(cropped, CONFIG['img_size'])
            dst_dir = os.path.join(CONFIG['cropped_test_path'], class_name)
            os.makedirs(dst_dir, exist_ok=True)
            save_path = os.path.join(dst_dir, test_file)
            cv2.imwrite(save_path, cropped_resized)
            test_saved += 1
        except:
            continue

    logger.log(f"\n Test images saved: {test_saved}")

# Print statistics
stats = cropper.get_stats()
logger.log(f"\n MediaPipe Detection Statistics:")

```



```
logger.log(f" Total processed: {stats['total']:,}")
success_rate = f"{stats.get('success_rate', 0):.1f}%"
fallback_rate = f"{stats.get('fallback_rate', 0):.1f}%"
logger.log(f" Successful detections: {stats['success']:,} ({success_rate})")
logger.log(f" Fallback crops: {stats['fallback']:,} ({fallback_rate})")

cropper.close()

# Save sample visualization
visualize_cropped_samples(class_names)

# Visualize processing statistics
visualize_mediapipe_stats(processing_stats, class_names)

return class_names
```


10. ✓ Data Loading and Augmentation

```
# DATA LOADING
class AugmentedDataGenerator(keras.utils.Sequence):
    """Custom data generator with model-specific preprocessing"""
    def __init__(self, image_paths, labels, batch_size, model_name,
                 augmentation=None, shuffle=True):
        self.image_paths = image_paths
        self.labels = labels
        self.batch_size = batch_size
        self.model_name = model_name
        self.augmentation = augmentation
        self.shuffle = shuffle
        self.indices = np.arange(len(self.image_paths))

    # Set preprocessing function based on model
    if model_name == 'EfficientNetV2B3':
        self.preprocess_fn = effnet_preprocess
    elif model_name == 'ResNet50':
        self.preprocess_fn = resnet_preprocess
    elif model_name == 'InceptionV3':
        self.preprocess_fn = inception_preprocess
    else:
        self.preprocess_fn = None

    self.on_epoch_end()

    def __len__(self):
        return int(np.ceil(len(self.image_paths) / self.batch_size))

    def __getitem__(self, idx):
        batch_indices = self.indices[idx * self.batch_size:(idx + 1) * self.batch_size]
        batch_paths = [self.image_paths[i] for i in batch_indices]
        batch_labels = [self.labels[i] for i in batch_indices]

        X, y = [], []
        for path, label in zip(batch_paths, batch_labels):
            img = cv2.imread(path)
            if img is None:
                img = np.zeros((CONFIG['img_size'][0], CONFIG['img_size'][1], 3),
                               dtype=np.uint8)
            else:
                img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

            # Apply augmentation
            if self.augmentation:
                try:
                    img = self.augmentation(image=img)['image']
                except:
                    pass

            # Ensure correct size
            img = cv2.resize(img, CONFIG['img_size'])

            # Apply model-specific preprocessing
            if self.preprocess_fn:
                img = self.preprocess_fn(img.astype(np.float32))
            else:
                img = img.astype(np.float32) / 255.0

            X.append(img)
            y.append(label)

        return np.array(X), np.array(y)

    def on_epoch_end(self):
        if self.shuffle:
            np.random.shuffle(self.indices)

def load_dataset(cropped_path, class_names):
    """Load cropped dataset"""
    class_to_idx = {name: idx for idx, name in enumerate(class_names)}
    image_paths, labels = [], []
```

```

for class_name in class_names:
    class_dir = os.path.join(cropped_path, class_name)
    if not os.path.exists(class_dir):
        continue

    images = [f for f in os.listdir(class_dir)
               if f.lower().endswith(('.jpg', '.jpeg', '.png'))]
    for img_name in images:
        image_paths.append(os.path.join(class_dir, img_name))
        labels.append(class_to_idx[class_name])

return np.array(image_paths), np.array(labels)

```

11. ✓ Augmentation Pipelines

```

# AUGMENTATION PIPELINES
def get_augmentation_pipeline(model_name):
    """Model-specific augmentation pipelines"""
    if model_name == 'ResNet50':
        return A.Compose([
            A.Rotate(limit=25, p=0.8),
            A.Affine(scale=(0.85, 1.15), translate_percent=(-0.15, 0.15),
                    rotate=(-20, 20), shear=(-10, 10), p=0.7),
            A.HorizontalFlip(p=0.5),
            A.OneOf([
                A.MotionBlur(blur_limit=5, p=1.0),
                A.GaussianBlur(blur_limit=5, p=1.0),
            ], p=0.4),
            A.RandomBrightnessContrast(brightness_limit=0.25, contrast_limit=0.2, p=0.6),
            A.CLAHE(clip_limit=2.0, p=0.3),
            A.GaussNoise(var_limit=(10.0, 40.0), p=0.25),
            A.CoarseDropout(num_holes_range=(1, 3), hole_height_range=(8, 16),
                           hole_width_range=(8, 16), p=0.25),
        ])
    elif model_name == 'EfficientNetV2B3':
        return A.Compose([
            A.Rotate(limit=30, p=0.9),
            A.Affine(scale=(0.80, 1.2), translate_percent=(-0.12, 0.12),
                    rotate=(-18, 18), p=0.7),
            A.HorizontalFlip(p=0.5),
            A.OneOf([
                A.GaussianBlur(blur_limit=3, p=1.0),
                A.MotionBlur(blur_limit=3, p=1.0),
            ], p=0.35),
            A.RandomBrightnessContrast(brightness_limit=0.25, contrast_limit=0.2, p=0.6),
            A.CLAHE(clip_limit=2.0, p=0.3),
            A.GaussNoise(var_limit=(5.0, 30.0), p=0.2),
            A.CoarseDropout(num_holes_range=(1, 4), hole_height_range=(6, 20),
                           hole_width_range=(6, 20), p=0.25),
        ])
    elif model_name == 'InceptionV3':
        return A.Compose([
            A.Rotate(limit=15, p=0.6),
            A.Affine(scale=(0.9, 1.1), translate_percent=(-0.1, 0.1),
                    rotate=(-10, 10), p=0.5),
            A.RandomBrightnessContrast(brightness_limit=0.15, contrast_limit=0.15, p=0.5),
            A.CLAHE(clip_limit=1.5, p=0.2),
        ])

return None

```

12. ✓ Model Builders

```
# MODEL BUILDERS
def build_resnet50(num_classes):
    """ResNet50 with fine-tuning"""
    base = ResNet50(weights='imagenet', include_top=False,
                      input_shape=(*CONFIG['img_size'], 3))

    # Freeze early layers
    for layer in base.layers[:-30]:
        layer.trainable = False

    model = models.Sequential([
        base,
        layers.GlobalAveragePooling2D(),
        layers.BatchNormalization(),
        layers.Dropout(0.5),
        layers.Dense(512, activation='relu', kernel_regularizer=keras.regularizers.l2(1e-
4)),
        layers.BatchNormalization(),
        layers.Dropout(0.4),
        layers.Dense(256, activation='relu', kernel_regularizer=keras.regularizers.l2(1e-
4)),
        layers.Dropout(0.3),
        layers.Dense(num_classes, activation='softmax')
    ], name='ResNet50_AS1')

    return model

def build_efficientnet(num_classes):
    """EfficientNetV2B3 with fine-tuning"""
    try:
        base = EfficientNetV2B3(weights='imagenet', include_top=False,
                                  input_shape=(*CONFIG['img_size'], 3))
    except:
        base = EfficientNetV2B3(weights=None, include_top=False,
                                  input_shape=(*CONFIG['img_size'], 3))

    # Freeze most of the base model
    for layer in base.layers[:-80]:
        layer.trainable = False

    model = models.Sequential([
        base,
        layers.GlobalAveragePooling2D(),
        layers.BatchNormalization(),
        layers.Dropout(0.5),
        layers.Dense(512, activation='relu', kernel_regularizer=keras.regularizers.l2(1e-
4)),
        layers.BatchNormalization(),
        layers.Dropout(0.35),
        layers.Dense(256, activation='relu', kernel_regularizer=keras.regularizers.l2(1e-
4)),
        layers.Dropout(0.25),
        layers.Dense(num_classes, activation='softmax')
    ], name='EfficientNetV2B3_AS1')

    return model

def build_inception(num_classes):
    """InceptionV3 with fine-tuning"""
    base = InceptionV3(weights='imagenet', include_top=False,
                       input_shape=(*CONFIG['img_size'], 3))

    # Freeze most of the base model
    for layer in base.layers[:-60]:
        layer.trainable = False

    model = models.Sequential([
        base,
        layers.GlobalAveragePooling2D(),
        layers.BatchNormalization(),
        layers.Dropout(0.45),
        layers.Dense(512, activation='relu', kernel_regularizer=keras.regularizers.l2(1e-
```

```
4)),
    layers.BatchNormalization(),
    layers.Dropout(0.35),
    layers.Dense(256, activation='relu', kernel_regularizer=keras.regularizers.l2(1e-
4)),
    layers.Dropout(0.25),
    layers.Dense(num_classes, activation='softmax')
], name='InceptionV3_ASL')

return model

MODEL_BUILDERS = {
    'ResNet50': build_resnet50,
    'EfficientNetV2B3': build_efficientnet,
    'InceptionV3': build_inception
}
```

13. ✓ Model Information Collection

```
# MODEL INFO COLLECTION
def collect_model_info(num_classes):
    """Collect comprehensive information about all models BEFORE training
    This function:
    1. Creates each model architecture
    2. Collects detailed layer info, parameter counts
    3. Tests if existing models can be loaded
    4. Stores all info in a structured format
    5. Saves visualizations of model architectures
    """
    logger.section("MODEL INFORMATION COLLECTION")
    model_info = {}

    for model_name in CONFIG['train_models']:
        logger.log(f"\n{'='*60}")
        logger.log(f"ANALYZING: {model_name}")
        logger.log(f"{'='*60}")

        info = {
            'name': model_name,
            'status': 'unknown',
            'existing_model_path': None,
            'existing_model_found': False,
            'will_resume': False,
            'total_params': 0,
            'trainable_params': 0,
            'non_trainable_params': 0,
            'trainable_ratio': 0,
            'num_layers': 0,
            'base_model_layers': 0,
            'custom_layers': 0,
            'input_shape': None,
            'output_shape': None,
            'layer_summary': [],
            'memory_estimate_mb': 0
        }

        # Check for existing model
        existing_path = CONFIG['existing_models'].get(model_name)
        info['existing_model_path'] = existing_path

        if existing_path and os.path.exists(existing_path):
            info['existing_model_found'] = True
            info['will_resume'] = True
            info['status'] = 'RESUME - Existing model found'
            logger.log(f"\nEXISTING MODEL FOUND: {existing_path}")

            # Try to load and get info from existing model
            try:
                model, format_type = load_model_any_format(existing_path, model_name)
                if model is not None:
                    info['total_params'] = model.count_params()
                    info['trainable_params'] = sum([tf.size(w).numpy() for w in
model.trainable_weights])
                    info['non_trainable_params'] = info['total_params'] -
info['trainable_params']
                    info['num_layers'] = len(model.layers)
                    info['input_shape'] = str(model.input_shape)
                    info['output_shape'] = str(model.output_shape)
                    info['model_format'] = format_type

                    # If it's SavedModel (inference-only), mark for scratch training
                    if format_type == 'savedmodel':
                        info['will_resume'] = False
                        info['status'] = 'SCRATCH - SavedModel (inference-only)'
                        logger.log(f"\nSavedModel format is inference-only, will train
from scratch")

                    del model
                    keras.backend.clear_session()
            else:
                # format_type contains error message
```

```

        raise Exception(format_type)
    except Exception as e:
        logger.log(f"\n Could not load existing model: {e}")
        info['will_resume'] = False
        info['status'] = f'SCRATCH - Load failed: {str(e)}'
    else:
        info['status'] = 'SCRATCH - No existing model'
        if existing_path:
            logger.log(f"\n NO EXISTING MODEL at: {existing_path}")
        else:
            logger.log(f"\n NO EXISTING MODEL configured for {model_name}")
            logger.log(f" Will train from SCRATCH")

# Build fresh model to get architecture info
try:
    model = MODEL_BUILDERS[model_name](num_classes)

    info['total_params'] = model.count_params()
    info['trainable_params'] = sum([tf.size(w).numpy() for w in
model.trainable_weights])
    info['non_trainable_params'] = info['total_params'] - info['trainable_params']
    info['trainable_ratio'] = info['trainable_params'] / info['total_params'] if
info['total_params'] > 0 else 0
    info['num_layers'] = len(model.layers)
    info['input_shape'] = str(model.input_shape)
    info['output_shape'] = str(model.output_shape)

    # Estimate memory (rough: 4 bytes per parameter for float32)
    info['memory_estimate_mb'] = (info['total_params'] * 4) / (1024 * 1024)

    # Count base vs custom layers
    for layer in model.layers:
        if hasattr(layer, 'layers'): # It's a nested model (base)
            info['base_model_layers'] = len(layer.layers)
        else:
            info['custom_layers'] += 1

    # Collect layer summary (last 15 layers)
    for layer in model.layers[-15:]:
        # Safely get output shape (TF 2.18+ compatibility)
        try:
            out_shape = str(layer.output.shape) if hasattr(layer, 'output') and
layer.output is not None else 'N/A'
        except:
            out_shape = 'N/A'

        layer_info = {
            'name': layer.name,
            'type': layer.__class__.__name__,
            'output_shape': out_shape,
            'trainable': layer.trainable,
            'params': layer.count_params()
        }
        info['layer_summary'].append(layer_info)

    # Print detailed info
    logger.log(f"\n Architecture Details:")
    logger.log(f" Input Shape: {info['input_shape']}")
    logger.log(f" Output Shape: {info['output_shape']}")
    logger.log(f" Total Parameters: {info['total_params']:,}")
    logger.log(f" Trainable Params: {info['trainable_params']:,}")
    logger.log(f" Non-trainable Params: {info['non_trainable_params']:,}")
    logger.log(f" Trainable Ratio: {info['trainable_ratio']:.1%}")
    logger.log(f" Total Layers: {info['num_layers']}")
    logger.log(f" Base Model Layers: {info['base_model_layers']}")
    logger.log(f" Custom Layers: {info['custom_layers']}")
    logger.log(f" Est. Memory: {info['memory_estimate_mb']:.1f} MB")

    logger.log(f"\n Layer Summary (last 15):")
    logger.log("-" * 70)
    for li in info['layer_summary']:
        trainable_mark = "✓" if li['trainable'] else "x"
        logger.log(f" [{trainable_mark}] {li['name']:<30} {li['type']:<20}
{li['output_shape']:<20} {li['params']:,} params")
    logger.log("-" * 70)

```



```

        # Cleanup
        del model
        keras.backend.clear_session()
        gc.collect()
    except Exception as e:
        logger.log(f"\n Error building {model_name}: {e}")
        info['status'] = f'ERROR - {str(e)}'

    model_info[model_name] = info

# Save model info to JSON
model_info_path = os.path.join(CONFIG['metrics_path'], 'model_info.json')
# Convert to JSON-serializable format
save_info = {}
for name, info in model_info.items():
    save_info[name] = {k: v for k, v in info.items()}

with open(model_info_path, 'w') as f:
    json.dump(save_info, f, indent=2, default=str)
logger.log(f"\n Model info saved to: {model_info_path}")

# Create comparison visualization
visualize_model_architecture_comparison(model_info)

# Print summary table
logger.log(f"\n{'='*80}")
logger.log(f" MODEL SUMMARY TABLE")
logger.log(f"\n{'='*80}")
logger.log(f"{'Model':<20} {'Status':<25} {'Params':>15} {'Trainable':>15}")
logger.log(f"\n{'-'*80}")
for name, info in model_info.items():
    logger.log(f"{name:<20} {info['status']:<25} {info['total_params']:>15,}
{info['trainable_params']:>15,}")
    logger.log(f"\n{'='*80}")

return model_info

```


14. ✓ Training Pipeline

```
# TRAINING
class DetailedMetricsCallback(keras.callbacks.Callback):
    """Custom callback to print detailed metrics during training"""
    def __init__(self, model_name, total_epochs):
        super().__init__()
        self.model_name = model_name
        self.total_epochs = total_epochs
        self.best_val_acc = 0
        self.best_epoch = 0
        self.start_time = None
        self.epoch_times = []

    def on_train_begin(self, logs=None):
        self.start_time = time.time()
        print(f"\n{'='*80}")
        print(f" TRAINING STARTED: {self.model_name}")
        print(f"{'='*80}")

    def on_epoch_begin(self, epoch, logs=None):
        self.epoch_start = time.time()
        print(f"\n Epoch {epoch+1}/{self.total_epochs}")

    def on_epoch_end(self, epoch, logs=None):
        epoch_time = time.time() - self.epoch_start
        self.epoch_times.append(epoch_time)

        # Get metrics
        train_acc = logs.get('accuracy', 0)
        val_acc = logs.get('val_accuracy', 0)
        train_loss = logs.get('loss', 0)
        val_loss = logs.get('val_loss', 0)
        lr = float(keras.backend.get_value(self.model.optimizer.learning_rate))

        # Track best
        if val_acc > self.best_val_acc:
            self.best_val_acc = val_acc
            self.best_epoch = epoch + 1
            best_marker = " NEW BEST!"
        else:
            best_marker = ""

        # Calculate ETA
        avg_epoch_time = sum(self.epoch_times) / len(self.epoch_times)
        remaining_epochs = self.total_epochs - (epoch + 1)
        eta_seconds = avg_epoch_time * remaining_epochs
        eta_str = f"{eta_seconds/60:.1f}min" if eta_seconds < 3600 else f"
{eta_seconds/3600:.1f}h"

        # Print detailed metrics
        print(f" {'-'*60}␣")
        print(f" | Train Acc: {train_acc*100:6.2f}% | Val Acc: {val_acc*100:6.2f}% |
{best_marker}")
        print(f" | Train Loss: {train_loss:6.4f} | Val Loss: {val_loss:6.4f} |")
        print(f" | LR: {lr:.2e} | Time: {epoch_time:.1f}s | ETA: {eta_str:<8} |")
        print(f" | Best Val Acc: {self.best_val_acc*100:.2f}% @ Epoch {self.best_epoch} |")
        print(f" {'-'*60}␣")

    def on_train_end(self, logs=None):
        total_time = time.time() - self.start_time
        print(f"\n{'='*80}")
        print(f" TRAINING COMPLETE: {self.model_name}")
        print(f"{'='*80}")
        print(f" Total time: {total_time/60:.1f} minutes")
        print(f" Best val accuracy: {self.best_val_acc*100:.2f}% @ Epoch
{self.best_epoch}")
        avg_epoch_time = sum(self.epoch_times)/len(self.epoch_times) if self.epoch_times
else 0
        print(f" Average epoch time: {avg_epoch_time:.1f} seconds")
        print(f"{'='*80}\n")

def train_model(model_name, train_paths, train_labels, val_paths, val_labels,
                num_classes, class_names, force_scratch=False, training_mode='AUTO'):
```

```

"""Train or resume training for a model with intelligent mode selection
Args:
    training_mode: One of 'AUTO', 'SCRATCH', 'RESUME', 'FINE_TUNE'
    - AUTO: Automatically decide based on existing model
    - SCRATCH: Train new model from scratch
    - RESUME: Resume training from checkpoint (moderate epochs, low LR)
    - FINE_TUNE: Light fine-tuning (few epochs, very low LR)
"""
logger.section(f"TRAINING: {model_name}")
logger.log(f" Training mode: {training_mode}")

# Handle force_scratch legacy parameter
if force_scratch:
    training_mode = 'SCRATCH'

# Check for existing model (unless forced to scratch)
if training_mode == 'SCRATCH':
    logger.log(f"\n FORCED training from SCRATCH (ignoring existing model)")
    existing_model, model_path = None, None
else:
    existing_model, model_path = check_existing_model(model_name)

# Determine training parameters based on mode
if training_mode == 'FINE_TUNE' and existing_model:
    # Fine-tuning: very few epochs, minimal learning rate
    logger.log(f"\n FINE-TUNING from: {model_path}")
    model = existing_model
    epochs = CONFIG.get('fine_tune_epochs', 5) # Very few epochs
    initial_lr = 5e-6 # Very low LR for fine-tuning
    is_resume = True
    logger.log(f" • Fine-tune epochs: {epochs}")
    logger.log(f" • Fine-tune LR: {initial_lr}")
elif training_mode == 'RESUME' and existing_model:
    # Resume: moderate epochs, low learning rate
    logger.log(f"\n RESUMING training from: {model_path}")
    model = existing_model
    epochs = CONFIG['resume_epochs']
    initial_lr = 1e-5 # Low LR for continued training
    is_resume = True
elif existing_model and training_mode == 'AUTO':
    # Auto mode with existing model - treat as resume
    logger.log(f"\n AUTO mode: RESUMING from: {model_path}")
    model = existing_model
    epochs = CONFIG['resume_epochs']
    initial_lr = 1e-5
    is_resume = True
else:
    # Scratch training (new model or forced)
    logger.log(f"\n Training from SCRATCH")
    if training_mode not in ['SCRATCH', 'AUTO']:
        logger.log(f"\n No existing model found - falling back to SCRATCH")
    if training_mode == 'AUTO':
        logger.log(f"\n No existing model found at:
{CONFIG['existing_models'].get(model_name, 'N/A')}")
        model = MODEL_BUILDERS[model_name](num_classes)
        epochs = CONFIG['epochs']
        initial_lr = 1e-3
        is_resume = False

# Compile model
optimizer = keras.optimizers.Adam(learning_rate=initial_lr)
model.compile(
    optimizer=optimizer,
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

# Print detailed model info
total_params = model.count_params()
trainable_params = sum([tf.size(w).numpy() for w in model.trainable_weights])
non_trainable_params = total_params - trainable_params
logger.log(f"\n{'=' * 60}")
logger.log(f" MODEL ARCHITECTURE: {model.name}")
logger.log(f"{'=' * 60}")
logger.log(f" Total parameters: {total_params:,}")

```

```

logger.log(f" Trainable parameters: {trainable_params:,}")
logger.log(f" Non-trainable params: {non_trainable_params:,}")
trainable_ratio = trainable_params/total_params*100 if total_params > 0 else 0
logger.log(f" Trainable ratio: {trainable_ratio:.1f}%")
logger.log(f"{'=' * 60}")
logger.log(f" Epochs: {epochs}")
logger.log(f" Initial LR: {initial_lr}")
logger.log(f" Batch size: {CONFIG['batch_size']}")
logger.log(f" Training samples: {len(train_paths)}")
logger.log(f" Validation samples: {len(val_paths)}")
steps_per_epoch = len(train_paths) // CONFIG['batch_size']
logger.log(f" Steps per epoch: {steps_per_epoch}")
logger.log(f"{'=' * 60}")

# Print layer summary to terminal
logger.log("\n Layer Summary (last 10):")
logger.log("-" * 60)
for i, layer in enumerate(model.layers[-10:]): # Last 10 layers
    trainable_str = "✓" if layer.trainable else "x"
    # Safely get output shape (TF 2.18+ compatibility)
    try:
        out_shape = layer.output.shape if hasattr(layer, 'output') and layer.output is
not None else 'N/A'
    except:
        out_shape = 'N/A'
    logger.log(f" [{trainable_str}] {layer.name}: {out_shape}")
logger.log("-" * 60)

# Create data generators
aug_pipeline = get_augmentation_pipeline(model_name)

# Preview augmentation
visualize_augmentation_preview(train_paths, train_labels, aug_pipeline, model_name)

train_gen = AugmentedDataGenerator(
    train_paths, train_labels, CONFIG['batch_size'], model_name,
    augmentation=aug_pipeline, shuffle=True
)
val_gen = AugmentedDataGenerator(
    val_paths, val_labels, CONFIG['batch_size'], model_name,
    augmentation=None, shuffle=False
)

# Callbacks
callbacks = [
    DetailedMetricsCallback(model_name, epochs), # Custom detailed metrics
    ModelCheckpoint(
        os.path.join(CONFIG['models_path'], f'{model_name}_best.keras'),
        save_best_only=True, monitor='val_accuracy', mode='max', verbose=1
    ),
    EarlyStopping(
        monitor='val_accuracy', patience=8, restore_best_weights=True, verbose=1
    ),
    ReduceLROnPlateau(
        monitor='val_loss', factor=0.3, patience=4, min_lr=1e-7, verbose=1
    ),
    keras.callbacks.TerminateOnNaN()
]

# Train
logger.log(f"\n Starting training...")
logger.log(f" Using GPU: {HAS_GPU}")
start_time = time.time()

try:
    history = model.fit(
        train_gen,
        validation_data=val_gen,
        epochs=epochs,
        callbacks=callbacks,
        verbose=1
    )
except Exception as e:
    logger.log(f"\n Training failed: {e}")
    # Save crash model

```

```
crash_path = os.path.join(CONFIG['models_path'], f'{model_name}_crash.keras')
try:
    model.save(crash_path)
    logger.log(f"\n Crash model saved to: {crash_path}")
except:
    pass
raise

training_time = time.time() - start_time

# Save final model
final_path = os.path.join(CONFIG['models_path'], f'{model_name}_final.keras')
model.save(final_path)
logger.log(f"\n Final model saved to: {final_path}")
logger.log(f" Training time: {training_time/60:.1f} minutes")

# Plot training history
plot_training_history(history, model_name)

return model, history
```

15. ✓ Evaluation and Visualization

```
# EVALUATION
def evaluate_model(model, model_name, test_paths, test_labels, class_names):
    """Comprehensive model evaluation with ALL metrics"""
    logger.section(f"EVALUATING: {model_name}")

    # Set preprocessing function
    if model_name == 'EfficientNetV2B3':
        preprocess_fn = effnet_preprocess
    elif model_name == 'ResNet50':
        preprocess_fn = resnet_preprocess
    elif model_name == 'InceptionV3':
        preprocess_fn = inception_preprocess
    else:
        preprocess_fn = None

    logger.log(f" Using preprocessing: {preprocess_fn.__name__ if preprocess_fn else
'None'}")

    # Load ALL test data
    X_test, y_test = [], []
    logger.log(f" Loading ALL {len(test_paths)} test samples...")
    for path, label in zip(test_paths, test_labels):
        img = cv2.imread(path)
        if img is None:
            continue
        img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
        img = cv2.resize(img, CONFIG['img_size'])
        if preprocess_fn:
            img = preprocess_fn(img.astype(np.float32))
        else:
            img = img.astype(np.float32) / 255.0
        X_test.append(img)
        y_test.append(label)

    X_test = np.array(X_test)
    y_test = np.array(y_test)

    logger.log(f" Test samples: {len(X_test)}")
    logger.log(f" Data shape: {X_test.shape}")
    logger.log(f" Data range: [{X_test.min():.2f}, {X_test.max():.2f}]")

    if len(X_test) == 0:
        logger.log("\n No test samples - skipping evaluation")
        return None

    # Predictions
    y_pred_probs = model.predict(X_test, verbose=0)
    y_pred = np.argmax(y_pred_probs, axis=1)

    # COMPREHENSIVE METRICS
    # Basic metrics
    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred, average='weighted', zero_division=0)
    recall = recall_score(y_test, y_pred, average='weighted', zero_division=0)
    f1 = f1_score(y_test, y_pred, average='weighted', zero_division=0)

    # Advanced metrics
    balanced_acc = balanced_accuracy_score(y_test, y_pred)
    cohen_kappa = cohen_kappa_score(y_test, y_pred)
    try:
        mcc = matthews_corrcoef(y_test, y_pred)
    except:
        mcc = 0.0

    # Multi-class ROC-AUC (one-vs-rest)
    try:
        roc_auc = roc_auc_score(y_test, y_pred_probs, multi_class='ovr')
    except:
        roc_auc = None

    # Log loss
    try:
```

```

        logloss = log_loss(y_test, y_pred_probs)
    except:
        logloss = None

# Per-class metrics (ALL classes)
precision_per_class = precision_score(y_test, y_pred, average=None, zero_division=0)
recall_per_class = recall_score(y_test, y_pred, average=None, zero_division=0)
f1_per_class = f1_score(y_test, y_pred, average=None, zero_division=0)

# Macro averages
precision_macro = precision_score(y_test, y_pred, average='macro', zero_division=0)
recall_macro = recall_score(y_test, y_pred, average='macro', zero_division=0)
f1_macro = f1_score(y_test, y_pred, average='macro', zero_division=0)

# Top-K accuracy
try:
    top3_acc = top_k_accuracy_score(y_test, y_pred_probs, k=3)
    top5_acc = top_k_accuracy_score(y_test, y_pred_probs, k=5)
except:
    top3_acc = None
    top5_acc = None

# Confidence analysis
pred_confidences = np.max(y_pred_probs, axis=1)
avg_confidence = np.mean(pred_confidences)
correct_mask = y_pred == y_test
avg_conf_correct = np.mean(pred_confidences[correct_mask]) if np.any(correct_mask) else
0
avg_conf_wrong = np.mean(pred_confidences[~correct_mask]) if np.any(~correct_mask) else
0

# Confusion matrix
cm = confusion_matrix(y_test, y_pred)
per_class_acc = cm.diagonal() / cm.sum(axis=1) if cm.sum() > 0 else
np.zeros(len(class_names))

# Most confused pairs
cm_no_diag = cm.copy()
np.fill_diagonal(cm_no_diag, 0)
confused_pairs = []
for i in range(len(class_names)):
    for j in range(len(class_names)):
        if cm_no_diag[i, j] > 0:
            confused_pairs.append({
                'true': class_names[i],
                'pred': class_names[j],
                'count': int(cm_no_diag[i, j])
            })
confused_pairs.sort(key=lambda x: x['count'], reverse=True)

# DISPLAY COMPREHENSIVE RESULTS
logger.log(f"\n{'='*70}")
logger.log(f" COMPREHENSIVE EVALUATION RESULTS for {model_name}")
logger.log(f"{'='*70}")
logger.log(f"\n OVERALL METRICS:")
logger.log(f" {'-'*60}┐")
logger.log(f" | Accuracy: {accuracy:.4f} ({accuracy*100:.2f}%) |")
logger.log(f" | Balanced Accuracy: {balanced_acc:.4f} ({balanced_acc*100:.2f}%) |")
logger.log(f" | Precision (weighted): {precision:.4f} |")
logger.log(f" | Recall (weighted): {recall:.4f} |")
logger.log(f" | F1-Score (weighted): {f1:.4f} |")
logger.log(f" | F1-Score (macro): {f1_macro:.4f} |")
logger.log(f" | Cohen's Kappa: {cohen_kappa:.4f} |")
logger.log(f" | Matthews CC: {mcc:.4f} |")
if roc_auc:
    logger.log(f" | ROC-AUC (weighted): {roc_auc:.4f} |")
if logloss:
    logger.log(f" | Log Loss: {logloss:.4f} |")
if top3_acc:
    logger.log(f" | Top-3 Accuracy: {top3_acc:.4f} ({top3_acc*100:.2f}%) |")
if top5_acc:
    logger.log(f" | Top-5 Accuracy: {top5_acc:.4f} ({top5_acc*100:.2f}%) |")
logger.log(f" {'-'*60}└")

logger.log(f"\n CONFIDENCE ANALYSIS:")

```



```

logger.log(f" • Average Confidence: {avg_confidence:.4f}")
logger.log(f" • Conf (Correct Preds): {avg_conf_correct:.4f}")
logger.log(f" • Conf (Wrong Preds): {avg_conf_wrong:.4f}")
confidence_gap = avg_conf_correct - avg_conf_wrong
calibration = "(excellent calibration)" if confidence_gap > 0.2 else "(good
calibration)"
logger.log(f" • Confidence Gap: {confidence_gap:.4f} {calibration}")

# Classification report (shows ALL classes)
logger.log(f"\n CLASSIFICATION REPORT (ALL {len(class_names)} CLASSES):")
report = classification_report(y_test, y_pred, target_names=class_names, digits=4)
logger.log(report)

# Per-class metrics table
logger.log(f"\n PER-CLASS METRICS:")
logger.log(f" {'Class':<8} {'Accuracy':>10} {'Precision':>10} {'Recall':>10} {'F1':>10}
{'Support':>8}")
logger.log(f" {'-'*56}")
for i, cls_name in enumerate(class_names):
    if i < len(per_class_acc):
        support = int(cm.sum(axis=1)[i]) if i < len(cm) else 0
        logger.log(f" {cls_name:<8} {per_class_acc[i]:>10.4f}
{precision_per_class[i]:>10.4f} "
                    f"{recall_per_class[i]:>10.4f} {f1_per_class[i]:>10.4f}
{support:>8}")

# Most confused pairs
logger.log(f"\n MOST CONFUSED PAIRS (Top 10):")
for pair in confused_pairs[:10]:
    logger.log(f" {pair['true']} → {pair['pred']}: {pair['count']} mistakes")

# Save detailed report
report_path = os.path.join(CONFIG['metrics_path'],
f'{model_name}_classification_report.txt')
with open(report_path, 'w') as f:
    f.write(f"{'='*70}\n")
    f.write(f"Model: {model_name}\n")
    f.write(f"{'='*70}\n\n")
    f.write(f"OVERALL METRICS:\n")
    f.write(f" Accuracy: {accuracy:.4f}\n")
    f.write(f" Balanced Accuracy: {balanced_acc:.4f}\n")
    f.write(f" Precision (weighted): {precision:.4f}\n")
    f.write(f" Recall (weighted): {recall:.4f}\n")
    f.write(f" F1-Score (weighted): {f1:.4f}\n")
    f.write(f" F1-Score (macro): {f1_macro:.4f}\n")
    f.write(f" Cohen's Kappa: {cohen_kappa:.4f}\n")
    f.write(f" Matthews CC: {mcc:.4f}\n")
    if roc_auc:
        f.write(f" ROC-AUC: {roc_auc:.4f}\n")
    if logloss:
        f.write(f" Log Loss: {logloss:.4f}\n")
    if top3_acc:
        f.write(f" Top-3 Accuracy: {top3_acc:.4f}\n")
    if top5_acc:
        f.write(f" Top-5 Accuracy: {top5_acc:.4f}\n\n")

    f.write(f"CLASSIFICATION REPORT:\n")
    f.write(f"{report}\n\n")
    f.write(f"MOST CONFUSED PAIRS:\n")
    for pair in confused_pairs[:20]:
        f.write(f" {pair['true']} → {pair['pred']}: {pair['count']}\n")

logger.log(f"\n Detailed report saved to: {report_path}")

# 3D CONFUSION MATRIX VISUALIZATIONS
# Restore original cm (diagonal was modified for confusion analysis)
cm = confusion_matrix(y_test, y_pred)
n_classes = len(class_names)

# Create 3D Surface confusion matrix
logger.log(f"\n Creating 3D confusion matrix for {model_name}...")
fig_cm = make_subplots(
    rows=1, cols=2,
    specs=[[{"type": "surface"}, {"type": "surface"}]],
    subplot_titles=['Raw Confusion Matrix', 'Normalized Confusion Matrix'],

```



```

        horizontal_spacing=0.05
    )

    # 3D Surface for raw confusion matrix
    fig_cm.add_trace(
        go.Surface(
            z=cm,
            x=list(range(n_classes)),
            y=list(range(n_classes)),
            colorscale='Blues',
            showscale=True,
            colorbar=dict(title='Count', x=0.45, len=0.8),
            hovertemplate='True: %{y}<br>Pred: %{x}<br>Count: %{z}<extra></extra>',
            contours=dict(
                z=dict(show=True, usecolormap=True, highlightcolor="white", project_x=True)
            )
        ),
        row=1, col=1
    )

    # Normalized confusion matrix
    cm_normalized = cm.astype('float') / cm.sum(axis=1)[:, np.newaxis]
    cm_normalized = np.nan_to_num(cm_normalized)

    fig_cm.add_trace(
        go.Surface(
            z=cm_normalized * 100,
            x=list(range(n_classes)),
            y=list(range(n_classes)),
            colorscale='RdYlGn',
            showscale=True,
            colorbar=dict(title='%', x=1.0, len=0.8),
            hovertemplate='True: %{y}<br>Pred: %{x}<br>Rate: %{z:.1f}%<extra></extra>',
            contours=dict(
                z=dict(show=True, usecolormap=True, highlightcolor="white", project_x=True)
            )
        ),
        row=1, col=2
    )

    # Update both scenes
    for col in [1, 2]:
        fig_cm.update_scenes(
            dict(
                xaxis=dict(title='Predicted', tickvals=list(range(n_classes))),
                ticktext=class_names,
                yaxis=dict(title='True', tickvals=list(range(n_classes))),
                ticktext=class_names,
                zaxis=dict(title='Count' if col == 1 else 'Rate %'),
                camera=dict(eye=dict(x=1.5, y=-1.5, z=1.0))
            ),
            row=1, col=col
        )

    fig_cm.update_layout(
        title=dict(
            text=f' {model_name} - 3D Confusion Matrix (Rotate to explore!)',
            font=dict(size=18, color='#333'),
            x=0.5
        ),
        height=700,
        template='plotly_white'
    )

    cm_path = os.path.join(CONFIG['metrics_path'], f'{model_name}_confusion_matrix.png')
    save_and_show_plotly(fig_cm, cm_path, f'{model_name} 3D Confusion Matrix')

    # Return comprehensive metrics
    return {
        'accuracy': accuracy,
        'balanced_accuracy': balanced_acc,
        'precision': precision,
        'recall': recall,
        'f1_score': f1,
        'f1_macro': f1_macro,
    }

```

```
'cohen_kappa': cohen_kappa,  
'matthews_corrcoef': mcc,  
'roc_auc': roc_auc,  
'log_loss': logloss,  
'top3_accuracy': top3_acc,  
'top5_accuracy': top5_acc,  
'avg_confidence': avg_confidence,  
'avg_conf_correct': avg_conf_correct,  
'avg_conf_wrong': avg_conf_wrong  
}
```

16. ✓ Final Comparison and Main Pipeline

```
# FINAL COMPARISON
def plot_model_comparison(results):
    """Plot final model comparison with SOLID 3D bars"""
    logger.section("MODEL COMPARISON")

    # Filter valid results
    valid_results = {k: v for k, v in results.items() if v is not None}
    if not valid_results:
        logger.log("\n No valid results to compare")
        return

    # Create comparison dataframe
    df = pd.DataFrame(valid_results).T
    df.index.name = 'Model'
    logger.log("\n Final Results:")
    logger.log(df.to_string())

    # Find best model
    best_model = df['accuracy'].idxmax()
    best_acc = df.loc[best_model, 'accuracy']
    logger.log(f"\n Best Model: {best_model} ({best_acc:.4f})")

    # Save results
    results_path = os.path.join(CONFIG['metrics_path'], 'final_results.csv')
    df.to_csv(results_path)
    logger.log(f"\n Results saved to: {results_path}")

    # Create 3D model comparison visualization with SOLID bars
    models = list(valid_results.keys())
    n_models = len(models)
    metrics = ['accuracy', 'precision', 'recall', 'f1_score', 'cohen_kappa']

    fig = go.Figure()

    # Create SOLID 3D bars for each model and metric
    metric_colors = ['#2ecc71', '#3498db', '#e74c3c', '#9b59b6', '#f39c12']
    for m_idx, metric in enumerate(metrics):
        values = [v.get(metric, 0) or 0 for v in valid_results.values()]
        for i, (model, val) in enumerate(zip(models, values)):
            # Create solid 3D bar with proper position
            x_center = i + (m_idx - 2) * 0.18 # Spread metrics within model group
            bar_width = 0.15
            bar_depth = 0.4

            bar = create_3d_bar_mesh(
                x_center=x_center,
                y_center=0,
                z_height=val,
                bar_width=bar_width,
                bar_depth=bar_depth,
                color=metric_colors[m_idx],
                opacity=0.95,
                name=metric.replace('_', ' ').title() if i == 0 else '',
                hovertemplate=f'<b>{model}</b><br>{metric}: {val:.4f}<extra></extra>'
            )
            # Set legend group manually
            bar.update(showlegend=(i == 0), legendgroup=metric)
            fig.add_trace(bar)

    # Add text labels
    fig.add_trace(go.Scatter3d(
        x=list(range(n_models)),
        y=[0] * n_models,
        z=[1.08] * n_models,
        mode='text',
        text=models,
        textfont=dict(size=12, color='black', family='Arial Black'),
        hoverinfo='skip',
        showlegend=False
    ))

    fig.update_layout(
```

```

        title=dict(
            text=' 3D Model Comparison (Rotate to explore!)',
            font=dict(size=20, color='#333'),
            x=0.5
        ),
        scene=dict(
            xaxis=dict(title='Model', tickvals=list(range(n_models)), ticktext=models),
            yaxis=dict(title='', showticklabels=False, range=[-0.8, 0.8]),
            zaxis=dict(title='Score', range=[0, 1.15]),
            camera=dict(eye=dict(x=1.5, y=1.5, z=0.8)),
            bgcolor='rgba(250,250,250,0.9)'
        ),
        height=650,
        template='plotly_white',
        legend=dict(
            title=dict(text='Metrics'),
            yanchor="top",
            y=0.99,
            xanchor="left",
            x=1.02
        )
    )
)

```

```

comparison_path = os.path.join(CONFIG['metrics_path'], 'model_comparison.png')
save_and_show_plotly(fig, comparison_path, '3D Model Comparison')

```

MAIN PIPELINE

```

def main():
    """Main training pipeline
    FLOW:
    1. Collect and store all dataset information
    2. Process dataset with MediaPipe
    3. Collect all model information (architecture, params, status)
    4. Test existing models BEFORE training (baseline metrics)
    5. Train models (resume or from scratch)
    6. Evaluate trained models
    7. Compare results (baseline vs trained)
    """
    start_time = time.time()
    print("\n" + "=" * 80)
    print(" ASL ALPHABET CLASSIFICATION - UNIFIED TRAINING PIPELINE")
    print("=" * 80)
    print(f" Start time: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}")
    print(f" GPU Available: {HAS_GPU}")
    print(f" TensorFlow: {tf.__version__}")
    print("=" * 80 + "\n")

    # PHASE 1: DATASET INFO
    logger.section("PHASE 1: DATASET INFORMATION")
    logger.log("Collecting comprehensive dataset information...")
    metrics = analyze_dataset()
    if metrics['total_classes'] == 0:
        logger.log("\n No valid classes found. Please check CONFIG paths.")
        return

    # PHASE 2: MEDIAPIPE PROCESSING
    logger.section("PHASE 2: MEDIAPIPE PROCESSING")
    class_names = process_dataset_with_mediapipe()
    if len(class_names) == 0:
        logger.log("\n No classes after processing. Please check dataset.")
        return

    num_classes = len(class_names)

    # Save class names
    class_names_path = os.path.join(CONFIG['models_path'], 'class_names.json')
    with open(class_names_path, 'w') as f:
        json.dump(class_names, f, indent=2)
    logger.log(f"\n Class names saved to: {class_names_path}")

    # PHASE 3: MODEL INFO COLLECTION
    logger.section("PHASE 3: MODEL INFORMATION")
    logger.log("Collecting comprehensive model information...")
    model_info = collect_model_info(num_classes)

```

```

# PHASE 4: LOAD DATASET
logger.section("PHASE 4: LOADING DATASET")
train_paths, train_labels = load_dataset(CONFIG['cropped_train_path'], class_names)
test_paths, test_labels = load_dataset(CONFIG['cropped_test_path'], class_names)
logger.log(f"\n Training samples: {len(train_paths)}")
logger.log(f" Test samples: {len(test_paths)}")
logger.log(f" Classes: {num_classes}")

if len(train_paths) == 0:
    logger.log("\n No training data found.")
    return

# PHASE 5: BASELINE TESTING
if len(test_paths) > 0:
    logger.section("PHASE 5: PRE-TRAINING BASELINE TEST")
    logger.log("Testing existing models to get baseline metrics...")
    baseline_results = test_models_before_training(
        test_paths, test_labels, class_names, model_info
    )
else:
    logger.log("\n No test data available for baseline testing")
    baseline_results = {}

# PHASE 6: TRAIN/VAL SPLIT
logger.section("PHASE 6: PREPARING TRAINING DATA")

# Split train/val
try:
    train_paths, val_paths, train_labels, val_labels = train_test_split(
        train_paths, train_labels,
        test_size=CONFIG['val_split'],
        random_state=42,
        stratify=train_labels
    )
except Exception as e:
    logger.log(f"\n Stratified split failed: {e}. Using random split.")
    split_idx = int(len(train_paths) * (1 - CONFIG['val_split']))
    train_paths, val_paths = train_paths[:split_idx], train_paths[split_idx:]
    train_labels, val_labels = train_labels[:split_idx], train_labels[split_idx:]

logger.log(f"\n Training samples: {len(train_paths)}")
logger.log(f" Validation samples: {len(val_paths)}")
logger.log(f" Test samples: {len(test_paths)}")

# PHASE 7: INTELLIGENT TRAINING DECISIONS
logger.section("PHASE 7: INTELLIGENT TRAINING ANALYSIS")
results = {}

# INTELLIGENT TRAINING DECISION THRESHOLDS
SKIP_THRESHOLD = 0.98 # If accuracy >= 98%, skip training (model is excellent)
GOOD_THRESHOLD = 0.95 # If accuracy >= 95%, model is good but can be fine-tuned
MODERATE_THRESHOLD = 0.30 # If accuracy >= 30%, resume training
# Below 30% = retrain from scratch (model incompatible)

# Analyze models and create training plan
training_plan = {}
for model_name in CONFIG['train_models']:
    baseline = baseline_results.get(model_name, {})
    action, reason, details = analyze_model_decision(model_name, baseline)
    training_plan[model_name] = {
        'action': action,
        'reason': reason,
        'details': details,
        'baseline': baseline
    }

# Display training plan
display_intelligent_training_summary(training_plan)

# Train or skip models based on decisions
for model_name in CONFIG['train_models']:
    plan = training_plan[model_name]
    action = plan['action']

    if action == 'SKIP':

```

```

logger.log(f"\n SKIPPING {model_name} - Already excellent!")
baseline = plan['baseline']
# Use baseline results as final results
results[model_name] = {
    'accuracy': baseline.get('accuracy', 0),
    'balanced_accuracy': baseline.get('balanced_accuracy', 0),
    'precision': baseline.get('precision', 0),
    'recall': baseline.get('recall', 0),
    'f1_score': baseline.get('f1_score', 0),
    'f1_macro': baseline.get('f1_macro', 0),
    'cohen_kappa': baseline.get('cohen_kappa', 0),
    'matthews_corrcoef': baseline.get('matthews_corrcoef', 0),
    'roc_auc': baseline.get('roc_auc', None),
    'log_loss': baseline.get('log_loss', None),
    'top3_accuracy': baseline.get('top3_accuracy', None),
    'top5_accuracy': baseline.get('top5_accuracy', None),
    'avg_confidence': baseline.get('avg_confidence', 0),
    'skipped_training': True,
    'reason': 'Excellent baseline performance'
}
else:
    # Train the model
    try:
        model, history = train_model(
            model_name, train_paths, train_labels,
            val_paths, val_labels, num_classes, class_names,
            training_mode=action # Pass the intelligent decision
        )

        # Evaluate the trained model
        model_results = evaluate_model(
            model, model_name, test_paths, test_labels, class_names
        )
        results[model_name] = model_results

        # Clear memory
        del model
        keras.backend.clear_session()
        gc.collect()
        time.sleep(3)
    except Exception as e:
        logger.log(f"\n {model_name} failed: {e}")
        import traceback
        logger.log(traceback.format_exc())
        results[model_name] = None

# PHASE 8: FINAL COMPARISON
logger.section("PHASE 8: FINAL COMPARISON")
plot_model_comparison(results)

# SUMMARY
total_time = time.time() - start_time
logger.section("TRAINING COMPLETE")
logger.log(f"\n Total time: {total_time/60:.1f} minutes ({total_time/3600:.2f} hours)")
logger.log(f" Models saved to: {CONFIG['models_path']}")
logger.log(f" Metrics saved to: {CONFIG['metrics_path']}")
logger.log(f" Log saved to: {logger.log_file}")

# Print summary of all saved files
logger.log(f"\n OUTPUT FILES SUMMARY:")
logger.log(f"{'-' * 50}")
if os.path.exists(CONFIG['metrics_path']):
    for f in sorted(os.listdir(CONFIG['metrics_path'])):
        filepath = os.path.join(CONFIG['metrics_path'], f)
        size = os.path.getsize(filepath) / 1024 # KB
        logger.log(f" {f} ({size:.1f} KB)")

if os.path.exists(CONFIG['models_path']):
    logger.log(f"\n SAVED MODELS:")
    logger.log(f"{'-' * 50}")
    for f in sorted(os.listdir(CONFIG['models_path'])):
        if f.endswith('.keras'):
            filepath = os.path.join(CONFIG['models_path'], f)
            size = os.path.getsize(filepath) / (1024 * 1024) # MB
            logger.log(f" {f} ({size:.1f} MB)")

```



```

print("\n" + "=" * 80)
print(" TRAINING PIPELINE COMPLETE!")
print("=" * 80 + "\n")

# RUN
if __name__ == "__main__":
    main()
    filepath = os.path.join(CONFIG['models_path'], f)
    size = os.path.getsize(filepath) / (1024 * 1024) # MB
    logger.log(f" {f} ({size:.1f} MB)")

print("\n" + "=" * 80)
print(" TRAINING PIPELINE COMPLETE!")
print("=" * 80 + "\n")

# RUN
if __name__ == "__main__":
    main()
    logger.log(f"{'-' * 50}")
    if os.path.exists(CONFIG['metrics_path']):
        for f in sorted(os.listdir(CONFIG['metrics_path'])):
            filepath = os.path.join(CONFIG['metrics_path'], f)
            size = os.path.getsize(filepath) / 1024 # KB
            logger.log(f" {f} ({size:.1f} KB)")

    if os.path.exists(CONFIG['models_path']):
        logger.log(f"\n SAVED MODELS:")
        logger.log(f"{'-' * 50}")
        for f in sorted(os.listdir(CONFIG['models_path'])):
            if f.endswith('.keras'):
                filepath = os.path.join(CONFIG['models_path'], f)
                size = os.path.getsize(filepath) / (1024 * 1024) # MB
                logger.log(f" {f} ({size:.1f} MB)")

print("\n" + "=" * 80)
print(" TRAINING PIPELINE COMPLETE!")
print("=" * 80 + "\n")

# RUN
if __name__ == "__main__":
    main()

```